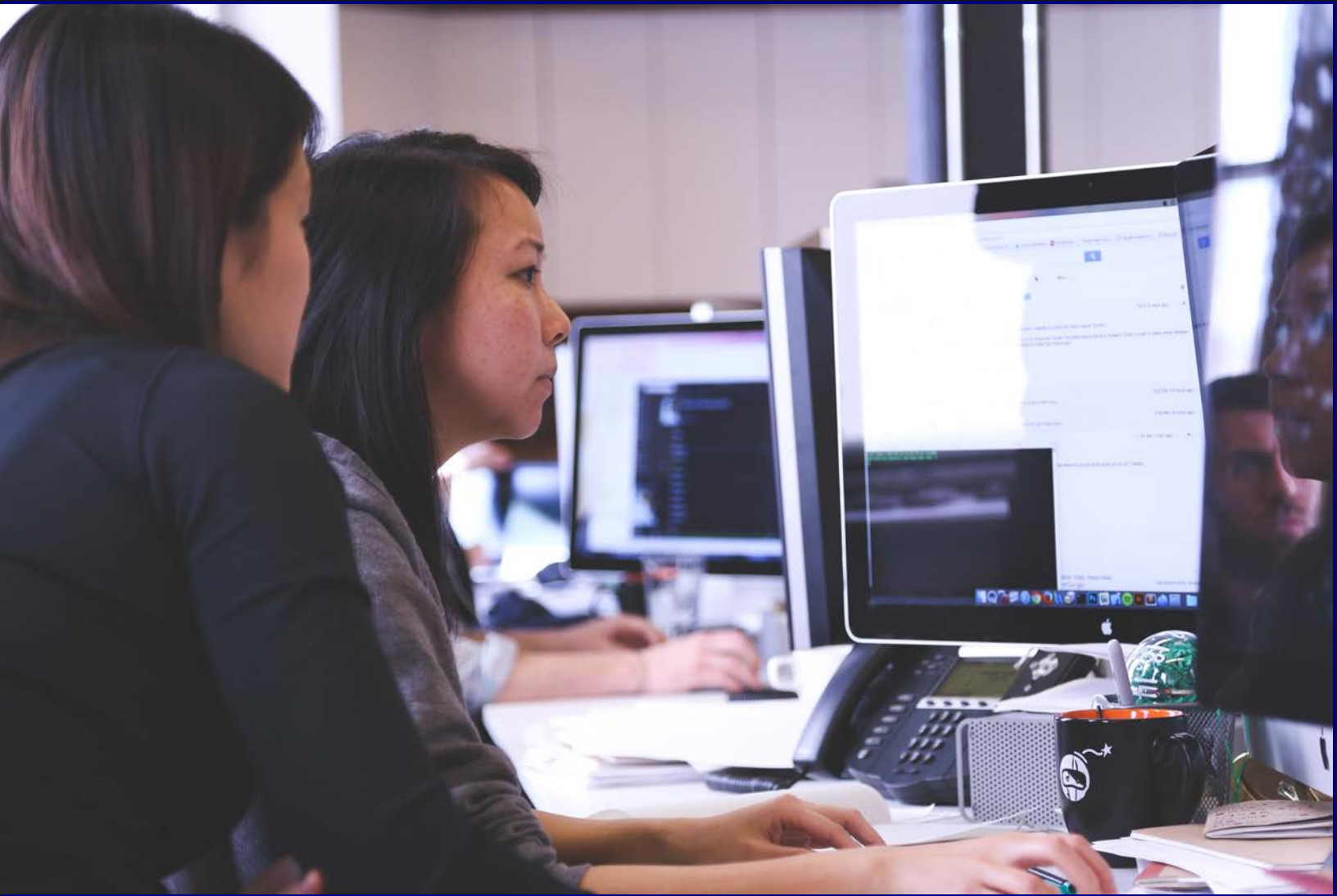




Effective Methods of Software Testing

Student Manual

Version 2.0

© Copyright 2019 by Rod Davison. All rights reserved.

No part of this work may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage or retrieval, for commercial purposes without prior written permission of Rod Davison. Creating and sharing copies for personal use is freely permitted as long as the copies are provided free of charge.

This material references, quote and cites published standards, best practices and material created by members of the software testing and quality communities, published authors as well as various professional and certification organizations. Any copyright claims by Rod Davison are intended to apply only to the material uniquely created by the author for this work and is not intended to apply to any of the work from other sources or authors that is cited, quoted or referenced in this material.

Original material in this document may be quoted provided that the quotations are both in accordance with the principles of fair use, properly cited and attributed to this source. Quotes of material which is the work of others should be properly attributed to the original sources and authors.

Any products that may be described in this document may be protected by one or more U.S. patents, foreign patents or pending patents or copyrights. Trademarked names may appear in this material. Rather than use a trademark symbol for every occurrence of a trademarked name, the name is used only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

Every effort has been made to ensure all quotations and images from published copyrighted sources have been cited in accordance with accepted academic practices of fair use or are used in compliance with their stated copyright licence. Any unattributed quotes or misused images are oversights that should be reported to the author so that the appropriate corrections can be made.

The information in this publication is distributed on an “as is” basis. While every precaution has been taken in the preparation of this material, the author assumed no responsibilities for errors or omissions, or for damages resulting from the use of the information contained herein.

Terms of Usage

All copies of this publication are licensed to only be used as course materials in the course "Effective Methods of Software Testing". This manual is not licensed for resale in any form or for use in any course or by any individual or organization except where authorized by Rod Davison.

Document Information

Course Title: Effective Methods of Software Testing
Author: Rod Davison
e-mail: rod@exgnosis.ca
Version: Student Manual 2.0
Release Date: March 1, 2019
ProTech ID: PT4047 DEL-120,287
Dates: April 1-3, 2019
Document ID: STQ101-02v2

"The primary objective of copyright is not 'to reward the labour of authors, but to promote the Progress of Science and useful Arts.' To this end, copyright assures authors the right to their original expression, but encourages others to build freely upon the ideas and information conveyed by a work. This result is neither unfair nor unfortunate. It is the means by which copyright advances the progress of science and art."

Justice Sandra Day O'Connor

Preface

This course has been developed for three basic groups of students:

1. Software testers who want a refresher on some basics of software testing, or to supplement their own self taught skills with some formal training, or even to just get up to speed on some of the new trends in the industry.
2. Non-testers who need a general understanding of software testing, either to help them work with techniques like test driven development or because they work with testers and need a basic understanding of what testers are doing, for example being technical lead on an Agile team.
3. Students who are just starting out on their testing careers and want to get "the big picture" before immersing themselves in the details of the processes and techniques of the profession.

This means that we will be covering a wide range of topics in the field, but because we can't do everything in a three day course, we won't go into level of theoretical and operational detail that the more advanced courses do. The result is a compromise that, hopefully, is interesting, useful and focuses on the main ideas and concepts of software testing.

The course has taken me a while to refine because I wanted to ensure that the requirements of each of my main three types of students are met – something I think that is possible to do in this course the way it is currently structured.

Of course, that means that not everything that every student wants is covered in the level of detail they might prefer, but as a compromise, I hope that this course does meet most of the needs of most of the students taking it.

For Software Testers

I get a lot of software testers in this course, and why they are enrolled, some of the common answers I get tend to be:

1. "I'm a self-taught and want to reality check what I am doing against the 'official' way of doing testing."
2. "I want to validate what I'm doing to ensure I'm doing it right."
3. "It's been a long time since I had any training and I just wanted to catch up on any new developments."
4. "I'm just looking for any tips or new ideas that I can incorporate into my work to make my job easier and my testing better."

I've responded to those general requirements by focusing on both the theoretical aspects of the profession, like test quality, while trying to show how the standard techniques protocols implement these ideas in a practical manner, while also emphasizing how this also makes testing more effective and efficient.

For Testing Newcomers

Software testing is a broad and complex field. It's easy for a new comer to get overwhelmed by the jargon, the formalism and the level of detail that is often required by the various types of testing; a challenge compounded by the fact that all the different forms of testing do things differently.

For the newcomers who are just starting out as testers, my goal is to have this course provide you with a high level survey of the field rooted in a solid understanding of the reasons and rationales for what we do as software testers. I hope that after this course, you will be able to use what you have learned to facilitate the learning process in the more advanced courses you will be taking.

For Non-Testers

One of my mentors once told me to study some basic accounting and business law no matter what sort of business I wanted to go into. When I protested that there was no need for me to do that because that was why we had lawyers and accountants, he said "You don't have to know enough accounting to be an accountant, or enough law to be a lawyer; but you had better know enough about each so that you can work with your lawyer and accountant, and can understand what they are telling you."

Most of the non-testers in this course are here to learn just enough testing so that they can work with testers effectively. For example, developers who are using test driven development need to work with testers to ensure their test cases are good enough to support their development. Modern Agile techniques in particular, including DevOps, require this closer collaboration between developer and tester, but it does require that each party know enough about what the other is doing so that they can actually work together productively. For these students, hopefully the course provides a basis for understanding the terminology and concepts that are important in understanding what their testing colleagues are doing and talking about.

The other sort of non-tester that is often in this course is responsible for managing a team or project that includes testers, and who is responsible for developing a overall project plan that incorporates the various testing activities. For these students, the course should provide the big picture of how testing works and how it interacts with the other development and projects dimensions so that you can get the maximum benefits of having testers on your team.



Exercises and Labs

This course is mostly lecture and discussion which should keep it very interactive. There are some lab exercises at various points in the course, but these are intended more to illustrate the ideas presented by engaging in a practical problem rather than being provided to master a particular technique. Not all of the exercises may necessarily be done during class, but I have ensured that there are enough labs, discussion questions and topics so that you can both select as a class what is of most interest to you, and also have some left over as for your own practice and follow-up.

Certification

There are a number of software testing certifications available. This course is not intended as a certification preparation course for two reasons:

1. It doesn't cover a lot of the certification topics to the level of detail required by the certifications; and
2. More importantly, it is intended to be a practical course and, rather than being focused on passing a test, deals with the concepts and ideas that support the student's understanding of the subject matter.

Currently, there are more advanced specialized courses under development that will be suitable for those who are looking for certification or compliance with official standards. Those courses should, at a minimum, cover the required topics but also maintain the focus of "what can my students learn practically that will make them better testers, even if it's not part of a certification or standard?"

However, I have tried to ensure that the material in this class is consistent with the ISTQB certification syllabus as well as the established ISO, IEEE and other industry standards. However, the different standards, experts and organizations do not always agree on everything, so I have tried to present the material in a balanced manner pointing out where these disagreements take place, over what topics and why there are disagreements. Of course, any errors in presenting these different viewpoints is entirely mine.

Follow-up

I am more than happy to provide follow-up for any student about anything covered in the course. Please email me at support@exgnosis.ca. Thank you.

Introduction to Software Testing

Module One

*It's scary to think that the infrastructure of the industrialized world
is increasingly based on software like this*

Stephen Wolfe referring to AutoCAD rev. 12

*If history repeats itself, and the unexpected always happens,
how incapable must Man be of learning from experience.*

George Bernard Shaw

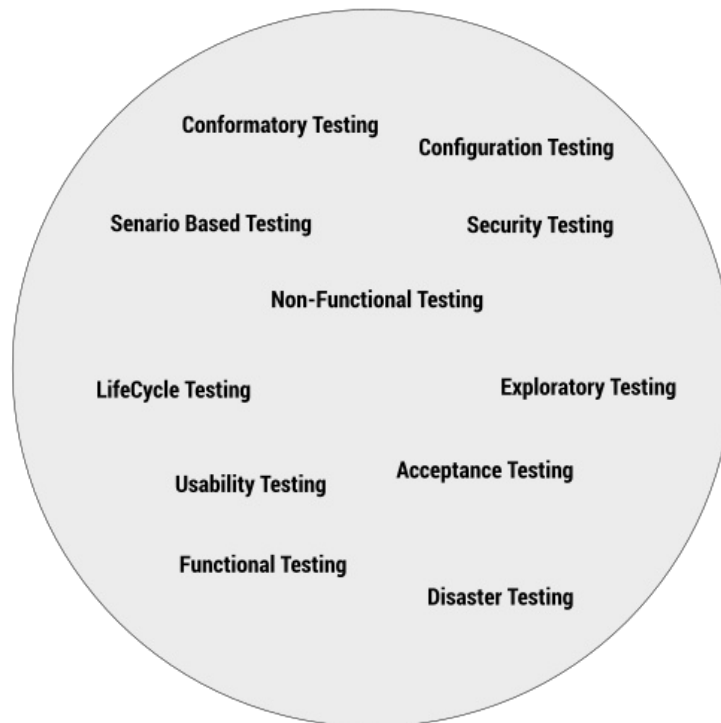
Those who cannot remember the past are condemned to repeat it.

George Santayana



1.1 Defining Software Testing

The standard approach in most software testing courses is to start by defining what we mean by software testing. However, there is no "official" or definitive "definition" that we can use as a starting point because the field of software testing encompasses a wide range of different activities and subject matter. All of these very different and diverse "things" are collectively mashed together into something called software testing.



For example, Glenford Myers in his 1979 book *"The Art of Software Testing"* says:

Testing is the process of executing a program or system with the intent of finding errors.

This is an excellent definition for functional or black box testing, but doesn't really cover usability testing, exploratory testing or even non-functional testing. We could go through the exercise of listing all sorts of definitions of software testing and identify how each is deficient in some way, but that is not a productive activity. Instead, we will start with Bill Hetzel's definitions:

*Testing is any activity aimed at evaluating an attribute or capability of a program or system.
Testing is the measurement of software quality. (1984)*

Software Testing is a process consisting of all life cycle activities concerned with checking software and software-related work products. (1988 with Gelperin)

Part of the problem with the 1984 definition is that it is not clear what is meant by "software quality." For the purposes of this course, we will rewrite Hetzel's definitions as:

Software testing is measuring and evaluating properties or attributes of software or the development process used to produce it.

The selection of attributes or properties being tested is often what defines a particular type of testing; for example, measuring the throughput, loading and other performance characteristics of an application is called non-functional testing, while evaluating how easy or difficult users find the software to use falls under the subject of usability testing.

The core idea you should take away from this section is that all software testing is about measuring software or measuring the process we are using to develop it. The rest of the course will explore this fundamental idea in detail.

1.1.1 The Testing Profession

Software is not the only thing that gets tested in this world; testing is part of every profession and it's also an everyday activity that everyone engages in, both objectively and subjectively. For now, we will say that testing is objective when there is some external standard that we are using as our basis for measurement and evaluation.

Just a quick sampling of some objective testing scenarios:

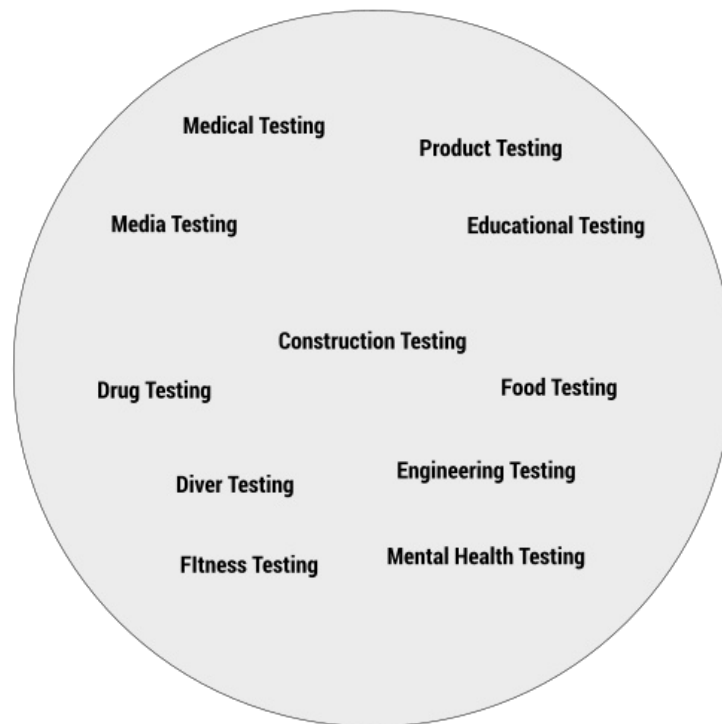
1. Restaurants are not allowed to operate by law unless they have been inspected by a health inspector and meet the official health standards.
2. New drugs are not allowed into the marketplace until they have been fully tested in controlled double-blind trials to show they actually provide benefits while minimizing potential risks to patients.
3. Companies engage in market research to test whether or not customers are happy with one of their existing products. The decision about whether or not a new product should be developed is made by testing how well it is likely to sell.
4. Before construction can start on a building, the plans and designs have to be evaluated and approved by an appropriate authority. Then, while the building is under construction, there are on going evaluations on all aspects of the construction.
5. Before anyone can get a software testing certification, they have to pass a test that measures their knowledge of the subject matter.

Subjective testing is where the tester is using their own experience or skill to test and evaluate something. For example:

1. Good cooks will taste food they are preparing to ensure the final result will be as expected.
2. A job interview provides an opportunity for a potential employer to "get a feel" for the person being interviewed and whether or not they would be a fit for the job and organization.



3. A jury has to decide whether or not, based on the evidence presented, that a defendant is guilty "beyond a reasonable doubt."
4. Most car buyers will make a final decision about buying a car after a test drive where they get an opportunity to decide if they like the car.
5. A doctor often decides when to discharge a patient from a hospital based on their evaluation of the patient's test results.



One important fact that should jump out at you is that "subjective testing" does not mean that the testing is trivial or unimportant. Medical decisions and jury judgments are not minor or trivial, but are also very difficult, if not impossible, to turn into some form of objective testing. Later in the course, we will look at the issue of developing objective test criteria.

1.1.2 General Testing and Contextual Testing

There already exists a large and well-established body of knowledge, experience and techniques that have been developed by testers, whether they come from medicine, engineering or the social sciences. We can divide this body of knowledge into two layers.

Contextual Testing

The idea of contextual testing is expressed in the ISTQB as

The Testing Principle - Testing is context dependent. Testing is done differently in different contexts.

This is true in the larger testing community: how we test cars is different than how we do medical testing – the type of testing we do depends on both on what is being tested and why we are testing it.

We also see contextual testing within software testing; usability testing uses very different techniques than security testing, and both use very different techniques than functional testing.

General Testing

However, even though the specific ways we test something are contextual, there are common, universal principles of testing that are shared by all types of testing. While a civil engineer may use very different sorts of testing methods than a medical researcher, they would agree on a number of basic concepts about how tests should be designed, how to plan testing activities, identifying and correcting errors in test design and test execution and the general professional approach that should be used in all testing activities.

As software testing has matured, it has borrowed and adapted a range of techniques from other fields of testing; for example, functional testing comes from applying engineering testing methodologies to software. Similarly, many usability testing techniques have their origins in both market research and social sciences research methodologies

The influence of other forms of testing on the field of software testing shows up in common ideas, techniques and methodologies, but the one thing to keep in mind, even if it is not obvious, is that there are practical and pragmatic reasons why testers in all fields do similar types of things and think about testing in similar ways.

We will return to these topics later in the course.



1.2 Why We Test

A standard part of every software testing course or book is a list of reasons why we engage in software testing. It is left as a discussion exercise to come up with a number of different reasons that would be appropriate for your specific industry.

Any list of reasons we do testing are all instances of four different functions of testing which are true across all forms of testing. In general terms, these are:

1. **Validation:** Testing to validate that we are doing or building the right thing.
2. **Verification:** Testing to verify that we are building things correctly.
3. **Quality:** Testing to ensure that we are building things to the right level of quality.
4. **Risk:** Testing to identify potential business, project and product risk.

Before we get into how these are translated into software testing, it's useful to take a look at how these ideas might translate into testing methods in the kitchen of a Michelin four star restaurant in preparing a meal for a customer.

Validation Testing

Suppose that a customer decides to order a lovely rare Rib Eye steak with baby carrots and sauteed mushrooms. The waiter writes down the order and then engages in a form of validation testing by reading back the order to the customer. However, our waiter was distracted and wrote down "sauteed onions" instead of "sauteed mushroom."

This is validation testing. The waiter is "testing" whether or not he wrote down the correct meal. If not done any validation testing, then the customer would have gotten the wrong meal. We probably would see similar testing as the chef repeats back the order to the waiter to ensure the kitchen is going to deliver the correct meal.

When working with software, the primary goal of validation testing is to ensure that the product being developed meets the user requirements and meets the underlying need that motivated the development of the software. Traditionally this called acceptance testing, where the end user validates that the software does in fact meet their requirements but validation testing appears in a number of different modalities of software testing.

Verification Testing

Back to our restaurant. The correct meal has been confirmed and the various tasks delegated out to different kitchen staff to start preparing the meal.

The preparation of the mushrooms is assigned to one of the junior chefs who, unfortunately, doesn't really know how to saute mushrooms correctly. The sous chef checks on the work of the junior chef and discovers that the mushrooms are being prepared incorrectly and either re-assigns the task to another chef who can do the job right, they may show the junior chef the correct way to saute mushrooms.

This is verification testing. The right dish was being prepared, but it was not being done correctly; the verification testing caught the mistake which allowed for a correction to be made so that the kitchen didn't serve a meal that was not correctly prepared.

When working with software, this sort of testing ensures that the software does what it is supposed to, and doesn't do anything it shouldn't. This type of testing usually focuses on whether or not the software functionality is correct. However, this kind of testing can also be applied to the software development process itself, something we will explore in the some later modules.

Quality Testing

However, our restaurant might have another problem: if the meat being used has been left over from last week and isn't really that fresh, it could affect the quality of the final result and produce a steak which just tastes "off." However, because of the routine kitchen testing being done by the chef to ensure quality standards, these things don't happen. The chef inspects all the meat in the cooler at the start of each day to ensure it meets the quality levels of the chef; if not, it's donated to a local charity.

This is quality testing because it has nothing to do with cooking the right meal or cooking a meal right, but is more about meeting quality expectations of the customers. If this were Joe's Discount Diner, the customers might be willing to accept a steak that's not quite to quality, but this is a four star restaurant where customers expect the meat being served is of the highest quality.

For software, as we will see, there are a number of quality dimensions that we often want to evaluate ranging from ease of use to how much value the customer feels they get for the money spent on the software.

Quality testing is always inexorably intertwined with software quality issues, but can be thought of as the process of measuring the quality of some aspect of the software to some quality standard. Quality testing is often implemented across different kinds of testing including acceptance testing, usability testing, and functional testing.

Exploratory Testing

What if our restaurant gets tainted meat from one of its suppliers? Or if the power keeps going out and meals are ruined, or if the restaurant starts to become the favourite establishment of local Mafioso to the point where other customers stay away? There are many things that can go wrong that have nothing to do with validation, verification or quality.



This is focus of exploratory testing which asks the "what if?" questions to find out where the risk is that is often is overlooked, especially when the risk is not immediately obvious. As Douglas Adams who wrote *The Hitchhiker's Guide to the Galaxy* observed:

The major difference between a thing that might go wrong and a thing that cannot possibly go wrong is that when a thing that cannot possibly go wrong goes wrong it usually turns out to be impossible to get at or repair.

Exploratory testing is a fascinating new kind of software testing that brings together a number of ideas about working with risk that have been common across many different testing methods. Software testers have been doing exploratory testing for years on an ad hoc basis, but it is only recently that it has moved from a sort of informal way of managing risk to something that is more structured and accessible to new testers.

The meme below which has been circulating on reddit, sums up the difference between regular testing and risk testing in a delightfully irreverent manner.



Brenan Keller
@brenankeller

[illegible]

First real customer walks in and asks where the bathroom is. The bar bursts into flames, killing everyone.

Security Testing as Specialized Risk Testing

Over the last decade, the rise in hacking and sophisticated attacks on computer systems by malicious actors has created a whole new area of testing is now considered a new discipline within software testing. One of the reasons for separating out security testing from regular risk testing is that risk testing looks mostly at typical risks arise as a result of the systems operations, while security testing looks at risks that arise from planned and coordinated attacks by a third party.

1.3 Software Testing Evolution

Software Testing Evolution refers to three distinct but related types of "evolutions" at different scales:

1. **Historical Evolution:** How the profession of software testing has changed over the years as it has matured and incorporated testing concepts and methodologies from other areas of testing.
2. **Corporate Evolution:** How an organization becomes better at testing as it moves through a series of testing maturity levels. This process is described in a framework called the Testing Maturity Model and will be discussed in a later module.
3. **Personal Maturity:** How an individual tester matures in his profession as he becomes better at his craft. We will take an initial look at Boris Beizer's "Levels of a Tester's Development" in this module and return to it later in the Agile Testing module.

Whether it is corporate, historical or individual, all of these evolutionary models demonstrate a similar progression. It is left to a class discussion to try and propose some ideas as to why this is so.

1.3.1 Historical Evolution of Software Testing

These eras are of a merged result from classical sources like Gelperin and Hetzel's 1988 article "The Growth of Software Testing", Beizer's own observation and the insights of more modern Agile testers like Lisa Crispin and Janet Gregory. All dates are very approximate with lots of overlap since these represent gradual changes over time.

1. **1950s – mid 1970s:** Debugging oriented period.
2. **Mid 1970s – early 1980s:** The introduction of engineering style specifications and methods for testing to see if software meets the specification.
3. **Early 1980s – early 1990s:** Risk analysis and robustness concerns emphasize testing to ensure that software doesn't break.
4. **Early 1990s to mid 2000s:** Introduction of well defined software development processes lead to "testing throughout the development cycle" and applying QA principles to reduce the overall risk in development.
5. **Mid 2000s to present:** Agile and DevOps continuous development processes require constant collaboration between testers, analysts and developers.



The Testing as Debugging Period

As Beizer notes:

Stage 1 thinking was appropriate to an environment characterized by expensive and scarce computing resources, low cost (relative to hardware) software, lone programmers, small projects and throw away software.

This was the era of what I called the “code cowboys.” Programming was considered an art form that you either had a talent for or didn’t. Code was viewed as an inspired creation like a musical composition or a piece of art, and when it broke, you looked through its structure to find the part that was broken because it would just “look broken.”

In spite of that mindset, the debugging process was often quite rigorous and systematic. Test cases were generated by a “what would happen if we tried..” approach that tested the code and logic structure of the software. While many authors give the impression that this was a rather primitive and chaotic process; in reality, a lot of time debugging took place at the machine code level and required a great deal of sophisticated analysis and skill.

Many of the techniques from this era have been refined into what we call white box or structural testing today.

1.3.1.2 The “Make it Work” Period

In the 1970s, the idea of treating programming and application development as an engineering activity led to the incorporation of the engineering model of testing – conformance to specification – into software testing.

These engineering practices; the approach of “read the spec, figure out what the software should do based on the spec and then try it to see if the application does what it’s supposed to do” found a home in engineering based organizations. Most of the software development at the time was taking place either in large corporations, government agencies or universities because they were the only ones who could afford computers. Remember, the only computers available were mainframes like the IBM 360 and mini-computers like the PDP-11. The programmers that worked on these systems tended to be engineers so they naturally started to rely on engineering type testing methods.

The engineering approach was popularized by Glenford Myers seminal book in 1979 “The Art of Software Testing” which was based to a great extent on his work at IBM where he and his colleagues were attempting to formally define how to apply engineering type testing to software development.

However, while testing for correctness now started to be an important aspect of evaluating software, robustness testing (“What happens if we provide bad input?”) was considered to be unnecessary. The basic philosophy of the time was “garbage in, garbage out,” which reflected the thinking of the time; it was assumed it was the responsibility of

the user to ensure correct inputs were provided in the same way that a driver of the car was responsible for not driving the car into a wall. The overall approach was nicely summed up by Doug Gwyn in this comment he made about Unix:

Unix was not designed to stop people from doing stupid things, because that would also stop them from doing clever things.

This was not an unreasonable view since writing and executing programs was done exclusively by a small cadre of IT specialists who would enforce the rules about what could and could not be supplied as data to a program.

For example, while I was working as a junior programmer in that era, one of our IT staff complained to the lead developer that when zero was entered for a person's height, the application crashed. The developer's reply was "Of course it did, you aren't supposed to enter 0 for height so don't do it again!" Problem solved.

The "Let's Break It" Period

Eventually robustness started to become increasingly important as the economic and other costs of systems that failed or were not able to respond to invalid or incorrect inputs started to become a major problem. As more software began to be used a larger and more diverse base of customers, failures started to become more common and customers started growing increasingly hostile. One of my colleagues at IBM at the time used to quote Charles Babbage in describing his customers:

I have been asked, 'If you put into the machine wrong figures, will the right answers come out?' I am not able rightly to apprehend the kind of confusion of ideas that could provoke such a question.

But the widespread use of software to automate business functions had started the era of mission critical software – a situation where an organization starts to become so dependent on its software for its day to day operations that it cannot function if the software fails. For example, all it takes is one pay period when the payroll system doesn't function, or one day when all the companies communications go down to start making robustness a very desirable quality for software to have.

In addition to positive testing – making sure valid inputs produce the right results – the need for robustness started to put emphasis to be put on negative testing – making sure the system could handle bad input or invalid conditions and still function correctly.

Out of this type of specification based positive and negative testing evolved what we call black box or functional testing.

The Process Era

For a long time, the standard model being used was that developers built software and testers tested the result. However in the late 1990s, techniques and methods in the areas of industrial process improvement, total quality management and quality assurance



started to be applied to software – a trend that got a massive boost with the Software Engineering Institute releasing the software process development maturity (the famous CMM levels).

There was also a shift in how testers looked at the work they did – they found their testing load at the end of a development cycle could be reduced drastically by engaging in testing throughout the development process and acting in more of a Quality Assurance role by trying things in terms of defect prevention rather than only in terms of defect detection. For example, by ensuring that specifications were good enough to write code to, testers could help reduce errors and resulting rework time by up to 80% in some cases.

Inspired by the impact of the quality movement in other fields, software testers started to look at the role they could play in continuously helping to make the software development process more efficient and effective. Many of the ideas we have about conducting reviews and inspection of various types come out of this approach, as well as practices like “good enough quality” and risk based testing, as well as the increasing emphasis on process improvement.

The Collaborative Era

With the rise of Agile in the late 2000s and the introduction of collaborative processes like DevOps and Acceptance Test Driven Development (ATDD), the tester has become embedded in the production process. Unlike the previous levels where the testers were testing what was produced at different points during the development process, the tester is now part of the development process and tests the process itself in addition to the actual software.

This new approach to testing, which I will refer to as confirmatory testing in this course, is where tests are developed early in the development process in collaboration with stakeholders, then the tests are used continuously during the development process to guide development by both validating and verifying the code and design as they are being developed.

This collaborative approach is now generally referred to as Agile Testing, and will be covered in a later module.

1.3.2 Beizer's Levels

In the 1980s, Boris Beizer in his book *Software Testing Techniques*, commented that as a software tester got better at their profession, the way they approached testing evolved through a series of stages. Aside from renumbering to start at 1 instead of 0, the stages as he originally presented them are:

Phase 1: There is no difference between testing and debugging. Other than in support of debugging, testing has no purpose.

Phase 2: The purpose of testing is to show that software works.

Phase 3: The purpose of testing is to show where software doesn't work.

Phase 4: The purpose of testing is not to show anything, but to reduce the risk of not working to an acceptable value.

Phase 5: Testing is not an activity. It is a mental discipline that results in low-risk software without much testing effort.

There is a clear correlation between Beizer's levels and the historical development of software testing as a profession, and we will see the same sort of correlation with the Testing Maturity Model later in the course. The possible explanations for this congruence are interesting to speculate on but outside the scope of this course and are left as a discussion exercise.

One possible explanation to consider though is that companies were limited in their testing maturity by the state of development of the profession as a whole, and individuals are limited in their maturity by the maturity of the corporate environment they work in.



Good Testing, Bad Testing

Module Two

*It is not enough to do your best: you must know what to do, and
THEN do your best.*

W. Edwards Demming

*It is not a test that finds a bug but it is a human that finds a bug and
a test plays a role in helping the human find it.*

Pradeep Soundararajan

*Correctness is clearly the prime quality. If a system does not do
what it is supposed to do, then everything else about it matters
little.*

Bertrand Meyer



2.1 Effective or Good Testing

To understand what effective testing is, we will start this section by introducing some standard terminology. For the discussion that follows, we are considering testing as a process that produces some deliverable, which is the set of test results or reports.

1. **Effectiveness:** A process is effective when it produces the right result. No matter how well done something is or how excellent the quality of the result is, if it's the wrong result, the process is not effective.

If I order a steak in a restaurant and I get perfectly prepared pork chops, the restaurant process was not effective. It doesn't matter how good the pork chops are, they are not what was ordered. Effectiveness is doing the right thing.

2. **Efficiency:** A process is efficient when it is optimal in terms of how resources are used. One of the main symptoms of a process not being efficient is how much rework is done during the development. Rework is where previous work has to be done over because errors keep occurring and have to keep being corrected. Efficient processes make fewer errors, need less rework and get the job done faster and with less resource consumption.

If I order a rare steak in a restaurant and it comes to me well done, then I'll send it back and the kitchen will have to prepare a second stake because they didn't cook the first one right. Efficiency is doing things right.

How do these concepts relate to software testing? First, it's helpful to remember the reasons we test:

1. *Validation:* To ensure we are building the right software.
2. *Verification:* To ensure we are building the software right.
3. *Quality:* To ensure we are meeting the quality expectations of the client.
4. *Risk:* We understand and have mitigated business, project and product risks

Testing is effective when the measurements we make along each of these four dimensions are correct. We will define exactly what we mean by correct in the next section, but it essentially means that we can rely on the results of our testing to give us a true picture of how we are doing in our development process.

To avoid confusion with the concept of testing software for effectiveness, I will refer to effective testing as "correct" testing.

Efficient testing is easier to define. Our testing is efficient if do the right types of testing the right way while optimizing our use of resources available for testing. In the module on Test Management, we will look at test efficiency in much more detail.

2.2 Where Testing Fails

Poor testing or testing failures are always the result of not doing correct testing along one of the four dimension of testing that we have identified previously.

2.2.1 Failing in Validity Testing

Recall that validity testing is about ensuring we are building the right thing. In a more technical sense, building the right thing is about meeting the requirements by developing a specification that describes what is to be built that will meet the requirements.

One of the great computer scientists of the 20th century, Edsger Dijkstra commented:

The most important single aspect of software development is to be clear about what you are trying to build.

That suggests that valid software testing should involve testing to see if we do in fact know what we are building, because if we don't know what we are supposed to be building, then we will never know if we are building the right thing.

However, the data suggests we often don't know what we supposed to be building.

The Chaos Report

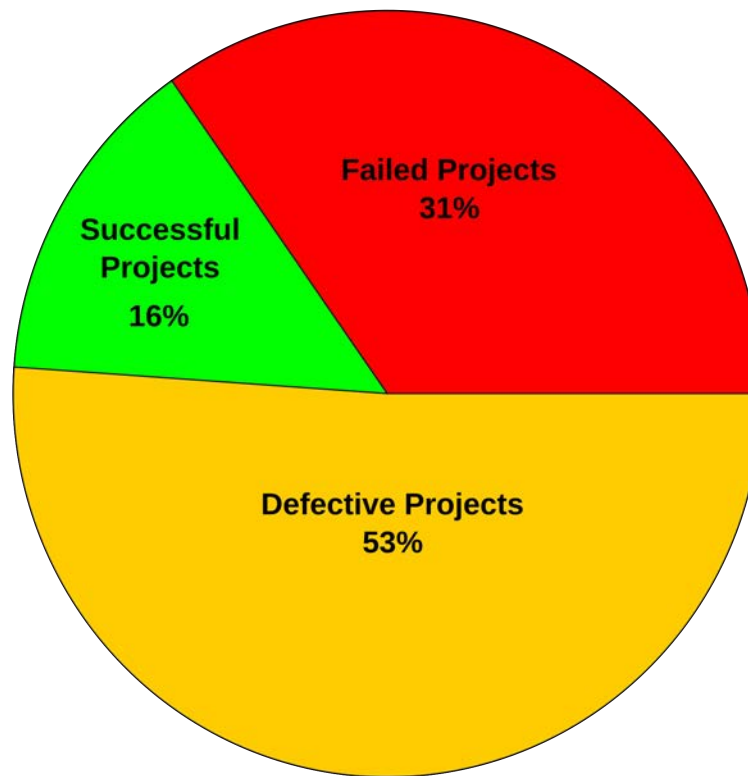
For more than a two decades, the Standish Group has produced an annual study called the Chaos Report, which is an empirical survey of IT project failures and the causes of those failures. The data used is drawn from different sized companies in a range of major industries like banking, pharmaceuticals, manufacturing and organizations at all levels of government. The original study in 1995 had a sample size was 365 organizations and 8,380 systems projects with subsequent studies maintaining this level of sampling. The data presented here is a aggregate of the various Chaos reports from 2002-2012.

The report defines three resolutions for technical projects:

1. *Resolution Type 1 – Successful Projects:* The project is completed on-time and on-budget, with all features and functions as initially specified. Acceptance tests all pass.
2. *Resolution Type 2 – Defective Projects:* The project is completed and operational but over-budget and/or late and/or is delivered fewer features and functions than originally specified, usually with acceptance test failures.
3. *Resolution Type 3 – Failed Projects:* The project is cancelled at some point during the development cycle.

Even more discouraging is that when only large organizations are considered the percentage of type 1 resolutions (project success) falls to 9% from 16%. Interestingly enough if we look at just Agile projects, the success rate increases dramatically – which





is probably attributable in part to the fact that Agile projects tend to use practices that address some of the causes of project failure.

The importance of the report for our purposes is the data that shows what factors were identified as contributing to each of the three resolutions by the people who were on the projects.

Successful Project Factors

The table on the next page lists the factors that were identified as being responsible or contributing to a project being successful. What we are interested in is how having good requirements and specification, which clarify exactly what is to be built, affect project outcomes.

The red italicized factors are those that are related to requirements and interactions with the users and stakeholders. Two of the top three items concern quality of requirements and getting the users involved. The fifth item, realistic expectations suggest that the developers had early input into the project to help the business understand what could be done within the time and budget constraints of the project.

Success Factors	%
<i>User Involvement</i>	16
Executive Management Support	14
<i>Clear Statement of Requirements</i>	13
<i>Proper Planning</i>	10
<i>Realistic Expectations</i>	8
Smaller Project Milestones	8
Competent Staff	7
Ownership	5
<i>Clear Vision and Objectives</i>	3
Hard Working and Focused Staff	2
Other	13

Defective Project Factors

The following table shows the factors that were identified as contributing to a project being challenged or defective. By definition a challenged project is one that was completed but either missed its budget targets, delivery date or did not perform as originally specified.

The top three reasons why projects were defective or challenged have to do with specifications, requirements and user involvement. The more chaotic the requirements and specification, the less the developers know what they should be building.

Defective Factors	%
<i>Lack of User Involvement</i>	13
<i>Incomplete Requirements & Specs</i>	12
<i>Changing Requirements & Specs</i>	12
Lack of Executive Support	8
Technology Incompetence	7
Lack of Resources	6
<i>Unrealistic Expectations</i>	6
<i>Unclear Objectives</i>	5
<i>Unrealistic Time Frames</i>	4
New Technology	4
Other	23



Failed Project Factors

The following table shows the factors that were identified as contributing projects being cancelled. Again, we see that requirements and specification issues and user involvement are right at the top.

Failure Factors	%
<i>Incomplete Requirements</i>	<i>13</i>
<i>Lack of User Involvement</i>	<i>12</i>
Lack of Resources	11
<i>Unrealistic Expectations</i>	<i>10</i>
<i>Changing Requirements & Specs</i>	<i>9</i>
Lack of Management Support	9
<i>Lack of Planning</i>	<i>8</i>
<i>Didn't Need It Any Longer</i>	<i>8</i>
Lack of IT Management	6
Technological Illiteracy	4
Other	10

The unrealistic expectations suggests that what the business wanted built was beyond the technical solutions that could be reasonably be delivered by the developers but for some reason that seems to never have been communicated to the end users.

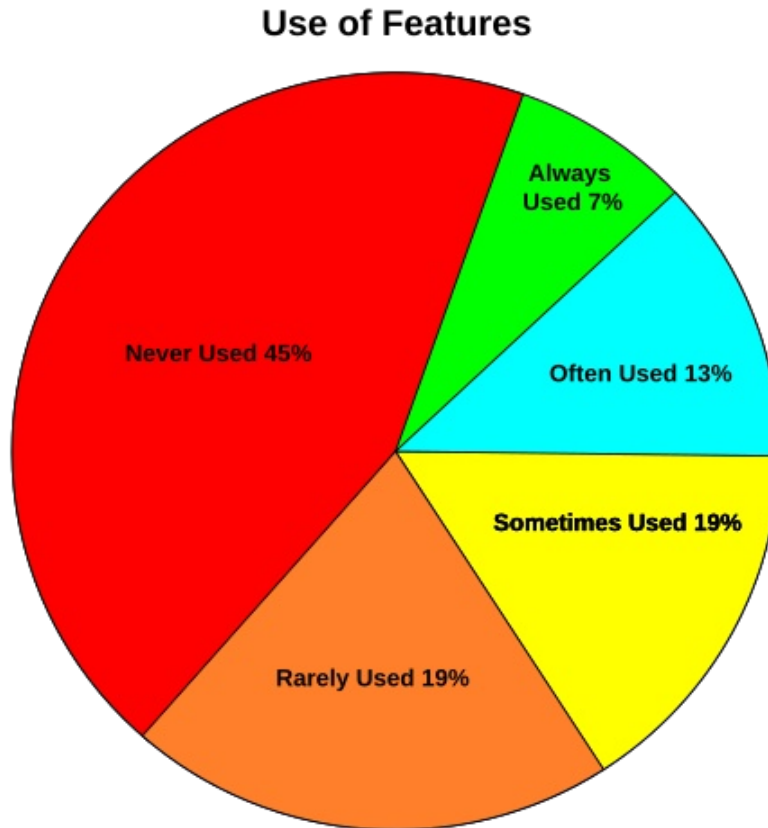
From a development perspective, if the developers are unclear on what they are supposed to be building, then projects tend not to be successful. However from a testing perspective, we also cannot do validity testing if we don't know what is supposed to be built. One of the themes we will be coming back to over and over in this course is:

If we can't write a test for something, then we can't write code for it either for the same reasons we can't write a test for it.

We will return to this whole idea of specifications and validity a number of times during the course.

Over Engineering

There is an old saying that the worst possible use of your time is to do something very well that never needed doing in the first place. When it comes to software, this is called over engineering. Essentially, it is adding features that add little to the value to the software.



The diagram above demonstrates that the Pareto principle seems to hold in software development. These numbers are from the Chaos report mentioned earlier. Only 20% of the delivered features are always or often used. Almost half the features are never used on average.

This means that 45% of the development effort is wasted. It's like cooking a meal with four dishes and the guests never eat two of them and only pick at the third while demolishing the fourth. If you are the cook, you would be asking yourself why even bother putting out that cost and effort into cooking dishes that no one is going to eat.

How does over engineering happen? Usually because the feature choice is made by developers working in the dark as to what the system is actually supposed to do and the requirements it needs to satisfy.

Over engineering is a combination of development ineffectiveness and inefficiency. Because we do not know what the end user really needs, we waste resources building things that never needed to be built at all.

In terms of development efficiency, if we focus on the most important features first and not even build features that contribute no value, we have increased the efficiency of the process without compromising effectiveness. Or as one sage noted:

The secret of success is to do first things first and other things not at all.



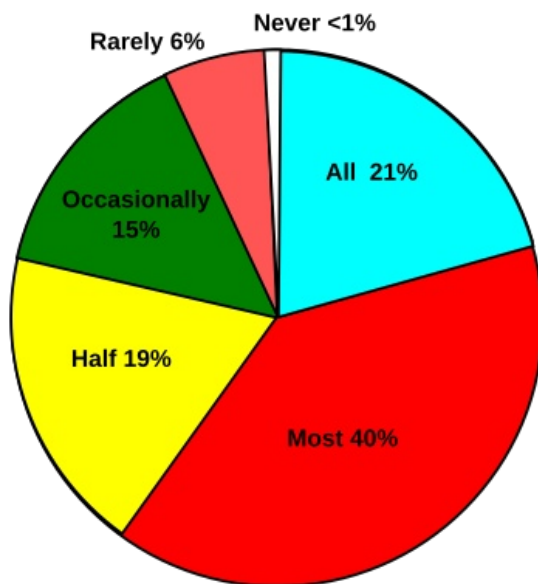
From a testing perspective, if we are doing good validity testing, then we should be able to prevent over engineering by identifying which features are not necessary based on the requirements and specifications.

2.2.2 Failing in Verification Testing

Remember that verification testing is about testing to ensure things are being done right. When they are not, we either get a defective or cancelled project or we have to do a lot of rework to correct errors that were made during development.

All of this so far can be a bit academic until we ask the really important question: how much of your time is taken up with fixing errors that could have been prevented?

A study done by Geneca entitled Doomed from the start asked that question in a survey of 600 IT professionals. The answer is enlightening.



Geneca Survey of IT Professionals:

"How much of your time do you spend on avoidable or preventable rework (eg. changing requirements, missing features)?"

Results are consistent with other studies.

Breaking it down, 61% of the IT professionals surveyed said they spent all or most of their time fixing things they felt were preventable. When we add in those who spend half their time fixing mistakes, then 80% of IT professionals spend at least half their time on is fixing mistakes that could have been prevented.

Now what if we could prevent those mistakes from taking place? Imagine how more efficient our software development process could be.

The question is what are the causes of those mistakes and can we realistically prevent them from happening?

Rework

When we are concerned with efficiency of a process, a concept that keeps recurring is that of rework, which we can define as work that has to be done over to correct errors of some kind.

If we tend to jump into a project before we understand exactly what we need to build or make, we are probably going to make errors. For example, if I am cooking a meal and start cooking before I do any planning, I might suddenly discover I am out of salt part way through the cooking process. However that means that I don't have enough salt for the recipe, and the meal winds up being inedible. So I throw the meal out and go to the store, which I should have done to start with, and recook the whole meal. A waste of time and money.

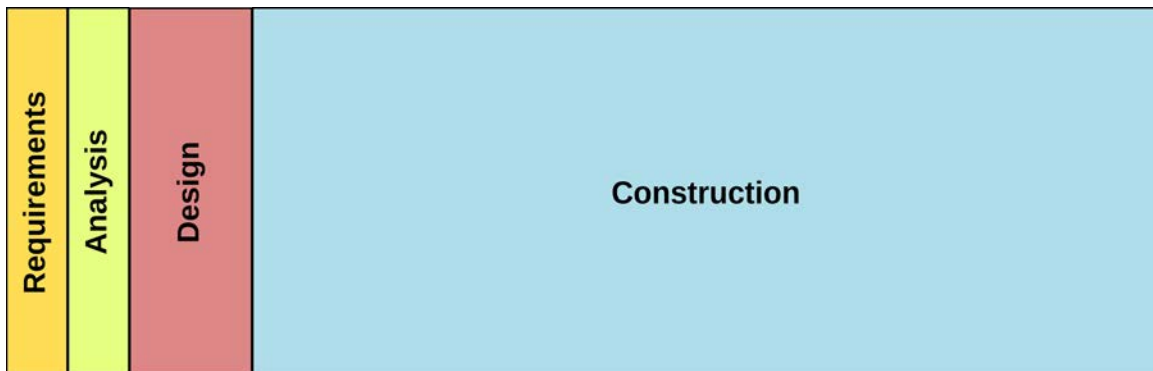
Exactly the same thing happens with in software. In fact, Ivar Jacobsen notes:

Requirements capture is the process of finding out, usually under difficult circumstances, what is to be built. In fact, it is so difficult that it is still not uncommon for project teams to start writing code (which is fairly easy) before they have firmed up just what the code is supposed to do (which is difficult).

Steve McConnell notes [Mccon96:71]:

Studies have found that reworking defective requirements, design, and code typically consumes 40 to 50 percent of the total cost of software development (Jones 1986). As a rule of thumb, every hour you spend on defect prevention will reduce your repair time from three to ten hours. In the worst case, reworking a software requirements problem once the software is in operation typically costs 50 to 200 times what it would take to rework the problem in the requirements stage (Boehm and Papaccio 1988).

It's easy to understand why. A 1-sentence requirement can expand into 5 pages of design diagrams, then into 500 lines of code, 15 pages of user documentation, and a few dozen test cases. It's cheaper to correct an error in that 1-sentence requirement at requirements time than it is after design, code, user documentation, and test cases have been written to it.



If you consider the four phases of an engineering project, we will get into a discussion off these phases later, the construction phase consumes probably about 80% of the total time and costs for the project. If we are trying to operate under the constraints of getting it right, on time and within budget, then we want to minimize the amount of rework in the



construction phase, which is expensive, by finding errors before and while we build, which reduces the amount of rework.

One of the main reasons we do efficiency testing is to ensure we are always doing things right and not wasting time and resources doing things incorrectly that we need to do over. As one common saying notes:

Why do we never have time to do it right, but always have time to do it over?

2.2.3 Poor Quality Testing

We will defer a discussion of quality issues until later in the course, but in a general sense we can see the results of poor quality testing when we deal with issues like:

1. The user interface to the application is difficult to use.
2. The information on a website is outdated or wrong or it is presented in a form that is hard to work with.
3. The functionality of the application is at odd with the actual business process that it automates.
4. The application is frustrating for users or the spend more time trying to get their work done with the application than without it.
5. The application keeps crashing or is taking unacceptably long times to respond.

As this list suggests, testing for quality cuts across a number of different types of testing like functional testing, usability testing and non-functional testing. In fact, we will find the issues of quality permeate almost everything we do in testing. More on that later.

2.2.4 Failures in Risk Testing

I'm updating this course shortly after most airlines have grounded the Boeing 737 MAX-8 after the Ethiopian Air Line crash. At the time I'm writing this, the cause seems to be a software issue in the autopilot that was known, but the deployment a fix was delayed because political and corporate issues. This is a product risk that was not apparently adequately tested and, as a result, people have died and there are suggestions that this will have a devastating impact on Boeing itself.

We can find examples of the results of failures of risk testing in many engineering and other journals, but to illustrate how difficult risk testing can be, this section will close with one of the most studied cases of risk failure testing.

The Therac-25 Case Study

This unfortunate episode of both software failure and testing failures has been one of the most studied software failures in history. As described in Wikipedia:

The Therac-25 was a radiation therapy machine... involved in at least six accidents between 1985 and 1987, in which patients were given massive overdoses of radiation. Because of concurrent programming errors, it sometimes gave its patients radiation doses that were hundreds of times greater than normal, resulting in death or serious injury.

One of the findings of investigators was that these while these errors were the result of poor engineering practices, poor testing allowed these errors slip through into the delivered production models. Some of the findings of the various investigations:

1. The software was designed so that it was realistically impossible to test it in a clean automated way.
2. The code was not independently reviewed or tested.
3. The design of the software was not examined in determining how it might fail.
4. There was poor evaluation of how the error system worked: "The system noticed that something was wrong and halted the X-ray beam, but merely displayed the word "MALFUNCTION" followed by a number from 1 to 64. The user manual did not explain or even address the error codes, so the operator pressed the P key to override the warning and proceed anyway."
5. There was no configuration testing of the Therac-25 with the various combinations of software and hardware until it was assembled at the hospital.

Point one suggests that validation and verification testing could not be done easily if at all. Point two suggests that while other test options, in this case code reviews, could have been done, there was a failure in the test planning process so that these never actually happened.

Point three above is an example where testing for risk did not occur. Point four suggests that the error reporting system was of poor quality, in this case being very difficult to use, which suggest a failure in testing for quality.

Failures of the Therac-25 resulted in a number of deaths that could have been avoided if the testing of the system had been done consistently with what are considered the professional best practices for software testing.

2.2.5 Impacts of Poor Testing

There are many cases similar to the Therac-25 case where missed errors have resulting in the failure of software systems that have significant negative impacts and consequences. For example, airline computer systems regularly cause air travel chaos, crashes of airplanes due to software errors, bank systems being unavailable causing economic havoc and many more situations.



Recall from a previous module that as we get more “mature” in our mental life as a tester, we go from running tests to thinking in terms of minimizing risk. The testing profession, not just software testers but all kinds of testers, understand that the role of their profession is to prevent buggy or poor quality products being used where they can have catastrophic outcomes. Hence the emphasis on “doing testing right.”

For many years, there was no significant impact if software failed, we could just do ctrl-alt-delete or reboot the system and carry on. However, software is now what we call “mission critical” in the words of Grady Booch who defines it this way:

[Industrial strength software is].. applications that exhibit a very rich set of behaviours, as, for example, as in reactive systems that drive or are driven by events in the physical world, and for which time and space are scarce resources; applications that maintain the integrity of hundreds if thousands of records of information while allowing concurrent updates and queries; and systems for the command and control of real-world entities, such as the routing of air or railway traffic. Software systems such as these tend to have a long life span, and over time, users come to depend on their proper functioning.

In this sense, there is a general understanding in the testing profession that if software testers fail in doing good testing, then bad things can happen, including people dying. In the next section, we will address the issues of what exactly “good” testing is.

2.3 Good Testing, Bad Testing

The idea of correct or good testing was introduced in the previous sections, but what exactly is good testing? How do we know if our testing is correct?

Fortunately we have an answer to this that comes from decades of exploring this issue in the larger testing community.

Good testing has four characteristics:

1. **Validity:** We are actually testing what we claim we are testing.
2. **Accuracy:** Our test are correctly reporting the existence of defects.
3. **Reliability:** Our tests results are not influenced by extraneous factors.
4. **Comprehensiveness:** Our tests meet the required quality and test objectives.

In the rest of this section we will do deep dives into each of these in turn.

2.3.1 Test Validity

A test is valid when it actually measures what it is supposed to measure. This is about the most basic requirement a good test can have, but yet it is surprising how much testing is done in many different testing domains that is not actually valid.

There are a number of on-going debates in the testing community about the different ways that can be invalid, but for the purposes of our discussion, we will focus on only three.

1. **Content Validity:** Our tests do not have content validity when we are claiming to test something that can't actually be measured or tested.
2. **Construct Validity:** Our tests do not have construct validity when we think we are testing one thing, but we are in fact testing something else.
3. **Predictive Validity:** Our tests do not have predictive validity if predictions made on the basis of the tests are incorrect.

Content Validity

Content validity has always been a major concern in the social sciences. For example, it was assumed for many years that IQ tests measured intelligence, until it was pointed out that no one really knew what intelligence was. So how could we claim to quantify something that we can't even defined?

We start to run into the same problem in software testing when we try to test things like "ease of use" or "usability" or "user experience" – because we can ask the exactly the same question as was asked about "intelligence" – what exactly is a "user experience?"



Based on the discussion so far, we run into problems with content validity for two main reasons:

1. The thing we want to measure is not defined in a way that we can test it.
2. The thing we want to measure is not directly measurable, often because it represents some internal state of our user or other stakeholder.

One of the ways that testers deal with the first issue is to use something called an "operational definition" which provides a restricted definition that we can measure of some poorly defined content. This is a technique routinely used in science and law, but is also common in software development where it is called "ubiquitous language" or "domain specific language."

For example, in geology, the concept of "hardness" is not measurable, so an operational definition of hardness is used which the score of rock on the Mohs hardness scale. We will look at this operational technique in testing later in the course.

The way we often deal with the second issue is to use some sort of qualitative testing where, for example, users evaluate subjectively how usable or easy to use something is. The use of qualitative techniques will also be covered later in the course.

Construct Validity

Our testing does not have construct validity when our tests are measuring something other than what we think they are measuring.

For example, a polygraph device is called "a lie detector," but doesn't measure a person's truthfulness. Instead it measures certain physiological responses such as respiration, heart rate and galvanic skin response under the hypothesis that when people lie, they get nervous which is manifested in sweaty palms, rapid breathing and other effects.

However the hypothesis is flawed. Psychopaths do not get nervous lying and almost anyone can be trained to lie without showing signs of nervousness. On the other hand, people answering truthfully can be made nervous by other factors including intimidation by the questioner or just the experience of taking a polygraph test.

In fact, the standard advice given by experts is to not take a polygraph if you are innocent because you might fail for other reasons, but if you are guilty, take one because you might pass it. The polygraph is not a valid test of truthfulness.

Tests may be subtly invalid which means that when we look at the tests they appear to test some feature of the system but when we examine what is actually happening when we run the test, we find that the result of the test is due to some other aspect of the system and not the feature we thought we were testing, as in the following example.

Testers for the login feature developed a series of tests to ensure that the login feature would reject any user id that was not in the data base. A test case was developed that used a made up id and password. When the test passed it was assumed that the underlying feature was able to correctly identify that a user id was not in the credentials database.

However on closer examination of the output of the logs associated with the test runs, it was discovered that the made up test password violated the rules for what a well formed password should be, for example, the password has to contain a digit, a special character, and whatever else. The login actually failed because the illegal password caused the login attempt to be rejected before any attempt was made to look up the credentials in the database.

Testers assumed that they were testing the credentials lookup but in fact they were testing the password validity check. Realizing their mistake, the test password was changed and suddenly a test that had previously passed now failed. The original test as constructed was invalid.

Predictive Validity

Predictive validity concerns us when we are attempting to improve our test process. For now, we can think of tests having predictive validity if they accurately predict other test outcomes or other outcomes in general.

For example, can we use the number of defects identified in our tests to predict the number of defects that will occur in a production environment? If so, then we have predictive validity. However if our tests show very low defects but our clients report high numbers of defects in operational environments, our tests then fail to have predictive validity.

Ease of use and usability testing are particularly prone to predictive validity failures. For example, speech recognition system tests may have a 90% comprehension rate in our tests, but out in the real world, the background noise may cause the comprehension rate to fall to 20%. This predictive validity failure should make us reevaluate the testing process and, in this case, start testing in environments that more closely resemble the operational environment.

2.3.2 Test Accuracy

Consider the following matrix of all possible outcomes for running a single test. To keep it simple, we will assume that we are working with a single test intended to identify a particular class of faults. In testing terminology we say that the test passes when it does not detect a fault and that it fails when it detects the fault.

The two desirable cases occur when the test works exactly as it should: either there is a fault and our test fails telling us a fault exists, or the test passes because there is no fault.



	<i>Fault</i>	<i>No Fault</i>
<i>Test Passes</i>	False Negative	Good
<i>Test Fails</i>	Good	False Positive

False Negatives

A false negative occurs when there is a fault and the test passes or fails to detect the fault. The effects of a false negative can be catastrophic. For example:

1. A bridge passes its structural inspection but one of the tests passed instead of showing a structural failure. Several weeks later the bridge collapses during rush hour killing a number of commuters.
2. A standard AIDS test does not pick up a new mutation to the HIV virus. A carrier of the new mutation is declared to be HIV free and proceeds to spread the new mutation within the community.
3. A carbon monoxide detector fails to detect toxic levels of CO in the room causing several people to die in their sleep.

A lot of the activity that we do in software testing is trying to design tests that minimize the possible false negatives that could possibly occur. We tend to be more aware of false negatives because, as examples tend to indicate, false negatives often result in potential catastrophic failures.

False Positive

A false positive takes place when our test fails, meaning it indicates a fault exists but in reality there is no underlying fault. The consequences of a false positive are not nearly as dramatic as a false negative (no bridge ever collapsed because our test said there was a failure that didn't really exist), however the consequences can be just as problematic.

For example, the standard \$2 roadside drug detection test used by many US law enforcement agencies has, according to investigative reporting by ProPublica:

[The test] uses a single tube of a chemical called cobalt thiocyanate, which turns blue when it is exposed to cocaine. But cobalt thiocyanate also turns blue when it is exposed to more than 80 other compounds, including methadone, certain acne medications and several common household cleaners.

In other examinations of the test kit, there were several instances in which the test indicated the presence of cocaine but no sample had even been added to the tube. The problem is that these false positives have led to a number of innocent people being sent to jail, lives destroyed and, in the longer term, a massive potential liability for various governments who use the tests from those who have legal claims from having been wrongly convicted.

False positives can be costly also from an operational perspective. The false identification of an error can lead to expensive and time consuming debugging or investigative procedures that are essentially a waste of time – looking for errors that don't exist. These costs are compounded if we have to do recall or expensive upgrade process to try and fix something that was never broken.

However most of the damage that a false positive does occurs when it results in a business decision that itself has negative consequences. False positives can result from either a flawed test or the way in which the test is run. Consider this report from CBS news.

Four years ago, a Houston teenager named Josiah Sutton was convicted of rape and sentenced to 25 years in prison. He was convicted primarily on the basis of DNA tests performed by the Houston police department's crime lab.

Anna Werner and David Raziq decided to investigate. They dug up transcripts and lab reports from several cases and sent them to a group of experts, including University of California criminology professor William Thompson, who has reviewed DNA evidence from labs all over the country. w did the Houston police crime lab stack up?

"It's the worst I've seen," says Thompson. "This doesn't meet the standard of a good junior high school science project."

His findings convinced police to have the DNA from Sutton's case retested at an independent lab. That test was conclusive: Sutton's DNA was not found in the sample. That convinced a judge who ordered him released on bail. Today, he is waiting to see if he will be pardoned.

But the story doesn't end there. Hundreds of other convictions in Houston are now being called into question. Harris County district attorney Chuck Rosenthal and assistant DA Marie Munier must review 10 years worth of cases their office has prosecuted using evidence from the lab.

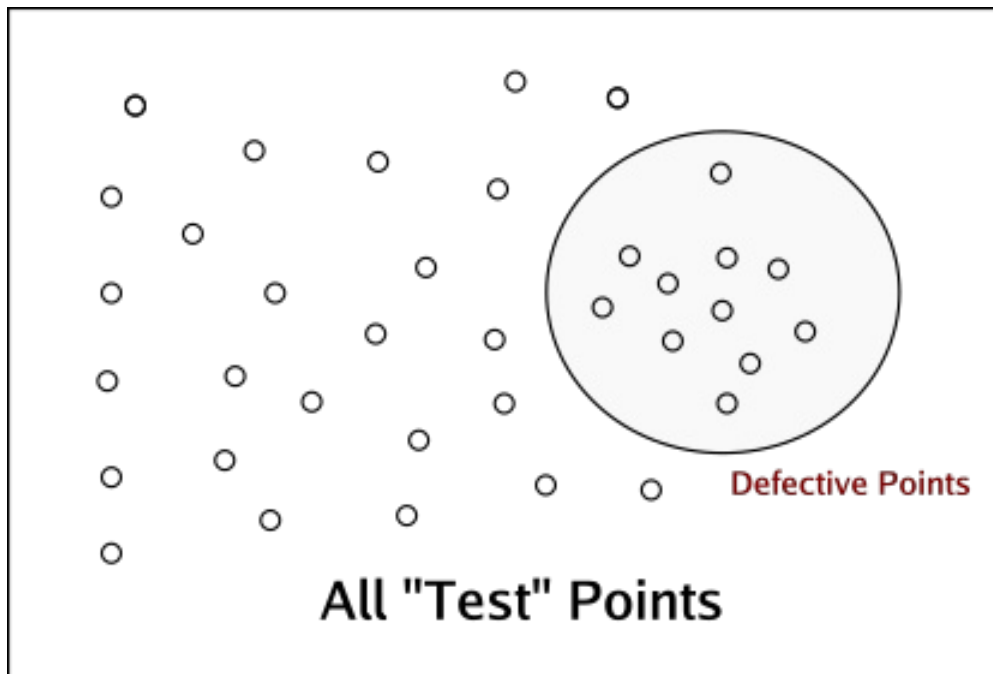
The realization that the testing process had produced at least one false positive had enormous repercussions. Not only does it cast into doubt the correctness of all the other positive test results, it has opened up the possible for massive damages from lawsuits brought by those who may have been wrongly convicted on the basis of a flawed test.

Test Sensitivity and Specificity

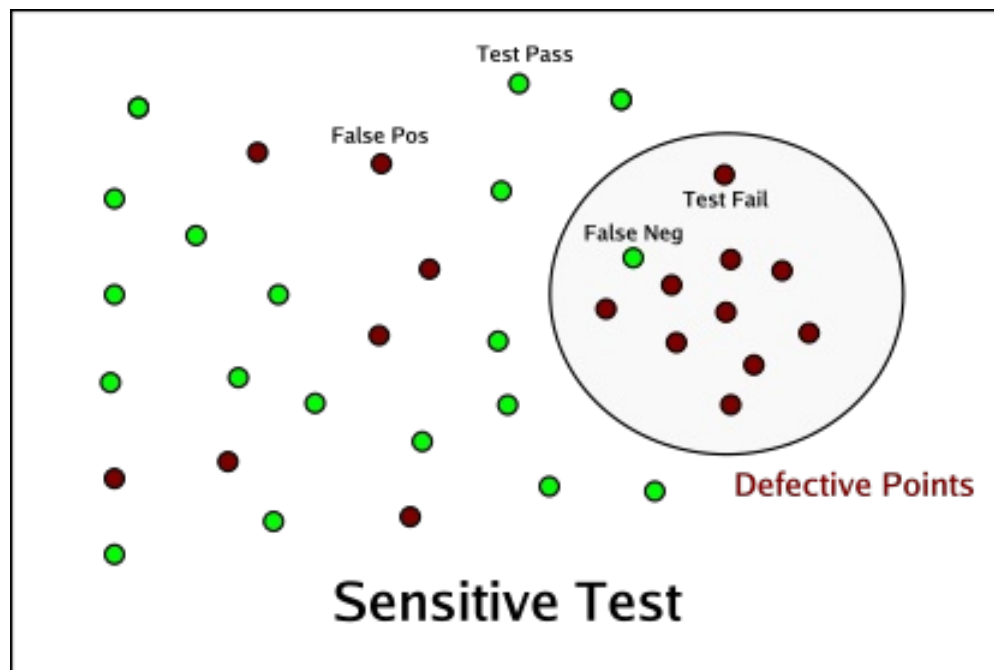
Although not extensively used in software testing right now, the use of metrics called specificity and sensitivity are often used to evaluate the accuracy of a test or a set of tests. To illustrate this, consider the following diagram of "test points" which are things



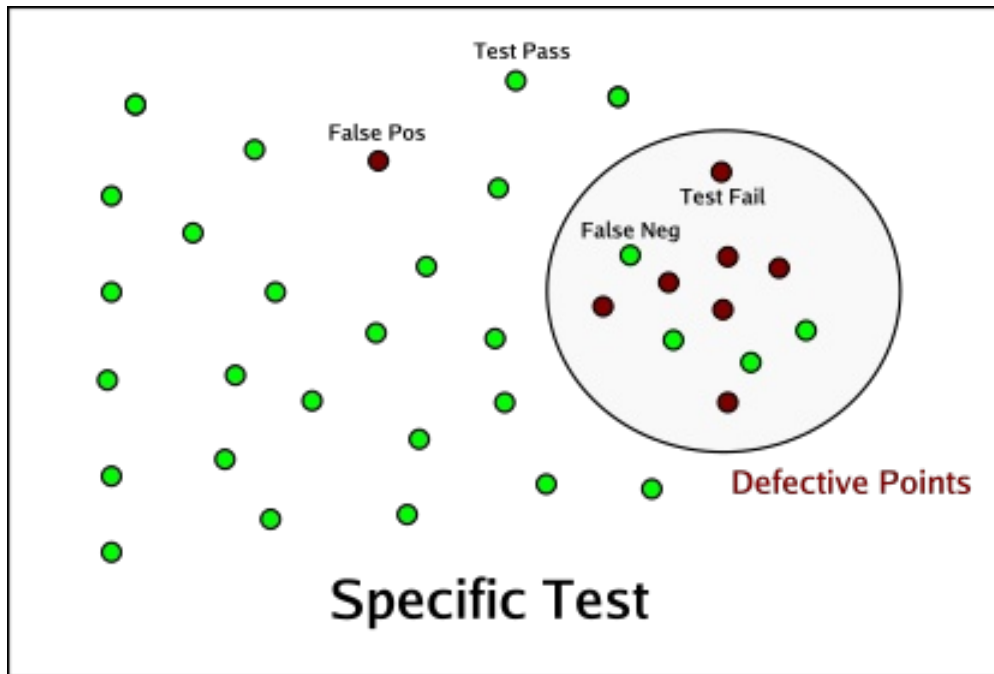
being tested – these could be anything from a person being tested for a disease to some aspect of a software feature. In the diagram, the defects or diseased people or defective items are in the circle labelled "Defective Points." Every point outside the circle is a correct software feature or a disease free persons or some non defective item.



Sensitivity refers to how well a test identifies defects – or equivalently, how good the test is at avoiding false negatives. This is shown in the diagram below. Sensitivity says nothing about false positives so there may be many false positives.



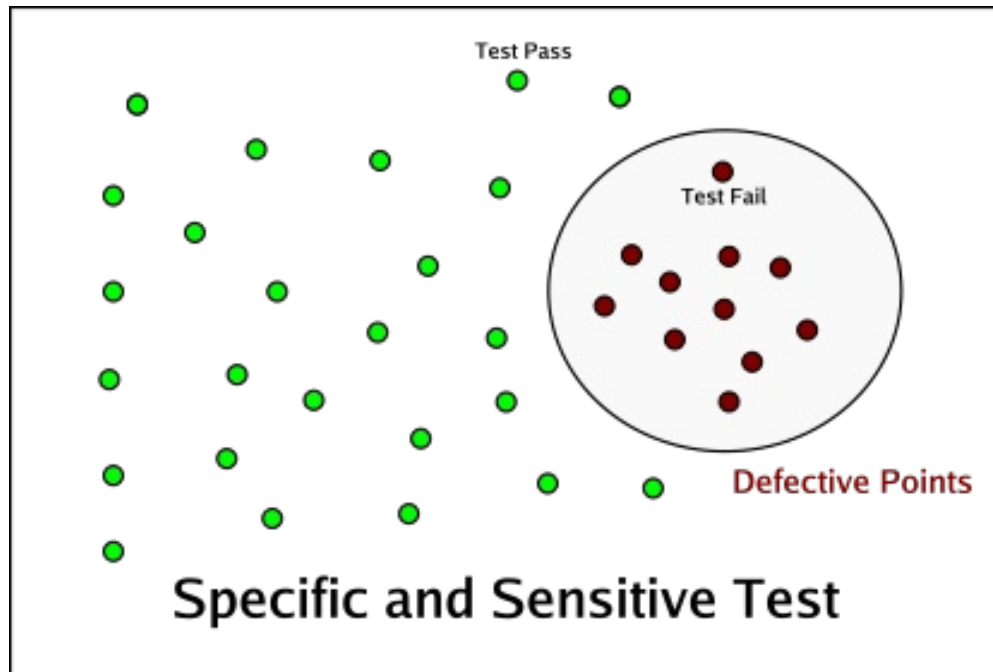
Specificity refers to how well the test avoid false positives. Another way to think of specificity is that tests are specific when avoid false positives. As you can see from the diagrams, these two measures are independent of each other, although at an underlying level, tests that are too sensitive tend to have low specificity and vice versa.



Depending on our testing goals, we may choose tests that are:

1. Sensitive because our test priority is to identify defects. For example, a heart monitor shouldn't have a high rate of false negatives or people will die, so we might be willing to deal with more false positives to make the system more test sensitive.
2. Specific because the cost of having to respond to a test failure is excessive. For example, if a psychiatric test has a lot of false positives, we might wind up committing a lot of sane people who might wind up suing us. In this case, we might be more inclined for very specific tests that reduce our liability while accepting the fact some people who should be committed will not be.

However, many testers operate under the assumption that their tests are always sensitive and specific, as shown below, and never perform any kind of accuracy analysis. In many cases, this is often a symptom of a failure to work through a rigorous risk analysis.



2.3.3 Test Reliability

Testing is said to be reliable when the results of the tests depend only on what is being tested. For example, if a set of tests are performed a number of times and the results depend on when the tests are run or who runs them, then the tests are not reliable.

If I run the tests this morning and find an error and my colleague runs the same tests this afternoon on exactly the same item under test and there is no error found, then is there an error or not? Do I have a false positive or false negative for one of the results?

The most common cause of unreliable test results is the failure to follow a process that can be replicated to verify the results later, which includes not using the same tests when we repeat the testing process. Just like in science where an experiment has to be able to be replicated in order to check the results, so should any form of testing.

When testing is done in a non-repeatable manner there is no way to evaluate how accurate the tests actually are. For example the \$2 road side cocaine tests are not just inaccurate but unreliable because:

The tests use three tubes, which the officer can break in a specific order to rule out everything but the drug in question — but if the officer breaks the tubes in the wrong order, that, too, can invalidate the results. The environment can also present problems. Cold weather slows the color development; heat speeds it up, or sometimes prevents a color reaction from taking place at all. Poor lighting on the street — flashing police lights, sun glare, street lamps — often prevents officers from making the fine distinctions that could make the difference between an arrest and a release.

It should be apparent that there is a strong link between reliable tests and accurate tests, but the terms do refer to different aspects of the testing process. Accuracy is a property of an individual test which means that we can execute the testing process flawlessly but we may still get a false positive or false negative. On the other hand, reliability is about how we execute the tests. We might have a highly accurate test but if we execute it in an unreliable manner, we can get false positives or negatives.

2.3.4 Test Comprehensiveness

This is actually a very important and central issue in testing and deals with how much testing we do, what sort of testing and to what level of exhaustiveness. A very common problem with non-comprehensive testing is that things “slip through the cracks.” We forget about things that might happen and focus only on the things that probably will happen. For example from CNN money:

Steve Valdez knew he'd face scrutiny when he went to a Bank of America branch to cash a check written to him by his wife, since he doesn't have a personal account with the bank.

Although the Tampa, Fla., resident brought two forms of picture identification, the teller said he needed to provide a thumbprint to cash the check.

The problem: Valdez was born without arms and wears a prosthetic. He pleaded with the bank manager to use the photos, but to no avail.

"It was really unfathomable," said Valdez, who was told that he could either bring in his wife or open an account of his own.

A thumbprint is a longstanding requirement for any non-account holder to cash a check in order to prevent check-cashing fraud, according to a BofA spokesman, who admits that the bank should have offered alternatives to Valdez.

Comprehensiveness is often described as the problem of knowing how much testing to do so we don't wind up with our own “Steve Valdez” case and yet not doing so much testing that we use up massive amounts of resources for rather small decreases in risk. We will return to the topic of comprehensiveness throughout this course.



2.3 Sources of Errors

At this point we have clearly identified the importance of doing good testing. Software testers, and testers in general, have developed a set of procedures and protocols for preventing errors in the testing process.

There are four main sources of error for testing:

1. Poor test planning.
2. Poor test design.
3. Poor test execution.
4. Systemic errors.

The first three of these will be dealt with in the modules that follow. The fourth point will be covered in the next module. One of the objectives of this course is to ensure that not only do you do software testing "correctly" but also that you understand why you are doing what you do and can develop ways to make your testing more effective.

It should also be pointed out that the main challenge of all software testers to balance the trade of between effectiveness and economy – how to optimize the quality of the testing given the testing resources available to us.



Testing Concepts and Protocols

Module Three

*Fundamentally you can't do your own QA, it's a question of seeing
you own blind spots.*

Ron Avitzur

I never make stupid mistakes. Only very, very clever ones.

John Peel

All human error is impatience, a premature renunciation of method
Kafka



3.1 Introduction

This module is going to build on two ideas that were presented in earlier modules. First, from module one, the basic reasons that we even do testing.

1. Validation: Testing to validate that we are doing or building the right thing.
2. Verification: Testing to verify that we are building things correctly.
3. Quality: Testing to ensure that we are building things to the right level of quality.
4. Risk: Testing to identify potential business, project and product risk.

In the last module, we looked at the importance of testing correctly and identified four sources of errors that can affect the quality of our testing. These are:

1. Poor test planning.
2. Poor test design.
3. Poor test execution.
4. Systemic errors.

In this module, we are going to focus on three basic topics which relate to both test planning and systemic errors.

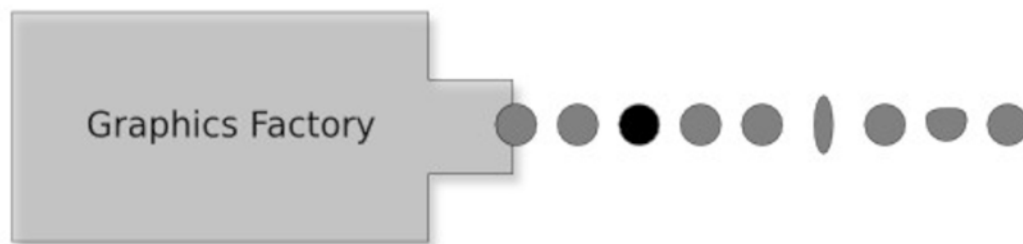
1. Some basic testing concepts and ideas.
2. Some basic quality ideas and how quality management is critical for test planning.
3. What sorts of systemic errors we have to deal with and how testers prevent those types of errors from occurring.

The other sources of errors will be addressed in subsequent modules.

3.2 Quality Control

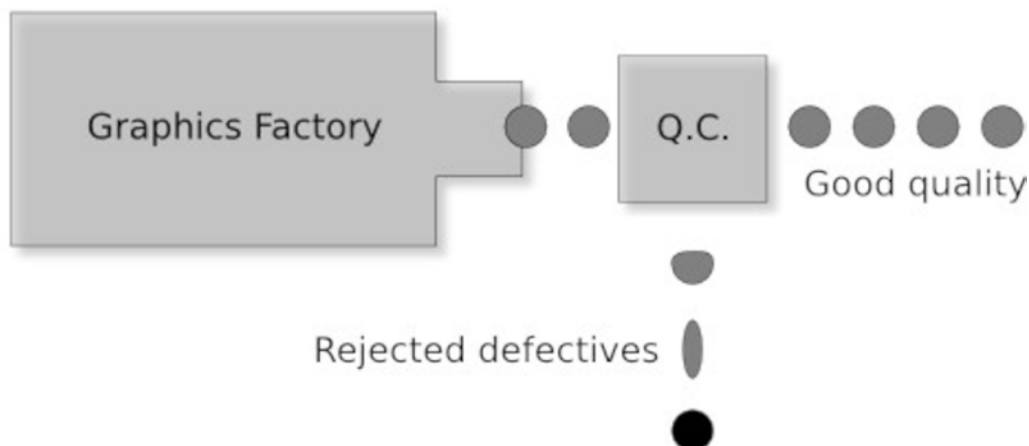
To introduce this section, let's consider a graphics factory that makes grey circles, all nice, round and the same size.

But, as you can see, a number of the circles are defective; some are dented, some are not grey, and some are not circles. If we are selling these circles to our customers, we will probably have some customers who refuse to pay or they send the circles back because they aren't good quality circles.



3.3.1 The Quality Control Function

To keep are customers happy, and to make sure their checks clear, we implement a quality control program to ensure that only good quality circles get shipped and any defective circles are thrown out. To do this, we examine each circle as is produced and decide whether or not to accept it as good enough to ship or if we should reject it as defective.



But in order to do quality control we need to have some way to answer the question "Should this circle be accepted or rejected?" Or more relevant to us, the question can be restated as "How do we test the circles to see which ones are defective?"

This issue has been a central problem in quality control for centuries. There have been three common ways to answer this problem.

“I’ll know it when I see it”

This approach is one that is characterized by the comment made in 1964 by Justice Potter Stewart when he tried to define what it meant for something to be called “obscene:”

I shall not today attempt further to define the kinds of material I understand to be embraced, but I know it when I see it

This notion of being unable to define “quality” is a common theme that was articulated by Robert Pirsig in 1975, in his book *Zen and the Art of Motorcycle Maintenance*: that quality can’t be defined but can be only identified when we encounter it – quality is a “feeling.”

While this is a common definition and can lead to many interesting philosophical discussions, this definition does nothing to answer our question about how we go about testing the circles other than by how they make us feel.

Fitness for use

The previous approach relied on the judgment of the producer – “We think we make good circles” – but ignores the question of “What makes a circle acceptable in the opinion of the customer?” The fitness for use approach says that a circle is good enough quality when it meets the needs of the customer.

We say that a circle is good enough when it meets the customer's requirement for how they want to use that circle. This is a considerable improvement from a testing perspective because we have moved the criteria for defective from our feelings about circles to a more pragmatic testing approach (ask the customer how good it has to be), and we can start to list the properties that our circles need to possess to be “fit for use.”

However, this approach starts to run into practical problems when we try and implement it. Consider the following very real situations we might encounter:

1. The customer cannot tell us what makes a good circle beyond saying “I can’t tell you what a good circle should be, but I’ll know it when I see it.” Then we are back to where we started.
2. The customer's criteria for what makes a good circle keep changing so that what was a good circle yesterday is now unacceptable today. Our testing is in chaos since we are always trying to hit a moving target.
3. We have different customers who all have different criteria for fitness of use – and they all conflict with each other.

The External Standard

The most common approach to quality control is to develop an external standard which can be considered as a contract between all the parties on what constitutes fitness for use and, therefore, defines acceptable quality

Now our testing problem becomes much more manageable because we compare each circle to the standard, and if it meets the standard, then it is a good enough circle to be shipped; and if it doesn't meet the standard, then it is rejected.

If we look at all the places where quality control is practised, we see that this is the approach that is often used. Think of building inspectors, testing consumer products, evaluating people for driver's licenses – in each case there is some standard, often a checklist, that has to be satisfied before the building is deemed to be fit for occupancy, the products fit for the marketplace or a driver fit to drive.

But we are still left with one big issue: how do we actually apply the standard, or in other words, how do we actually test or evaluate something against the standard?

3.2.2 Testing: The “Doing” Part of Quality Control.

Back to our circle problem. Suppose we have been given a standard to use in our circle quality control. But it reads “All circles shall be rejected that are not nice circles.”

Now we are back to the “I’ll know it when I see it” approach because we have no idea what it means for a circle to be nice. This standard is not testable. We will be looking a lot later on in this course about what being testable means in our testing environment, but for now, we can say intuitively that for my standard to be testable, it needs to have the following properties.

1. It needs to be quantified. I need my acceptable circle defined in terms of size, allowable eccentricity, colour and so on so that I can make a meaningful comparison between a circle and the standard.
2. It removes any element of subjectivity so that any two people using the same standard will come to the same conclusion about each circle.
3. There have to be clear boundaries as to what variance in measurements are acceptable and what variances are not. For example, “The circle should be very close to 3cm in diameter” leaves me not sure if 3.1 cm is close to 3cm or not. I need something more like “The circle should be 3cm in diameter plus or minus .05 cm”

At this point, we see that we have a dependency relationship between quality control and testing. Quality control is the process of quantifying exactly what test measurements need to be for something to be of acceptable quality, usually with reference to some standard. Testing is the actual set of procedures for measuring and evaluating the outputs against the standard to decide if they are acceptable or not.



3.2.3 Quality Management and Testing

A very important point to notice here is, which we will explore throughout this course, that it is a quality issue that motivates the need for testing.

When we start to do any sort of testing, we need to be able to set some standard parameters about how much testing we should be doing, to what level of rigour, and determining when we have done enough testing.

There are a couple of standard (and wrong) answers to the question: “When do stop testing software?” These are often along the lines of:

1. You never stop testing.
2. When you run out of time and money.
3. When they pry it out of your fingers.

All of these answers are symptoms of a bad testing process. There is only one right answer: we are finished testing when then testing has met the quality control objectives.

For a lot of organizations, this is almost impossible to do since there are no quality control goals, so testing becomes an ad hoc and, in many cases, almost random activity.

When setting up a testing process, we need to be able to answer a number of questions to help us plan and execute our testing activities:

1. How fine grained does the testing have to be? Do we need to test each item produced or would a random sample do? If we can use a sample, how big a sample do we need? If we were producing heart pacemakers, we would probably want to test each one; but if we are producing ball point pens, we probably would want to test samples from each batch produced.
2. How much resources should be expended on the testing effort? The type and severity of the risks we are looking at mitigating with our quality control influence the amount of effort we expend to perform the test activities. Again, we should probably spend more time testing heart pacemakers than testing video games; or more time testing code that runs a nuclear reactor cooling system rather than an iPhone game app.
3. Do we have to test everything? Can we test only the high risk or the most used features of the item or software?
4. What sort of defect levels are allowable? Can we tolerate letting a single defective item slip through (probably not for an artificial heart valve) or can we have a certain number of defectives ship (probably we can for something like paper clips).

These questions, and other similar questions, cannot be answered by looking at the testing activities, they can only be answered by looking at how quality is defined and managed for a specific product or project. Testers derive answers to these questions from understanding the quality objectives that the testing is supposed to support.

3.2.4 Defining Quality.

Perhaps the most insightful observation about what quality is comes from one of the gurus of the quality revolution in the 1970s, Philip B. Crosby [Crosby79:13]:

The problem of quality management is not what people don't know about it. The problem is what they think they do know.

Everyone is for it (under certain conditions of course). Everyone feels they understand it (even though they wouldn't want to explain it). Everyone thinks execution is only a matter of following natural inclinations (after all, we do get a long somehow). And of course, most people feel that all problems in these areas are caused by other people (if only "they" would take time to do things right).

That doesn't get us far but it does give us a sense of how much debate went into this discussion on what quality is – remember Pirsig's statement that “quality is a feeling.”

To understand quality we have to abandon the assumption that there is a single definition or rule that we can apply that completely, consistently and (perhaps most importantly) that universally defines quality. In fact, quality can never be defined exclusively in terms of the properties that a system, product or service can have or does not have.

Instead, quality is the result of the interplay of the features of a product and the expectations of various individuals or groups and what those features mean to them.

For example, a car may be declared by one driver to be a superb road machine while another may declare it to be a piece of junk. If the quality of the car were an inherent property of the car, like the colour, then we would not expect this variation in “quality.” The only conclusion is that quality, like beauty, lies in the eye of the beholder.

Quality Attributes

Nowadays we think in terms of “quality attributes” which we can define as those properties of a product which will be used by some stakeholder to assess the product's “quality.” This nicely complements the three definitions we saw before since both fitness for use and conformance to standards are both quality attributes. Quality is always perceived by stakeholders and is not an inherent property of the product.

Even the idea of fitness for use is a subjective quality assessment since different users may have different ideas of what would constitute “use”. Likewise, conformance to a standard really relies on whose standards we are using.

To stress the importance of perception, suppose in the example on the next page that the above cell phone plan is offered by a wireless phone company. The quality of the plan, and ultimately the decision to get the plan, will be based on what the value of the attributes have to each of the two stakeholders.

Philip Crosby in his 1992 book *Completeness* discusses the idea that quality is determined by how it impacts the success of the stakeholders, which he groups into employees, customers and suppliers. But the more important theme of the work represents a





shift away from his work of 30 years earlier that emphasized quality as “zero defects” towards a view that we create quality by understanding what the stakeholders expect from a product at a specific time.

Quality, in this view, is a combination of the attributes of the product as perceived through the filter of expectations that stakeholders have for that product. But the reality is that these expectations always change over time depending on the environmental, business and other factors that impact the stakeholders.

Quality and Business Impact Factors

Organizations are often concerned with quality attributes that have business impact; which can also be thought of as how the quality of the product mitigates different kinds of business risk.

For example, recently a massive meat recall occurred in California as sick and diseased cattle were used in slaughterhouses in violation of health standards. By using poor quality beef, the meat packing companies increased the amount of risk that they incurred; and of course the outcome was a massive recall that had a tremendous negative impact on the company financially.

In software, consider the problems that MicroSoft had with Vista. Microsoft assumed that quality would be measured by users to a great extent by the visual presentation and the slick graphics. But users looked at quality in terms of how much they had to spend to upgrade their hardware and how well the new OS ran their applications. Fortunately MicroSoft learned from this fiasco and improved their understanding of their users’ quality expectations to make Windows 7 a much more successful product.

From this perspective, a contributing factor to quality is that anything that reduces business risk is an improvement in quality, from the business perspective.

Quality and Business Constraints

Stephen Wright notes “You can't have everything...where would you put it?” Realistically we cannot do everything, or to rephrase that, we do not have the resources to reach dizzying levels of quality every time (as some gurus exhort us to do). Instead, everything we do is constrained by our resources – time, budget, staff and other factors.

McDonald's could create incredible burgers based on a gourmet's standards by using only Kobe beef, but there just isn't enough Kobe beef for billions of burgers. Of course that would affect the price of the burgers and reduce its quality in the eyes of McDonald's primary market. One quality attribute that is important to a high school student with only ten dollars to spend is “How much food per dollar can I get?” A \$40 Kobe beef burger is a poor quality item by that measure of quality while a pile of \$1 burgers is good quality to our customer.

Quality in this perspective is achieved by meeting business objectives within the existing constraints. We have to choose the level of quality that allows us to accomplish what we set out to do as a business without exceeding our ability to do so. Many companies suffered in the past as they over-extended themselves by trying to offer unrealistic levels of “world class customer service” without the ability to meet the new higher levels of expectations that they created in their customers.

The other interesting point to note here is that quality is not fixed, the level of acceptable or required quality is going to vary as the constraints and business objectives vary over time.

3.2.5 Quality Management

What constitutes quality varies according to a number of factors. Quality Management generally refers to how an organization defines and embeds quality within its organization's processes, structures and culture.

Quality management provides a set of guidelines and objectives for how to do quality control, which means that the organization may have different sets of quality control guidelines depending on a combination of business factors. Quality management also provides a larger scope for analysis of quality control efforts so that the organization can look at how to improve the quality control based on past experience.

Good Enough Quality (GEQ)

The idea behind GEQ is that even within an organization, we may need to adapt our idea of what quality means at different times and under different conditions. These in turn determine the metrics and processes that we use to achieve quality that is “good enough” for that specific set of circumstances.



The way we do quality control and how we go about our testing, particularly setting the parameters for testing, is determined by deciding what we need to do to achieve good enough quality.

In a sense, our good enough quality can be thought of as the “quality philosophy” that we use to frame our quality control decisions.

Some examples of GEQ approaches:

1. **Not Too Bad** ("We're not dead yet"). Our quality only has to be good enough so we can continue to stay in business. Make it good enough so that we aren't successfully sued.
2. **Positive Infallibility** ("Anything we do is good"). Our organization is the best in the world. Because we're so good, anything we do is automatically good. Think about success. Don't think about failure, because "negative" thinking makes for poor quality.
3. **Righteous Exhaustion** ("Perfection or bust"). No product is good enough; it's effort that counts. And only our complete exhaustion will be a good enough level of effort. Business issues are not our concern. We will do everything we possibly can to make it perfect. Since we'll never be finished improving, someone will have to come in and pry it from our fingers if they want it. Then they will bear the blame for any quality problems, not us.
4. **Customer Is Always Right** ("Customers seem to like it"). If customers like it, it must be good enough. Of course, you can't please everybody all the time. And if a current or potential customer doesn't like the product, it's up to them to let us know. We can't read their minds. Quality by market share?
5. **Defined Process** ("We follow a good process"). Quality is the result of the process we use to build the product. We have defined our process and we think it's a good process. Therefore, as long as we follow the process, a good enough product will inevitably result.
6. **Static Requirements** ("We satisfy the requirements"). We have defined quality in terms of objective, quantifiable, noncontroversial goals. If we meet those goals, then we have a good enough product, no matter what other subjective, non-quantifiable, controversial goals might be suggested.
7. **Accountability** ("We fulfill our promises"). Quality is defined by a contract. We promise to do certain things and achieve certain goals. If we fulfill our contract, that is good enough.
8. **Advocacy** ("We make every reasonable effort"). We advocate for excellence. Throughout the project, we look for ways to prevent problems, and to find and fix the ones we couldn't prevent. If we work faithfully toward excellence, that will be good enough.
9. **Dynamic Tradeoff** ("We weigh many factors"). With respect to our mission and the situation at hand, a product is good enough when it has sufficient benefits and no critical problems, its benefit sufficiently outweighs its noncritical problems, and it would cause more harm than good to continue improving it.

Software testing, like other forms of testing, is a set of techniques and tools for measuring something (in this case software) as part of the quality control function. The parameters of the testing are determined by the quality control process, which in turn derives those parameters from the larger scope of quality management.

Understanding these relationships is critical for the software tester to be able to avoid some of the common traps that are easy to fall into: such as not being able to establish GEQ parameters which can result in unfocused testing, and usually either doing too much testing or not enough testing and even doing the wrong type of testing.

More importantly, one of the responsibilities of a test manager is to ensure that a project has a realistic and accurately defined GEQ policy that both manages the risk of software not working to an acceptable standard and scoping how much work is needed in terms of robustness, functionality and other factors.

3.2.6 Software Quality Assurance

Suppose that our graphics factory makes 50% defective circles; a defect rate that is not consistent at all with our quality standards which says that we should have no more than a 10% defect rate.

This is no longer a problem that can be solved by quality control since no matter how much we measure or test, the defect rate remains the same. We need a new way of “assuring” that the quality of our circles is such that 90% of them are good circles.

Quality assurance is the set of activities needed to ensure the quality of whatever we are measuring in quality control. Quality assurance consists of creating, describing and implementing the techniques and practices in the business processes that assure that specific quality levels can be met. Quality assurance also includes interpreting the definition of what GEQ is for a specific activity: to specify what level of quality is appropriate and what that means for that activity, how it is to be evaluated at that point, and the policies and procedures that are used for a process to consistently meet these quality objectives.

For example, McDonald's has a quality management objective that all cheeseburgers made at any McDonald's should be identical, and the cheeseburgers made at different McDonald's should also be identical. Quality assurance would be the development of a standardized way (defined process) of producing burgers using standardized components and standard tools to produce identical burgers.

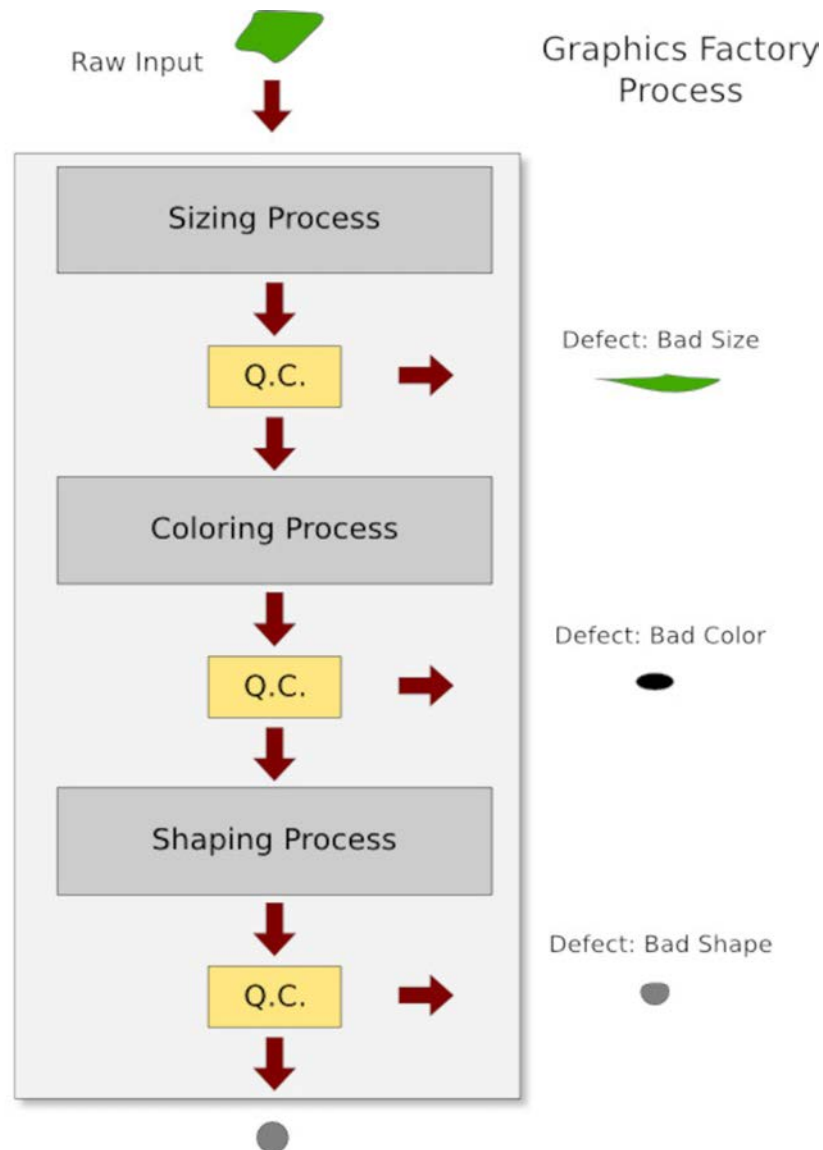
Consider for example our graphics factory. The production process takes raw inputs and puts them through three stages:

1. Sizing: the inputs are formed to the right size, inputs that cannot be shaped to the right size are rejected as defects.
2. Colouring: the sized inputs are coloured, those that are not the right colour afterwards are rejected as defects.



3. Shaping: the correctly sized and coloured inputs are shaped into circles, those that are not circular after shaping are rejected.

This is illustrated in the diagram. Notice that our quality control is now being done at each step of the process; something that is often called “just in time” testing or “testing throughout the development lifecycle.” We are testing each output from each step just in time to determine if it should move on to the next stage or be rejected.



Ideally, when we get to the end of the process, we should see very few defects coming out of our overall process, since we are pulling defects out during the process to ensure they are not in the final outputs.

On the surface we seem to have assured quality levels since now less than 10% of our outputs are now defective – but that is just quality control, NOT quality assurance.

From a business perspective what we want to do is to reduce our defect level to 10% of our inputs not 10% of our outputs. Why? Because each defect represents a cost to us.

Even though quality control prevented defects from shipping, it did not prevent defects from occurring, and preventing defect from occurring is the goal of quality assurance.

Root Cause Analysis

Once we have the production data to work with, we can perform what is called a root cause analysis which is answering the question "At what point are the defects being introduced into the process?" Once that point of failure is identified, corrective action is taken to ensure the defect is no longer being introduced.

The other aspect of this, which we will look at more detail later, is that we can our quality management insights allow for the establishment of best practices and standard operational processes that are designed to avoid errors being made.

Quality Assurance

In essence we can sum up quality assurance in the following way.

1. Defects, when they occur, add costs to our process. In some cases, these costs can be the factor that makes or breaks the business.
2. Defects should not occur if we are doing everything right. Therefore if we are getting defects there is root cause for each defect somewhere in our process – someplace where we are doing something wrong, or could be doing it better.
3. We use just in time testing to identify where in our process the defect is occurring.
4. We fix the process at that point to ensure that defects of that type do not occur in the future.

Another way to think about it is that quality assurance is testing and debugging some form of production process.

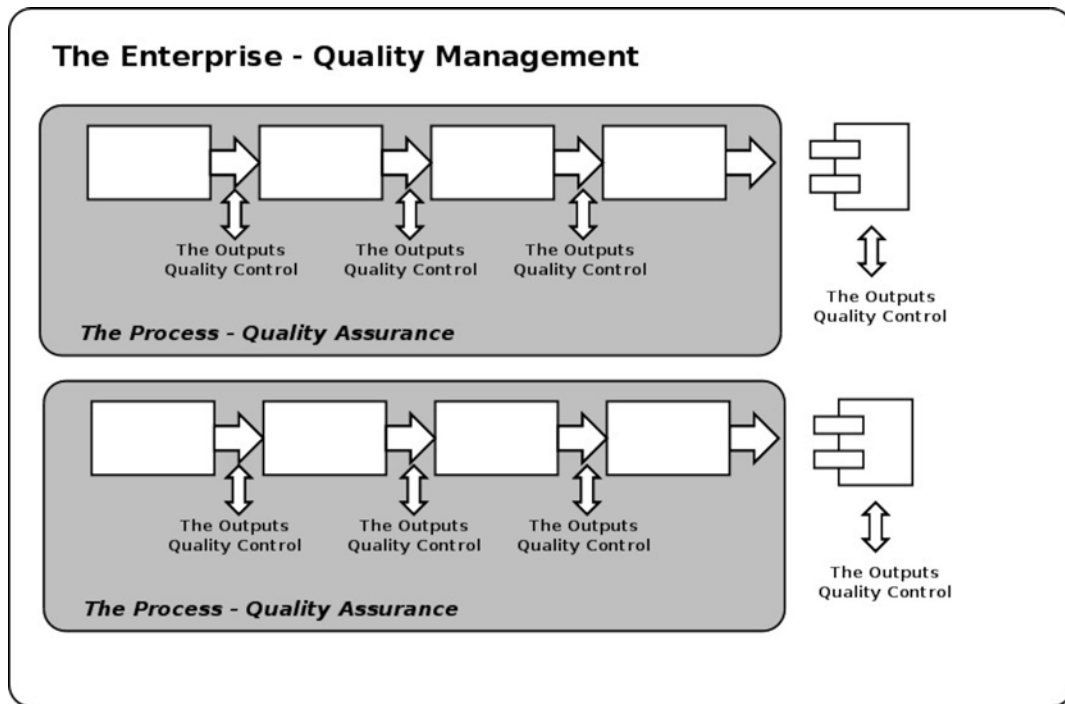
3.2.7 QM, QA, QC and Testing

This section has been about the relationship between the three aspects of quality and the process of testing. In some sense, we can think of the quality processes as the planning or the strategies we use to achieve our quality goals; and testing as the way we measure how well we are doing.

Because these areas are all intertwined, the line between doing testing and doing quality functions gets quite blurry and often has to be defined within each organization as to how they want to classify and organize these activities.

The diagram on the next page summarizes what we have looked at so far.





Quality management deals with the cultural of the organization (the “That’s the way we do it here” beliefs of a company) with regards to quality and process, and the other dimensions of staffing, education, infrastructure and resources necessary to support a quality program in an organization.

A critical role of quality management is to identify what the expectations of the organization are in terms of quality and then communicated in terms of some form of quality mission. The quality mission is a framework for the people in the organization that guides their decision making process on what to do about quality in their area of responsibility and in the various processes they participate in.

QM is the systematic way of ensuring that organized activities happen the way they are planned.

Quality control is the planning and organization of the various kinds of metrics we will be using to measure quality, as well as defining exactly what we are measuring and when. These metrics are used to measure how our quality efforts are succeeding as well as to provide data to be used in making quality decisions (for example, whether or not an output is defective).

Quality assurance is the evaluation of production processes in order to identify where defects are occurring and modifying the process to eliminate the source of the defects.

Testing is the actual practices, techniques and procedures we use to do the activities required by quality control. While the testing activities can be done anytime and in any context, the processes and practices that make for good testing are always the same.

The Failure of TQM (or learning from other people's mistakes)

In a number of industries, the quality management approach was seen as something that would fix a wide range of problems that organizations were experiencing. The market place was flooded in the 1980s with programs and management ideas like Total Quality Management (TQM).

However, as we look back at those attempts, there have been many more quality management failures than successes. In many informal (and usually off the record) discussions I have had with executives who have been through some failed quality management exercise, I consistently heard that quality management was not a realistic way to do business in their organization or that TQM was too “theoretical” to work in the real world. However, the fact that many organizations are able to do it shows that this generalization is not a correct analysis.

In fact, the reason for quality management failures has been known for decades. Unless an organization is totally committed to quality programs, from senior management on down, and has the maturity to stick to the quality program, they will fail. The critical aspect seems to be the that senior management has to set the pace, create the culture and then visibly not waver in their commitment.

Quality Maturity

One of the insights that Philip Crosby had back in the 1960s was that the success of any quality program depends on something he called the Quality Management Maturity Grid. In order for any organization to be able to effectively do QM, they had to grow through five stages of maturity within their organizational culture¹.

1. **Uncertainty:** Quality problems are always the fault of the Quality department.
2. **Awakening:** Quality management might actually fix problems.
3. **Enlightenment:** Management and quality people are working together to fix problems.
4. **Wisdom:** Quality management integrated in to the way we do things.
5. **Certainty:** We do not have quality problems because we understand everything.

It is interesting to note that there is a correlation between Crosby's Maturity Grid and the five levels of maturity that we will see in the Testing Maturity Model and Boris Beizer's levels of awareness for the software tester.



3.2.8 The GEQ Model

GEQ uses four factors and six perspectives in framing quality questions. The basic idea is to address the recurring problem quality by analyzing the differences in perspectives between stakeholders.

The GEQ factors are designed to be rules of thumb for software developers to use in assessing quality within the context of what they are trying to achieve by considering the stakeholders inputs.

A Framework for Good Enough Quality

Taken together, these four GEQ factors and six GEQ perspectives comprise a robust set of reminders that can help frame a conversation or make a convincing case about shipping a product, improving it, or implementing some better practice. For instance, when someone tells me that “good enough is not good enough,” I remember the stakeholder and critical purpose perspectives and translate that apparently paradoxical statement into something I can question, such as “good enough *for you* is not good enough *for me*” or “good enough to *survive* is not good enough to *succeed*.” Then the dialogue becomes one of examining whose values matter or what purpose we are really trying to achieve.

GEQ Factors

This four-part process expands upon the GEQ definition. It's not a rigorous formula, by any means. These factors are designed to help busy, stressed-out software people remember what to think about when weighing product quality.

1. Assess the benefits of the product.

- **Identification.** What are the known benefits or potential benefits for stakeholders of the product?
- **Likelihood.** Assuming the product works as designed, how likely are stakeholders to realize each benefit?
- **Impact.** How desirable is each benefit to stakeholders?
- **Individual criticality.** Which benefits, all by themselves, are completely indispensable?
- **Overall benefit.** Taken as a whole, and assuming no problems, are there sufficient benefits for stakeholders?

2. Assess the problems of the product.

- **Identification.** What are the problems or potential problems for stakeholders of the product?
- **Likelihood.** How likely are stakeholders to experience each problem?
- **Impact.** How damaging is each problem to stakeholders? Are there workarounds?
- **Individual criticality.** Which problems, all by themselves, are completely unacceptable?
- **Overall problems.** How do all the problems add up? Are there too many noncritical problems?

3. Assess product quality.

- **Overall quality.** With respect to the GEQ perspective, do benefits appear to outweigh problems?
- **Margin of safety/excellence.** By how much do we need or want benefits to outweigh problems?

4. Assess the logistics of improving the product (or holding out for something better).

- **Strategies.** What strategies can we use to improve the product?
- **Capabilities.** How able are we to implement those strategies? Do we know how?
- **Costs.** How much cost or trouble will improvement entail? Is that the best use of resources?
- **Schedule.** Can we ship now and improve later? Can we achieve improvement in an acceptable time frame?
- **Benefits.** How specifically will it improve? Are there any side benefits to improving it (for example, better morale)?
- **Problems.** How might improvement backfire (for example, introduce bugs, hurt morale, starve other projects)?

GEQ Perspectives

The GEQ factors just listed are necessary but not sufficient. In order to perform a responsible assessment, you must also examine each of the factors from six vital perspectives.

1. **Stakeholders.** Whose opinion about quality matters? (For example, project team, customers, trade press, courts of law.)
2. **Critical purpose.** What do we have to achieve? (Is it immediate survival, profitability, market share, market growth, customer satisfaction?)
3. **Time frame.** What is the time-sensitivity of quality perception? (Is it immediate, near-term, long-term, after critical events?)
4. **Alternatives.** How does this product compare to alternatives, such as competing products, services, or solutions?
5. **Consequences of failure.** What if quality is a bit worse than good enough? Do we need a contingency plan?
6. **Quality of assessment.** How confident are we in our assessment? Is it good enough?

3.3 Terminology

This section introduces some standard terminology.

3.3.1 Failure, Faults and Errors

To prevent things from going bad, we need to know how to identify and then eliminate the root causes of both inefficiency and ineffectiveness. We want to know exactly what went wrong, why it went wrong and what we can do to fix the current problem and prevent any future recurrences. The following are standard Quality Engineering terms that will be used in this course:

Defects



1. **Defect:** A defect is some problem with the software. Defect is a generic term that covers errors, faults, failures and exceptions.
2. **Failure:** A failure is where a software system or component does not perform its required functions according to the stated specifications or fails an acceptance test for that system or component.
3. **Fault:** A fault is an incorrect step, construct, process or data usage. A fault can occur in any artifact including code, design, algorithms and data models. Failures of the system are caused by faults in the deliverables of the software development process.
4. **Exception:** An exception is a fault in the environment the software runs in. File system corruption, a database failure or a network connection dropping would be examples of exceptions. We cannot fix an exception in our code, we can only handle them and respond to them. Failures can be the result of exceptions that are not handled by the software. One of the properties of robust code is that it handles exceptions well.

5. **Error:** A error is a human action that resulted in a fault. This can be writing the wrong specification, misunderstanding a requirement, not understanding a design, testing the wrong features or giving a programmer a task beyond their capabilities. An error is any thing we do that eventually leads to creating a fault somewhere in the software. These are what are referred to as the root causes of failures.

When a software has failures, it fails a test. A failure is not just a wrong test result, but is any place where the software did not do what it was supposed to or did something it was not supposed to do. For example, if the software runs too slow or does not cover the full range of possible data it may have to deal with, or if the interface is difficult to use or the system deletes data when it shouldn't – these are all failures. Whenever software does not perform as promised or expected for whatever reason constitutes a failure.

Faults cause failures. Faults occur when we build something wrong, which includes building something that was not required, not building something that was required, or building something the wrong way. Faults include coding defects but also cover design defects, data modelling defects, poor interface design and bad ergonomics.

Errors are the reasons faults happen. They are the things we do poorly as individuals and as teams that result in us making mistakes.

Here are some examples of common errors (some of these have already been mentioned):

1. Poor quality specifications. (We don't know exactly what we are building.)
2. Poor requirements. (We don't really understand why we are doing this or what it is supposed to be used for.)
3. Incomplete specifications. (The solution turned out to be lot more complicated that we originally thought it would be... a LOT more complicated.)
4. Communication errors between functional team roles. (The testers aren't talking to developers and no one is talking to the end user.)
5. Information and documentation being silo-ed and duplicated across functional team roles. (This group has the requirements, this other group has the test cases, and this group over here has the code, but they all are out of sync and everyone says their information is right and everyone else is wrong.)
6. Obsolete documentation or documentation that is hard to update. (Who has time for all this when there are deadlines looming?)
7. Over engineering. (We think it's a cool feature even though no one uses it apparently.)
8. Reliance on error prone manual processes. (Running tests can be very time consuming and boring and my mind tends to wander after a few hours. Same for updating documentation, proof-reading specifications.)
9. Lack of a verification process. ("But that isn't what the design meant. Why didn't you check with the designer before writing all that code?")

10. Validation done too late in the development process. (“What do you mean that isn't what they want?”)
11. Unable to modify/add late requirements or changes to requirements. (“We will have to rebuild the whole application architecture now that we finally have the security requirements. Sigh”)

3.3.2 Other Terms

Program faults versus Exceptions

A program fault is a fault in the component under development or test (specifically YOUR code). An exception is a failure in the environment a component runs in, which may either be another component which interacts with the component under test, or the operation environment itself (specifically someone else's code).

For example, writing the wrong code in a component is a program fault. Every time we run a test case which exercises the fault, the test case fails. We can make the test pass by going into the code and correcting the program fault.

An exception is when something outside the component is the error source of the fault. For example, the database being accessed is unavailable or the system runs out of memory. An exception cannot be fixed by correcting the code. An exception produces an intermittent failure with the same test case depending on the conditions the code is run under.

For example, suppose that a program opens a file to read data from. If the code is perfect, the program could fail if the input file is missing or is locked because it is open for writing by another application. The resulting failure is the result of an exception.

Exceptions cannot be fixed, they can only be anticipated and handled by the developer.

Component

A component refers to any software aggregate that has visibility in the development environment – for example, a method, a class, an object, a function, a module, an executable, a task, a utility subsystem, an application subsystem.

Test Case

A test case is a specification of the system state, input values and expected result. A test case contains the following information:

1. Specification of the state of the system required in order to run the test.
2. A test input.
3. An expected result based on validation or verification criteria.



Test Suite

A test suite is a collection of test cases designed to meet a set of testing objectives. There is some variation in what exactly is called a test suite, particularly with a number of the automated tools. The difference is often due to different ways the tools automate testing, but in general, a test suite is always a set of test cases that we have logical or pragmatic reason for grouping together.

Coverage and Confidence

Coverage is the degree to which a given test suite is subjectively felt to adequately meet the completeness requirement, which is defined differently for different kinds of tests. Confidence is a subjective measure of how good we feel a set of tests are at detecting bugs, or how sensitive they are to errors.

3.4 Systemic Sources of Errors

At this point we turn our attention to the tester. One of the most important ideas that you can come away with from this course is that the most critical part of effective software testing is the mindset of the tester and how the tester applies the testing methods.

Software testing is a complex process that requires a high degree of skill, organization and a particular way of getting things done. Although the methods we use in software testing are rigorous and formal, they are based on models that are attempts to emulate the best practices testers have evolved over centuries in many different testing environments.

Recall that a paradigm is a “way we do things round here.” You may have noticed that when we did the triangle problem, some students (not necessarily you) might have commented that they didn't even think to test some of the conditions. The reason was that those alternatives were not in their “paradigm,” meaning that it was not the way they were used to thinking.

The reason for this is that we all tend to fall into habits of thinking that are not necessarily optimal for the situation at hand. In fact these are often more than just habits, but tend to be artifacts of how our brains process information.

In particular, a software tester has to learn to think and work in a particular way and avoid what I call “Mind Forged Manacles” or tendencies that corrupt the testing process.

The kinds of mind forged manacles we can fall prey to are very common and well known to psychologists. One of the real insights of the testing community is that they had to be rigorous in ensuring that they did not fall into those traps in the course of testing. Here are several that are especially important for software testers to be aware of.

3.4.1 The Self-fulfilling Prophecy

This is also known as the “experimenter effect” and is one of the primary reasons that we use double blind experiments. The self-fulfilling prophecy works this way:

1. The tester or someone the tester reports to has certain expectations of the outcome of the testing based on their conscious or unconscious beliefs, such as “This component is bug free.”
2. Those expectations may be communicated those those to others through subtle cues, usually subconsciously.
3. People, including the one sending the cues, tend to respond to these cues by adjusting their behaviour to satisfy the communicated expectations.
4. The resulting effect of the adjusted behaviour is that the original expectation becomes true



In other words, a prediction is made based on a belief, behaviour becomes biased to do things to support that belief and then the result of that behaviour proves that belief. The important aspect of this effect is that it happens entirely subconsciously, you cannot decide to just not do it – your mind just doesn't work that way.

For example, one commonly cited examples of this effect is in the stock market, where if most investors believe that a crash is imminent, they respond by dumping their stock which actually causes the crash. Similarly in politics, if a candidate in an election states he does believe he can win, voters will not bother voting for him, and he winds up losing.

Robert Rosenthal, a Harvard professor of social psychology, cited the term self fulfilling prophecy (said to be been coined by Robert King Merton, a sociologist) in his work studying of teacher's expectations on students' academic and physical performance, the effects of experimenters' expectations on the results of their research, and the effects of clinicians' expectations on their patients' mental and physical health.

For example, two of the studies Rosenthal cites are:

A 1971 experiment in which Rosenthal told a group of students that he had developed a strain of super-intelligent rats that could run mazes quickly. He then passed out perfectly normal rats at random, telling half of the students that they had the new "maze-bright" rats and the other half that they got "maze-dull" rats. The rats believed to be bright improved daily in running the maze they ran faster and more accurately. The "dull" rats refused to budge from the starting point 29% of the time, while the "bright" rats refused only 11% of the time.[Rosenthal1982]

And another study which illustrates a version of the self-fulfilling prophecy called the Pygmalion Effect.

Rosenthal and Lenore Jacobson worked with elementary school children from 18 classrooms. They randomly chose 20% of the children from each room and told the teachers they were "intellectual bloomers." They explained that these children could be expected to show remarkable gains during the year. The experimental children showed average IQ gains of two points in verbal ability, seven points in reasoning and four points in over all IQ. The "intellectual bloomers" really did bloom! [Rosenthal1982]

A self-fulfilling prophecy in software testing manifests itself as a belief about the outcome of the testing held by the tester which subconsciously influences:

1. The kinds of test planned and the choice of test cases.
2. The way the tests are executed, the choice of data collected and how they are used and interpretation; specifically, it becomes more likely that data which supports the prophecy is collected and used while contrary data is not.

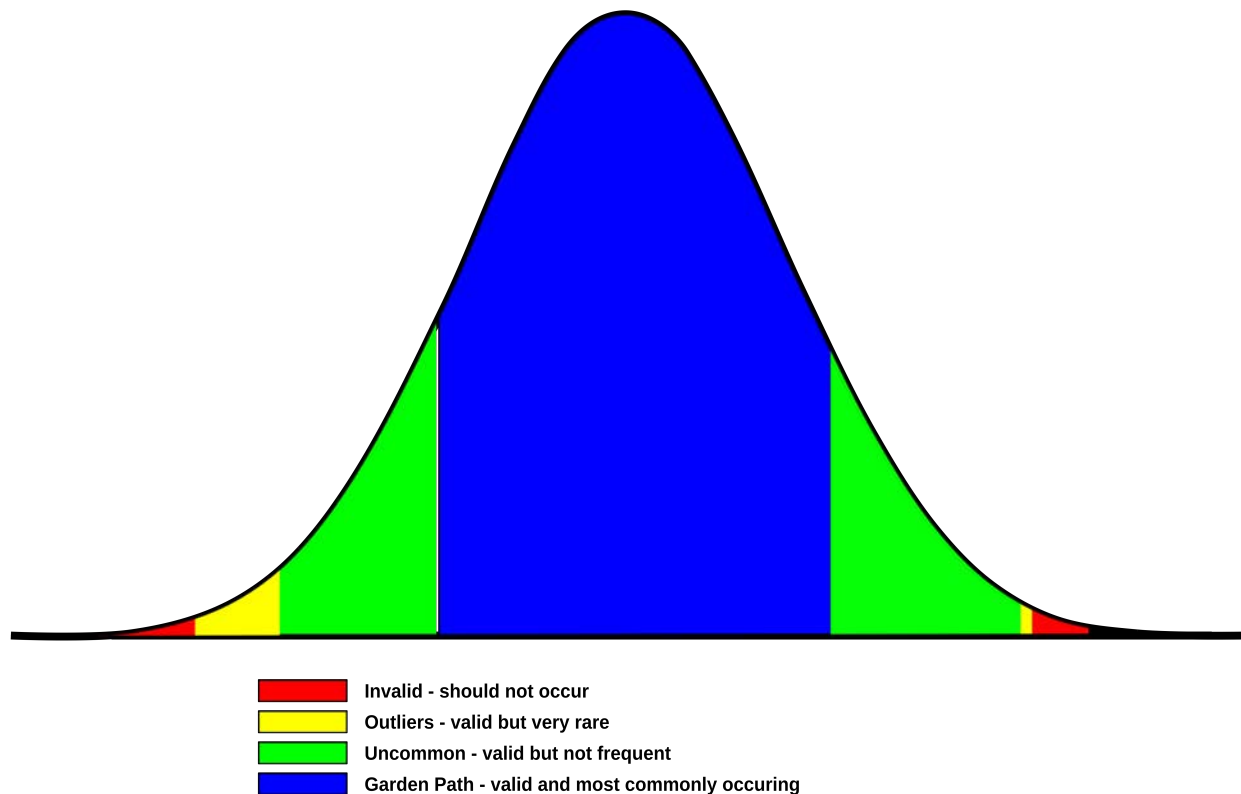
In software testing, what the tester defines testing to be often influences the tester to subconsciously bias the testing process to certain outcomes.

If testing is defined as To prove that a program functions correctly or that a program is error free, then there will be a tendency to choose test data that would not cause the program to fail.

If testing is defined as To prove that a program fails or crashes, then test data will be selected which will cause the program to crash, but the test data may not demonstrate the programs behaviour in cases where the program does not fail; specifically cases in which the program does not crash but produces invalid output.

3.4.2 Following the Garden Path

Another trap that testers can fall into is to test with data representative of the typical or most commonly expected valid data as opposed to unexpected or invalid data. Test data which may reveal the existence of program deficiencies are ignored because “that case would never happen in real life”. In other words, the tester chooses the “garden path” or most “normal” form of data.



Testing with data which is “realistic” or is statistically most representative of the mainstream cases is not as useful in finding bugs in programs as the extreme cases. (Note that will be using this fact to design test cases in the next module, but for now try and figure out for yourself why this would be so.)

This deficiency in testing can also arise from inaccurate generalizations about the population the input data will come from based on a non-representative sample. This is most likely to occur when the tester is unfamiliar with the data domain.

Thought exercise:

You are working as tester for a company that has developed a database for the Department of Homeland Security tracking adult American residents traveling abroad. The database contains fields for first name, last name, middle initial, gender, date of birth, citizenship, date of exit from the US, point of departure, expected date of return, and state of residence. The required fields are first name, last name, a country of citizenship, date of exit and point of departure; and one of SS Number, Green Card ID or Passport Number. All the other fields are optional.

What are “Garden Path” assumptions that have been made by the developers?

3.4.3 Selective Perception

Selective perception occurs when we respond to what we believe or expect to be there rather than what we actually perceive. This is the result of our brain “jumping to a perceptual conclusion” based on the identification of a few salient features.

For most of us, perception is reality. Yet, our perceptions do not always mirror reality, they are heavily influenced by our expectations and prior experiences. We observe stimulus with a preconceived notion of how it should appear and find it difficult replacing that notion with the facts. Instead of reality, we “selectively perceive what we expect and hope to see”. This biased phenomenon is defined as selective perception [Meisenhelder1]

Selective perception makes it next to impossible for us to proofread our own writing and to test our own code. Interesting fact: to avoid selective perception problems, professional proofreaders proofread text backwards.

For example, Jerome Bruner and Leo Postman (1949) completed one of the first experiments on selective perception. In their experiment, subjects reviewed a set of playing cards and were asked to distinguish between normal and ‘trick’ cards. A black three of hearts, for example, was a ‘trick’ card since it was not displayed in its traditional form. Bruner and Postman “found that it took people more than four times longer to recognize a trick card than a normal card” (Plous, 1993). Since the trick cards were unfamiliar, subjects responded in a state of ‘perceptual denial’ in which they saw the cards as they expected to see them. In the case of the black three of hearts, subjects perceived the trick card as either a normal three of spades or a normal three of hearts. Bruner and Postman contended that previous experiences and familiarity drove the respondents to draw erroneous conclusions. [Meisenhelder]

The relevance for testing is that testers have been known to declare that tests pass, i.e. the expected outcomes match the actual outcomes, but later a review of the results shows that this was not the case. The testers did not see the failed test cases when they examined the results of the tests because they expected to see a “success” result.

3.4.4 Obedience to Authority

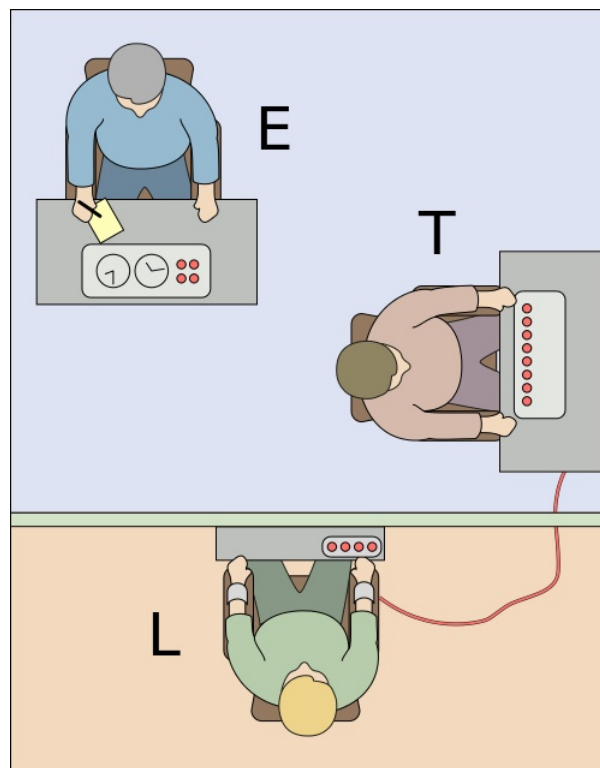
We have all heard the justification “I was just following orders”. There has been some interesting psychological research into whether the natural tendency to be obedient to authority can be strong enough that it causes a person to perform socially unacceptable acts, such as murder, or even faking software testing results (gasp!) for example.

In 1963, Stanley Milgram, a social psychologist performed a famous study on the effects of obedience which to this day is regarded as one of the most dramatic social psychology studies ever undertaken.

The Milgram study

Here, in Stanley Milgram's own words is the study:

In the basic experimental design two people come to a psychology laboratory to take part in a study of memory and learning. One of them is designated "a teacher" and the other a "learner." The experimenter explains that the study is concerned with the effects of punishment on learning. The learner is conducted into a room, seated in a kind of miniature electric chair, his arms are strapped to prevent excessive movement, and an electrode is attached to his wrist. He is told that he will be read lists of simple word pairs, and that he will then be tested on his ability to remember the second word of a pair when he hears the first one again. whenever he makes an error, he will receive electric shocks



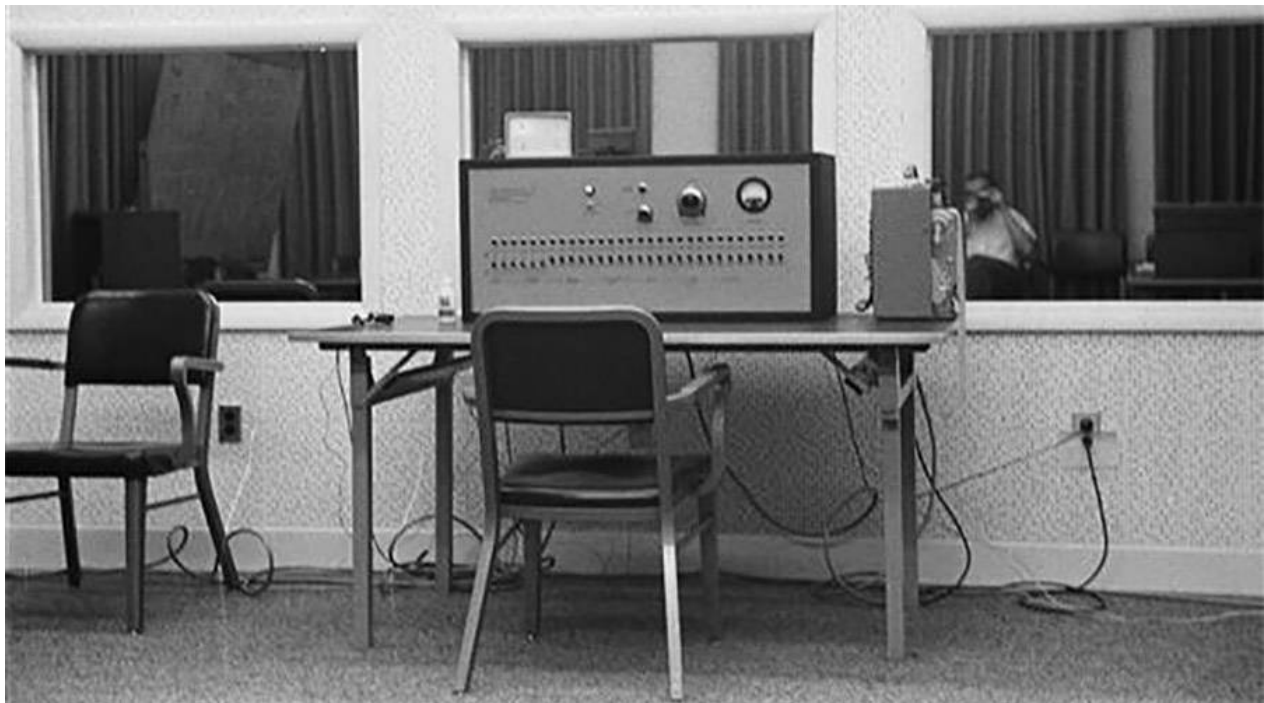
of increasing intensity.

The real focus of the experiment is the teacher. After watching the learner being strapped into place, he is seated before an impressive shock generator. The instrument panel consists of thirty lever switches set in a horizontal line. Each switch is clearly labeled with a voltage designation ranging from 14 to 450 volts.

The following designations are clearly indicated for groups of four switches. going from left to right: Slight Shock, Moderate Shock, Strong Shock, Very Strong Shock, Intense Shock, Extreme Intensity Shock, Danger: Severe Shock. (Two switches after this last designation are simply marked XXX.)

When a switch is depressed, a pilot light corresponding to each switch is illuminated in bright red; an electric buzzing is heard; a blue light, labeled "voltage energizer," flashes; the dial on the voltage meter swings to the right; and various relay clicks sound off.

The upper left hand corner of the generator is labeled SHOCK GENERATOR, TYPE ZLB. DYSON INSTRUMENT COMPANY, WALTHAM, MASS., OUTPUT 15 VOLTS -- 450 VOLTS.



Each subject is given a sample 45 volt shock from the generator before his run as teacher, and the jolt strengthens his belief in the authenticity of the machine.

The teacher is a genuinely naive subject who has come to the laboratory for the experiment. The learner, or victim, is actually an actor who receives no shock at all. The point of the experiment is to see how far a person will proceed in a concrete and measurable situation in which he is ordered to inflict increasing pain on a protesting victim.

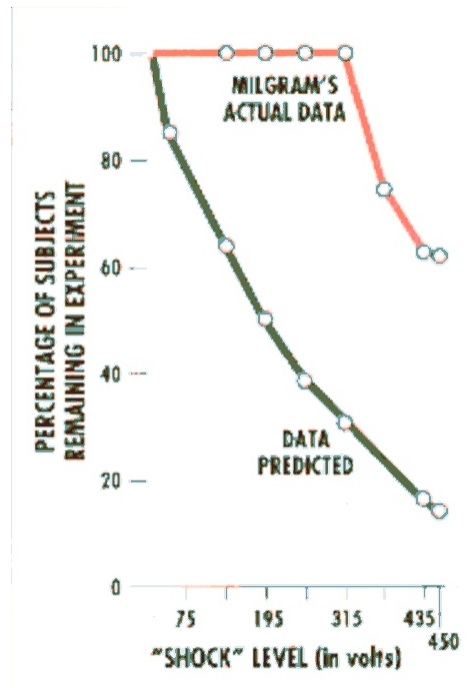


Conflict arises when the man receiving the shock begins to show that he is experiencing discomfort. At 75 volts, he grunts; at 120 volts, he complains loudly; at 150, he demands to be released from the experiment. As the voltage increases, his protests become more vehement and emotional. At 285 volts, his response can be described only as an agonized scream. Soon thereafter, he makes no sound at all.

For the teacher, the situation quickly becomes one of gripping tension. It is not a game for him: conflict is intense obvious. The manifest suffering of the learner presses him to quit: but each time he hesitates to administer a shock, the experimenter orders him to continue. To extricate himself from this plight, the subject must make a clear break with authority.

The Unexpected Outcome

Before the experiments, I sought predictions about the outcome from various kinds of people -- psychiatrists, college sophomores, middle-class adults, graduate students and faculty in the behavioral sciences. With remarkable similarity, they predicted that virtually all the subjects would refuse to obey the experimenter. The psychiatrist, specifically, predicted that most subjects would not go beyond 150 volts, when the victim makes his first explicit demand to be freed. They expected that only 4 percent would reach 300 volts, and that only a pathological fringe of about one in a thousand would administer the highest shock on the board.



These predictions were unequivocally wrong. Of the forty subjects in the first experiment, twenty-five obeyed the orders of the experimenter to the end, punishing the victim until they reached the most potent shock available on the generator. After 450 volts were administered three times, the experimenter called a halt to the session. Many obedient subjects then heaved sighs of relief, mopped their brows, rubbed their fingers over their eyes, or nervously fumbled cigarettes. Others displayed only minimal signs of tension from beginning to end.

Implications for software testing.

One of the pressures that software testers are under is testing while being ordered to make sure that a piece of software is proven to be compliant or safe or bug free, perhaps in order to make a release date. In this case the software testers are no longer actual testers, but are expected by the authority to rubber stamp their approval.

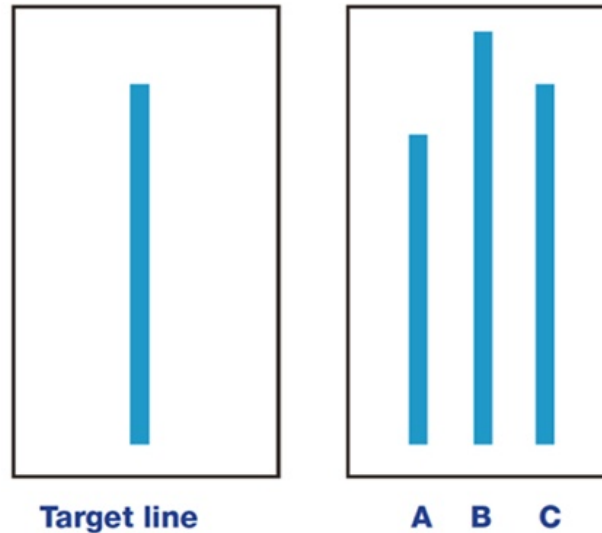
I personally have been face to face with senior management and told that they cannot understand why my testing team cannot just take the programmers word for it that there are no bugs in the software because there is a release date to be met.

3.4.5 Conformity to Peer Pressure

Solomon Asch in 1951, 1956 and 1958 demonstrated the powerful effects of peer pressure, or yielding to group pressures even when no specific request to do so is made. While not as dramatic as Milgram's study, only about 75% of his subjects displayed conforming behavior, it demonstrated the extent of subtle group pressures.



In Asch's study, a subject was placed in a room with six other people who portrayed as other subjects but were in fact experimenters. The subjects were told they were taking part in a perception study. They were then shown a series of cards with three vertical lines on them and asked to select the two lines that were closest in length. The real subject always answered last.



After a while, all of the fake subjects started to agree on what were clearly wrong answers, which then placed the real subject in the dilemma of either reporting accurately his perceptions contrary to the rest of the group, or conforming to group pressure and giving the same answer as the rest of the fake subjects had given, even though he knew it to be false.

In the original study in 1958, 37 of the 50 subjects conformed to the majority at least once, and 14 of them conformed on more than 6 of the 12 trials. On average, when confronted with a unanimous wrong answer by the other group members, the subject agreed with the group 32% of the time.

What is the explanation for this conforming behavior? After the experiment, most of them the subjects who conformed said that they knew their answers were wrong and did not believe their conforming answers, but agreed with the group out of fear of being ridiculed or thought "peculiar."

Other studies indicate that people conform for two main reasons. First, is to be accepted and liked by the group. The second reason, which is more important to us as software testers, is because they believe the group is better informed than they are.

Imagine a junior software tester in a meeting with a number of senior programmers, testers and software designers. The unanimous opinion of the group is that the software is functioning to spec. There is pressure on the software tester to ignore the stack of test results in front of him and agree with the group.

But can this ever really happen? Yes. When I worked as the manager of a software R&D company, I received glowing test results from our senior developer (we will call him Tom), who was also responsible for the testing function, which seemed to indicate the software was working perfectly.

When our client, a federal government department, sent out their assessment team to see the results of their research grant to us, Tom came to my office and told me the software didn't really work and he had faked the test results. I asked him why and he told me he could not deal with the negative responses from his colleagues when he produced bad test results when they all believed the software worked perfectly. It was not a good day.

3.5 The Way of the Tester

Software testing is not the only profession that has to worry about these sorts of human errors: teachers, doctors, therapists, researchers, investigators and a host of other professions have to deal with exactly the same effects. The reason is that these are because of both the way our brains are wired to process information and also because of the way we have learned to work and interact with others.

So how do we deal with it? There are a couple of commonly used ways to counterbalance these effects. Some of these are

3.5.1 Formal Protocols

Most of the errors we looked at are artifacts of how our brain processes information or how we work in social groups. We cannot just decide to not be involved in a self-fulfilling prophecy, for example, because the effect is either taking place below our threshold of awareness or we are incapable of acting in other ways.

We often use a formal and rigorous set of protocols so that everything is done in a standardized and uniform way by everyone. A good example is the double blind protocol for medical testing to avoid the experimenter and placebo effects. What a standardized protocol does is remove the “wiggle room” we have in making decisions.

There are several reasons for doing this. The first is that if we look at a lot of the errors we make as testers, they occur because we make subjective decisions. We can control the error introduced by the subjective decisions by having an explicit and formal way of making decisions that removes the subjective component.

The second reason is that by formalizing and externalizing the decision making process, others can review and identify possible errors we made by checking out decision making processes.

Consider for example the problem of defining what underage drinking is. The intent is generally stated as ensuring that people who drink are mature enough to make good decisions about alcohol consumption. In the real world, there are some people who are mature enough to make good decisions about alcohol at fifteen and others who can't make good decisions at thirty. If we pass a law that says you have to be mature enough to drink, we have chaos.

Now the decision of whether someone is mature enough to drink is left up to a subjective judgment call by a police officer, and different officers will undoubtedly make different calls, which is generally goes against the idea that the law should be applied equally and fairly to all. What we do in this sort of case is remove the need for the officer to make a judgment by stating that a person must be at least eighteen – or twenty-one or whatever age is chosen in that jurisdiction – to be considered mature enough to drink. Now the process is objective and removes the potential for bias on the part of the individual officers.



It should be pointed out that this is not a perfect solution since not everyone is suddenly able to make mature decisions when they turn twenty-one, but what it does do is remove subjectivity and bias from a decision making process, the advantages of which tend to outweigh the disadvantages.

Several of the techniques we use as part of our formal protocols are:

1. Algorithmic techniques as opposed to intuitive techniques. We will this in the Myers triangle problem coming up later in this module.
2. Engaging in reviews and evaluation of the testing process itself.
3. Following accepted principles of software testing. We will look at some of these in just a moment.

3.5.2 Formal and Repeatable Processes

All testing is planned and documented so that it is repeatable in the same way that a science experiment is documented so that it also is reproducible. We have standardized ways of working with the testing life cycle and processes in order to have reliable testing. When ad hoc or informal testing is being done, we often lose the quality or reliability.

We use formal and standardized processes for the following reasons.

1. It allows for repeatable tests and reliable results.
2. It allows for formal reviews of the testing process.
3. It allows us to use testing resources more effectively including the use of automated tools.
4. It is essential for testing process improvement.

It's not just enough to follow a rigorous process but we also need to be able to review and improve the process. There are three parts to this.

The first is to document everything. Keep in mind Demming's Principle: If you didn't write it down, it didn't happen. This is especially important in an Agile environment where there is constant churn of requirements and continuous change. Without accurate and up to date documentation, it is easy to descend into project chaos.

The second is identifying what the assumptions that are being made in the testing process and documenting them. We will always make assumptions but by identifying and documenting them, we can later go back and see if they may have influenced the results and need to be changed. We may find ourselves also having to document the assumptions of others, such as stake holders and developers.

Third, the testing profession is characterized by an adherence to a methodical and systematic process with as little reliance on subjective observation and recollection as possible. As a result, we try to rely on reviews and having many different eyes on a testing activity. In most organizations, a test case is not ready to be used until it has been critically reviewed by other testers.

3.5.2 Myers Testing Principles

In his 1979 book *The Art of Software Testing*, Myers identified a number of principles that he stated characterized good software testing. These have stood the test of time and form the basis of a lot of our current software testing practices and are still valid for Agile testing. These are reproduced here for discussion.

A necessary part of a test case is a definition of the expected output or result.

Without clearly stating the expected result we run into the problems of self fulfilling prophecies and selective perception – or as some testers put it “the eye sees what it wants to see”. Without being able to compare expected output to actual output, it is easy to miss errors. In the worst possible case, violating this principle could mean a test run which uncovered errors but were then ignored by the tester.

Programmers should avoid attempting to test their own code.

This principle clearly helps testers avoid a number of the problems we have discussed, just as we cannot proofread our own writing, we cannot test our own code. In fact anyone writing tests for their own code is prone to a self-fulfilling prophecy – they are likely to write tests that will be biased to pass.

More importantly, if errors have been introduced by an incomplete or faulty understanding of system requirements, then those same errors would not be caught by test cases designed under the same misunderstandings.

A programming organization should not test its own code.

The influence of conformity and authority are clearly minimized by doing the testing outside the organization which developed the code. In addition, the same types of mistakes made by programmers testing their own code can also occur at an organizational level. Beta testing is one way to implement this principle.

Thoroughly inspect the result of each test.

In many cases where code has failed, postmortems have revealed that earlier tests uncovered errors but that these were overlooked because nobody took the time to study the results. One of the best ways to ensure the results are thoroughly inspected is to use automated testing and reporting tools.



Test cases must be written for invalid and unexpected, as well as valid and expected input conditions.

The natural tendency is to concentrate our testing on typical cases characterized valid and expected inputs. However, programs in production often show errors when they are used in some new or novel manner. When we use invalid or unexpected input conditions, the likelihood of finding errors increases.

Examining a program to see if it does not do what it should is only half the battle. The other half is seeing whether the program does what it is not supposed to do.

Basically this means that a program has to be examined for unwanted side effects. A program which produces the correct results but corrupts the input file is doing something it is not supposed to do and thus still contains errors.

Avoid throw away test cases unless the program is truly a throw away program.

With an undisciplined software testing approach, test cases are created on the fly. The tester sits at a terminal, generates test input and submits it to the program. The test data disappear when the test is complete. This sort of approach just doesn't work in an Agile environment. The test cases are now part of the set of development artifacts and need to be subject to the same level of configuration management and control as the source code.

Such throw away cases represent a lost resource. Test cases should be saved and recycled for several reasons. First, when the system is modified, the saved tests can be used to test for all the scenarios identified in the original test plan and thus avoids the temptation to test only the newest additions. Second, saved test cases allow for regression testing to ensure that new features have not introduced unwanted side effects.

Do not plan a testing effort under the tacit assumption that no error will be found.

Testing should be viewed as a process which locates errors and not one that proves the program works correctly. To do otherwise opens us to a self fulfilling prophecy. By using a formal protocol, we can eliminate the sort of assumptions that would bias a test planning effort.

The probability of the existence of more errors in a section of a program is proportional to the number of errors already found in that section.

Meyers admits this result is counter intuitive but empirically verifiable. Basically this principle states that errors come in clusters. For example, in the IBM 370 operating system, 47% of errors found were located in 4% of the modules in the system. One possible explanation is that the rate of finding errors may be proportional to the of time that a section of code spends in execution compared to other sections. However as we shall see it is more likely that programmers tend to make the same mistakes over and over at specific points in the code.

Testing is an extremely creative and intellectually challenging task.

Historically, testing has been viewed as something of a lower level function in software organizations with the programmers occupying higher rungs on the corporate food chain. The influence of this view is often seen by the lack of resources that testers have to work with or the assumption that it is better in the long run to cut back on testing in order to give the programmers more time to code.

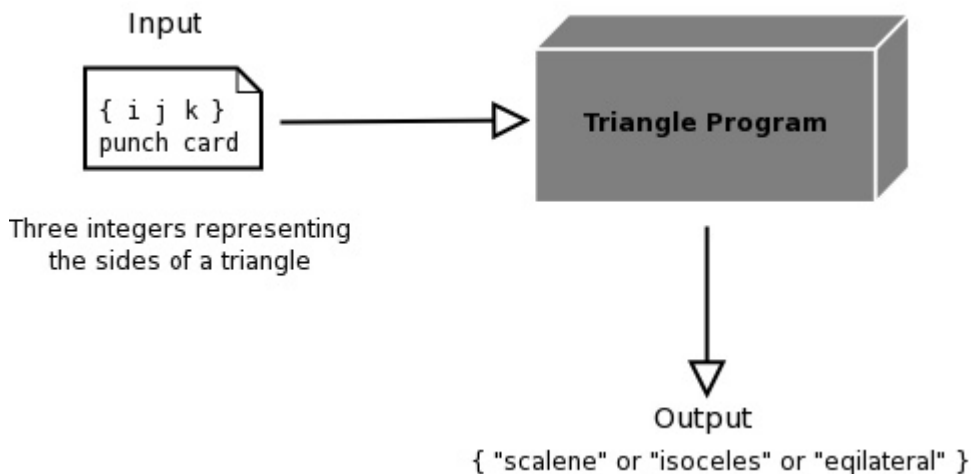
In one case, a company offered their testers a bounty of \$25 for each bug found. Much to their amazement, the level of discovered bugs sky rocketed. Or at least until they found that testers were paying programmers \$10 for each bug they deliberately placed into the code.

Testing is the process of executing a program with the intent of finding errors.

A good test case is one that has a high degree of probability of detecting an as-yet undiscovered error. A successful test case is one that detects an as-yet undiscovered error. This is one place where Myers is out of date. This is more a description of the sort of traditional

3.5.3 The Myers Challenge

To illustrate these concepts, we introduce a thought problem to illustrate what “thinking like software tester” looks like. Glenford Myers introduced the classic book *The Art of Software Testing* by challenging the reader to come up with a plan to test a program that:



reads three integer values from a card. The three values are interpreted as representing the lengths of the sides of a triangle. The program prints a message that states whether the triangle is scalene, isosceles, or equilateral. On a sheet of paper, write test cases (i.e., specific sets of data) that you feel would adequately test this program.

Types of Testing

Module Four

We cannot solve our problems with the same thinking we used when we created them

Albert Einstein

It's more about good enough than it is about right or wrong.

James Bach

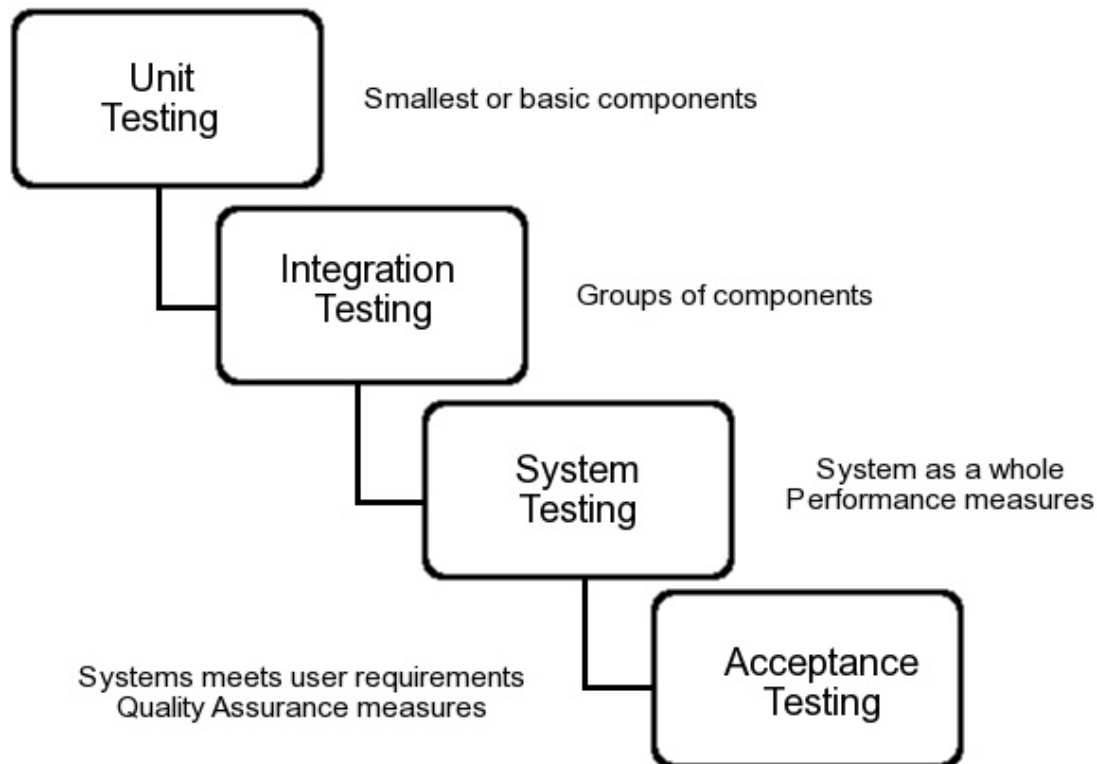
Ignorance more frequently begets confidence than does knowledge

Charles Darwin



4.1 Levels of Testing

Software testing is usually described as being carried out at different levels. For most systems, there are usually 3 to 4 levels of testing, each carried out in its own phase.



The “V” Model

The V model of testing appears in various forms. The one in the diagram on the next page is fairly typical of the general model.

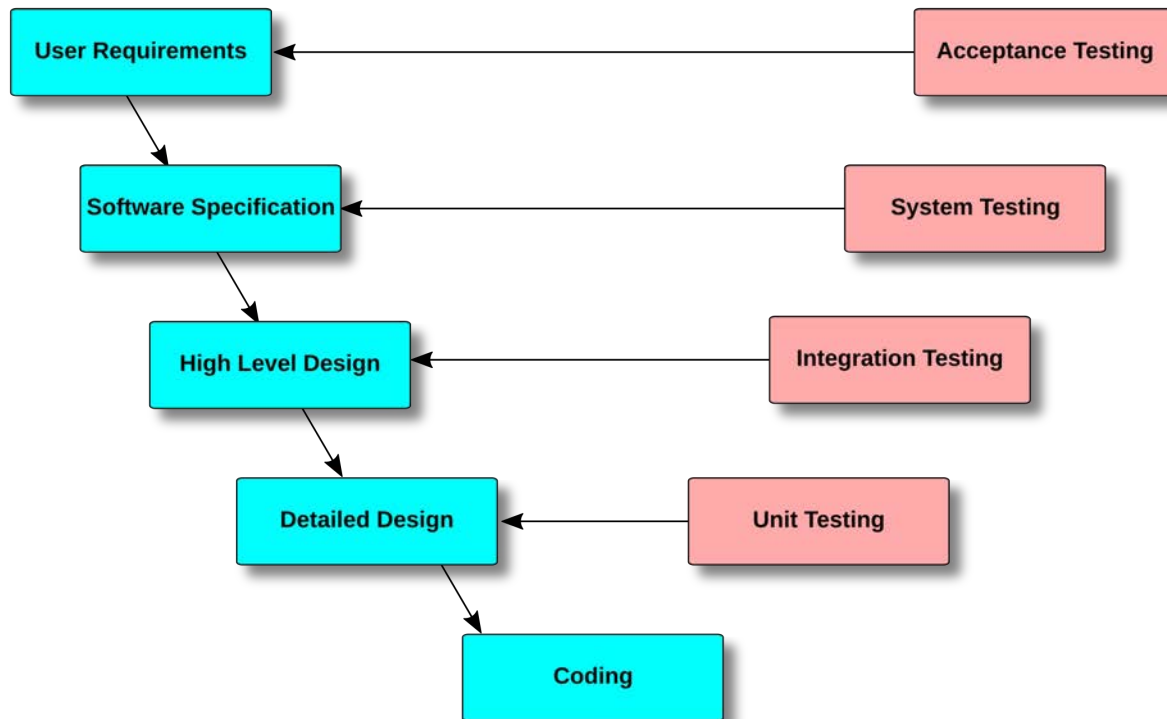
There is relationship between the level of development on the left and the level of testing we do on the right. More specifically, there is a dependency relationship between the testing level and the artifacts that are produced on the left at the corresponding level.

For example, acceptance testing depends on the requirements for validity; “In the stakeholder’s opinion, does the system as built meet their requirements?”

System testing depends on the specification “Does the system perform to the specified performance criteria?”

Integration testing depends on the architecture “Does the system contain the complete and correct functionality?”

Classical "V" Model of Standard Testing



Unit Testing

Unit testing depends on the interfaces and functional responsibility of the individual components. "Does each component satisfy its designed responsibilities? However, in this course, I will make the argument that unit testing is not an actual level of testing but rather a testing strategy that can be used to simplify complex testing problems.

Integration Testing

Integration testing is the testing of collections of components to ensure they are essentially "all fitting together" correctly.

Integration tests explore the interaction and consistency of successfully tested components. The reason we do integration testing is to identify specifically those bugs that are observable in the interaction between two components but not when each component is tested individually.

System Testing

System testing is done to explore systems behaviours that cannot be tested by unit, component or integration testing. This generally refers to non-functional testing, however, it can refer to any characteristic of the software that is a property of the system as



a whole and not some component.

Functional Testing

Most of what we do at the component, integration and unit testing is functional testing, and functional testing does account for a majority of the actual testing that is usually done. Modules five and six will be deep dives into testing functionality using black box and white box methods.

However, there is still something called "end to end" testing which is the overall functional testing of the whole system. This is usually done prior to delivery to the client to ensure that everything works as it should.

Volume Testing

The system is subjected to heavy loads of data in order to test for failures at high volumes. This testing is intended to find out whether the system can or cannot handle the volume of data specified in the specification.

Stress Testing

The system is subjected to the maximum capacity it is specified to handle for a specified period of time. Peak capacities of systems are usually stated in terms of a percentage of normal or average loads. For example, the telephone system peaks at 125% of normal capacity in the period right after the start of business. Stress tests are often avoided using the excuse that "that could never happen".

Usability Testing

Looks for ergonomic or user interface flaws. Such as: Is the user interface appropriate to the educational or training level of the users? Do the messages displayed by the system actually make sense or do they use jargon or unintelligible terms? Do error messages fail to give clear, corrective action? Are there inconsistencies in the user interfaces of the system? Does the system provide enough redundancy to prevent input errors, especially unrecoverable errors? Are there any system options which are useless to the user? Is there a failure to acknowledge user input? Is the system ergonomic or is it difficult to use.

However, as systems have moved into more mission critical and real time environments, and as they have gotten more complex, the issue of usability is becoming a very important aspect of testing.

Security Testing

Traditionally, security testing deal with attempts to find holes in the systems security procedures, such as password protection, file locking, data protection, memory protection and virus protection. However, over the last few years, security testing has exploded into as a discipline as hackers, criminal organizations and state actors are mounting

more frequent, sophisticated and powerful attacks.

Performance Testing

Attempts to locate areas where the system does not meet performance specs such as response time, CPU time, and throughput.

Data Testing

Attempts to find where storage objectives are not met. This is becoming increasingly important as we start to work with petabytes of data with technologies like Big Data. However, it's not a matter of how much data can be stored, but how much can be accessed and the performance issues associated with data transfer and bandwidth.

Another emerging form of data testing is the risk testing to ensure that data is accurate, timely and precise.

Configuration Testing

Tests for failures of the system to perform under all of the combinations of hardware and software configurations allowed in the specification, and in an increasingly interconnected world, how our system under test interacts with other systems. These sorts of questions are becoming very important as we see interactions taking place between autonomous agents and the rise of the IoT or Internet of Things.

Compatibility Testing

Checks with non compatibility with other parts of the existing organizational IT infrastructure and any connectivity requirements imposed by interactions with external systems.

Installability Testing

Tests for failures to meet specification regarding the installability of the system as well as the ability of the system to upgraded as required.

Reliability Testing

Tests for failures to meet specific reliability specifications. For example, a system may have a reliability specification of two hours down time per forty years of operation. Clearly this is difficult to test however there are mathematical models borrowed from engineering which can be used if the system has operational specification like mean-time-to-failure (MTTF = 20 hours), of no more than eight errors after being placed into production

Recovery Testing

Systems may fail for a variety of reasons and one important specification is how the system will recover from a disaster. A recovery test would check to see if the system recovered form a disaster according to spec.



Regression Testing

Regression testing is not actually a level of testing, but is a critical phase of testing and a key concept in testing. Regression testing is the retesting that occurs when changes are made to a system to ensure that the new version of the software still has the required functionality the previous version and no new bugs have been introduced as a result of the changes. The changes could be the result of an enhancement or as the result of a bug fix.

Consider for example, the following description by Andrew Tanenbaum on the IBM OS/360 which illustrates the endless loop that we can get into without doing any real regression testing.

There was no way that IBM (or anyone else) could write a piece of software to meet all those conflicting requirements. The result was an enormous and extraordinarily complex operating system, probably two to three orders of magnitude larger than FMS. It consisted of millions of lines of assembly language written by thousands of programmers, and contained thousands upon thousands of bugs, which necessitated a continuous stream of new releases in an attempt to correct them. Each new release fixed some bugs and introduced new ones, so the number of bugs probably remained constant in time. One of the designers of OS/360, Fred Brooks, subsequently wrote a witty and insightful book describing his experiences with OS/360. While it would be impossible to summarize the book here, suffice it to say that the cover shows a herd of prehistoric beasts stuck in a tar pit. [Tann87:9]

Regression testing should occur at every level of testing. However, in order to regression test a system, test cases, test procedures, and other test-related items from previous tests should be available so that these tests can be re-run with the new versions of the software.

Acceptance Tests

In all other testing, the role the user plays is limited to a source of specifications. In acceptance testing we deploy the system in the production type environment and let the users “test drive” the system.

Acceptance testing moves the test criteria from “does it work” to “does it work for you the user.”

One place we find acceptance testing particularly important is when a piece of software is contracted out to another organization and the final product has to be tested to see if it matches the contracting specifications.

Since acceptance testing sits on that fuzzy grey zone between testing and quality assurance, we will deal more with it later in this course.

4.2 The Unit Testing Strategy

Consider what used to be an annual Christmas related testing problem. At one time, when you had a string of Christmas lights, when one bulb burned out the whole string stopped working. Every year as people put up their tree, they spent hours trying to find the burned out bulbs.

The testing strategy that many often turned to was to throughout the existing strand of bulbs and just go and buy a new strand. However we will just assume for now that replacing the strands is not an option.

The brute force approach is to take a working bulb and replace each bulb in turn until the strand works. If there are 10 bulbs in the strand, then this should take at most 10 tests to find the burned out bulb

1. But what if there are two burned out bulbs? After the initial ten tests, we have to replace pairs of bulbs, then we need another 45 tests. If there are three burned out bulbs, then we need another 220 tests to test all triplets.
2. What if all the bulbs are working, and there is a break in the wire of the strand, then no amount of testing bulbs will find the error.

Notice that the issue is that when we are testing a bulb, the fault could be either an error (burned out bulb) or an exception (another bulb or the wire is the problem).

In our unit testing strategy we take the component to be tested, a bulb, and put into a test environment that we control so that there are no exceptions. In other words, if a test fails, then it must be because of the component we are testing (the bulb in this case).

When we isolate a component like this, we call it a “unit.”

How do we do this with our Christmas lights? First we get a working strand and then for each bulb in our original strand, we put that bulb under test into the new strand. If the strand goes out, the bulb is faulty. A couple of things to note:

1. With 10 tests, we can identify ALL of the burned out bulbs.
2. If the new strand works for all the bulbs, we can assume all the bulbs are good and there is a fault in the wire of the strand,

The whole purpose of unit testing is to reduce the amount of testing we have to do.

4.2.1 Managing Complexity

Consider a machine that has a single steel part. This machine can fail in one way – the part can break.



Part	1	2	3	4	5	6	7	8
Part 1	P	P	P	P	F	F	F	F
Bolt	P	P	F	F	P	P	F	F
Part 2	P	F	P	F	P	F	P	F

Lets add a second part the same as the first part, and bolt the two together. Now the machine can fail in 7 ways

The unit testing strategy is to break the complex machine into three components and then perform three unit tests. That provides us with enough information to determine where the fault is.

We can use this approach for complex data too.

For example, if we have program that reads an order that looks like this but a lot more complex:

Order

Customer Info

Name

Address

Line Item

Description

Quantity

Line Item

Description

Quantity

So how do we go about testing all the different criteria for this data? We unit test it.

The basic unit test strategy would work like this:

1. Create a baseline set of data that is perfectly valid, as typical as we can make it.
2. Select a level of data and hold the larger conglomerates and lower components constant and apply our testing procedure to the data at that level.
3. Once we are finished at that level, move on to the next level.

For our order above we have nested criteria for:

1. The Order: Must contain exactly one customer information, and at least one Line Item.
2. Line Item: Must contain one description, one stock number, one quantity and one price.
3. Stock Number: An 8 char alphanumeric code that has four letters in caps followed by four digits.

And so on.

We construct our baseline order and then hold the lower level components constant (notice that the part of the order that we hold constant – the environment- was designed to have no errors). Now we can construct tests at the order level. For example:

1. Testing with two customer information blocks, testing with no customer information block.
2. Testing with no line item blocks.
3. Testing with duplicated line item blocks.
4. Testing with the customer information block after the the first line item block.

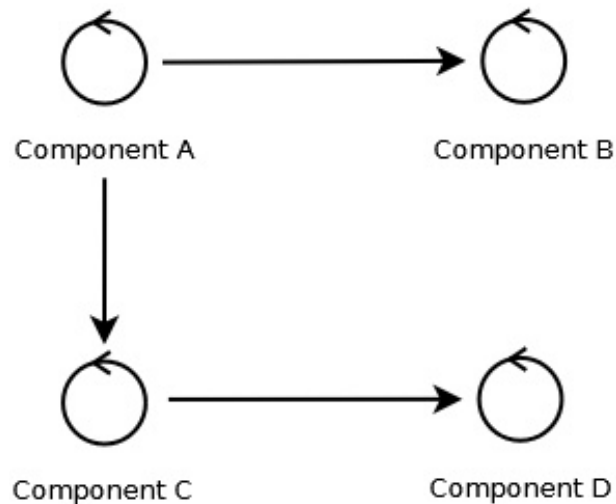
Then we can work down to the next level of structure.



4.3 Integration Testing

In unit testing, we take things apart. In integration testing, we put them back together.

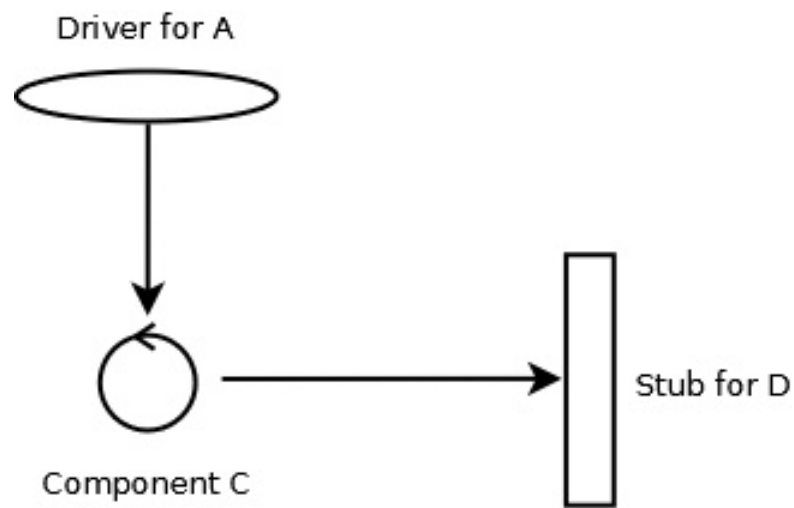
Consider the following system



A calls B and C, and C calls D. To unit test A, we replace B and C with two “stubs” which are components that have the same interface as B and C but always return exactly the right values for each test case.

To unit test B, we replace A with a “driver” which always passes exactly the right data to B for each test case. To unit test D, we need a driver to replace C. And to unit test C we need a stub for D and a driver for A.

For example, unit testing C would look like:



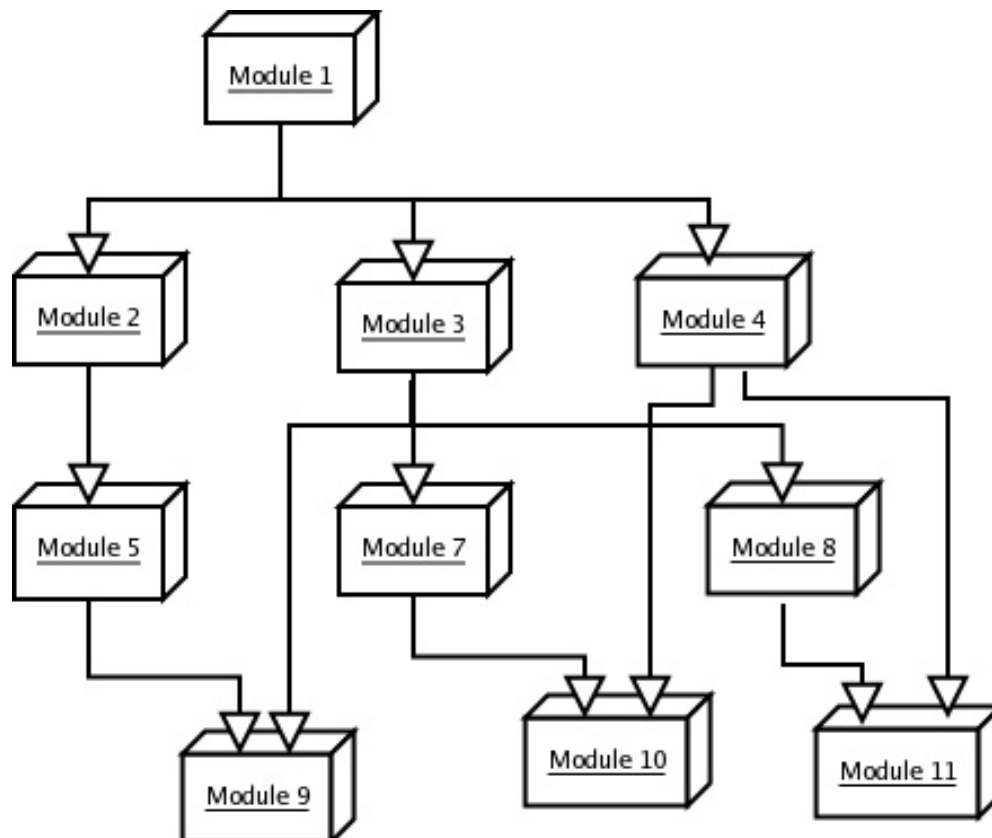
Integration testing is a term that is used to describe strategies for testing large systems. Integration testing means breaking down the system into a number of independent modules. These modules are then component tested and aggregations of modules integration tested until the whole system is completely tested.

What makes the integration testing strategy so effective is that it is not tied to any particular testing method and it can be applied recursively.

Integration testing strategies are used to test large systems where it no longer becomes feasible to test the whole system at the same time since test case design becomes too complex to be effective. Any error found during such massive testing would be hard to debug because it could have occurred at so many places in the system. The amount of test cases required also may grow exponentially with program size.

The advantage of integration testing is that it reduces the complexity of testing into manageable chunks and it allows for more accurate diagnosis of errors.

First we need to understand what a hierarchy chart is. Simply put it is a simple diagram that shows the modules under test and which of the modules call which other modules.



In the preceding chart, module 1 calls module 2, 3 and 4; module 2 calls module 5 and module 3 calls module 7. It doesn't really matter what the modules are, we are just interested in their dependency relationships.

4.3.1 Big Bang Testing

Big bang testing, or non-incremental testing, allows for individual testing of the individual modules then combines them all for a final integration test or big bang.

Advantages: Big bang testing has the advantages of integration testing, and allows a number of teams to test all of the modules in parallel.

Disadvantages: There is a large overhead, since a large number of drivers and stubs have to be written for the module tests. Errors discovered in the final big bang are hard to diagnose since all of the interfaces between modules are being tested at the same time.

The steps

1. Examine the structure or hierarchy chart for the system. Component test each module one by one. This implies that for each module being tested, a driver or dummy calling routine has to be written, and one stub to replace each actual called module.
2. Writing a driver means constructing a dummy mainline which passes the module being tested the appropriate input data for the test.
3. Each stub has to stand in for a real module called from the module being tested. Writing stubs is harder because the stub has to set the output variables the same way the real module would, otherwise artificial errors can be introduced.
4. When all modules have been unit tested, combine and test the entire system.

There do exist automated tools which can speed up the process of writing drivers and stubs. Also note that in the object oriented paradigms, both drivers and stubs are referred to collectively as "mocks."

4.3.2 Incremental testing

Incremental testing methods conduct tests by beginning with one module, then testing a second module by aggregating it with the first and testing the aggregation. Tests are complete when all the modules have been combined into the system.

There are two basic sequences of module combination in incremental testing: top-down and bottom-up.

Advantages : Incremental testing has the following advantages over big bang testing:

1. Overhead is reduced, less stubs and drivers need to be written.
2. Diagnosis of interface problems is simpler since we are adding only one interface between modules at a time.
3. With incremental testing, once a module has been tested, it is retested every time a new module is added and tested. This increased testing increases the likelihood of finding errors in that module.

Disadvantages

1. Parallel testing is difficult to implement.
2. The testing process takes longer since each incremental test causes a progressively larger program to run.

Top down testing

Top down testing is an incremental testing strategy which tests the top level module first, then progressively adds in lower level modules one at a time. In order to test a module, you have to have already tested at least one module which calls it (except for the top level module which is not called by any other module).

Advantages

1. All of the advantages of incremental testing apply.
2. No drivers need to be written.
3. Top-down testing provides an early working model of the program. This model can be presented to the user as a prototype for early acceptance testing. Design flaws can thus be detected early and corrected.

Disadvantages

Stubs still have to be written, which is a more difficult process than writing drivers.

Steps

1. Start with the top level module. Write a stub for each module it calls and unit test the top level module.
2. Choose any of the modules called by the top level module. Replace the stub with the selected module. Write stubs for all the modules called by the new module. Component and integration test this new aggregate.
3. Select another module called by the top level module and repeat step 2. When all stubs have been replaced by modules, select one of the modules called by the top level module and repeat the process.
4. Continue until all modules have been added and tested.



The sequence of selecting modules is mediated by two other principles

1. Test critical modules as soon as possible.
2. Test I/O units as soon as possible so that test cases can use the system I/O facilities.

Bottom-up testing

Bottom-up testing is a method for sequencing the testing of modules so that the bottom level units are tested first with drivers. units are incrementally tested and added into the system as the tester works his way up the structure chart. In order to be tested, a module must be a bottom level module (i.e. does not call any other modules) or all of the modules it calls must have already been tested.

Advantages

1. No stubs need to be written, only drivers.
2. Testing can be done in parallel with different teams starting at different bottom level modules.

Disadvantages: No early working model of the system is available for early acceptance testing.

Steps

1. Select a module from the bottom level. Where a number of modules are candidates, select a module which is an I/O module or a critical unit. If none are available in the bottom level, select a module which is the fewest levels removed from such a module.
2. Write a driver and component test the module.
3. Choose another module according to the criteria in (1) and test it.
4. After all the modules called by a higher level module have been tested, then the higher level module may be tested. The new aggregation will consist of the module just added, all the modules it calls and a driver.
5. Continue until the top level module is added.



Functional Testing

Module Five

More than the act of testing, the act of designing tests is one of the best bug preventers known. The thinking that must be done to create a useful test can discover and eliminate bugs before they are coded - indeed, test-design thinking can discover and eliminate bugs at every stage in the creation of software, from conception to specification, to design, coding and the rest.

Boris Beizer

Bugs lurk in corners and congregate at boundaries.

Boris Beizer

Good specifications will always improve programmer productivity far better than any programming tool or technique.

Milt Bryce



5.1 Functional Testing

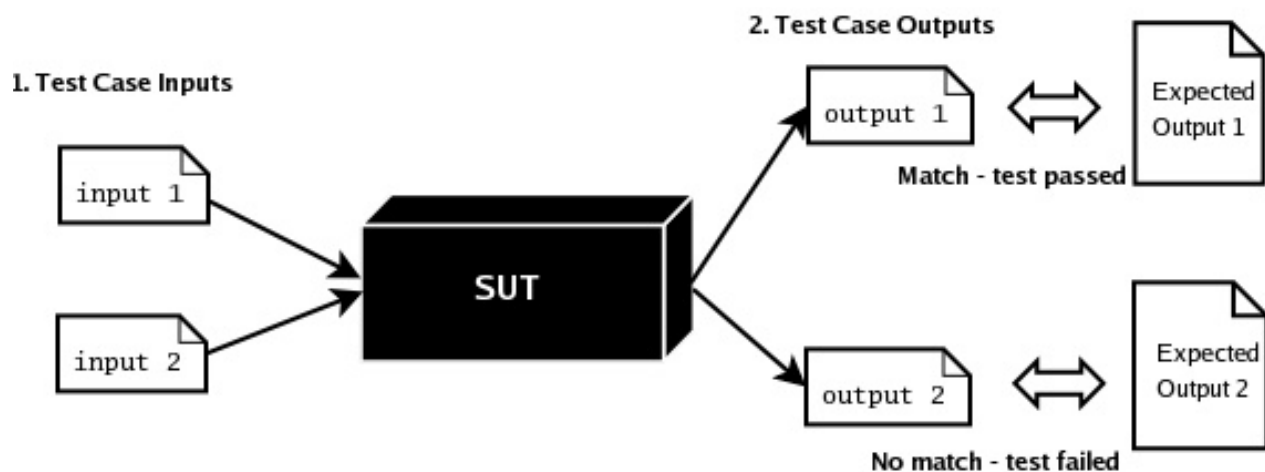
Functional testing, also referred to as behavioural testing and black box testing, is a test methodology adapted from testing methods used in engineering environments.

Functional testing treats the system under test (SUT) as a black box; which means we don't need to know anything about the internal structure or composition of the SUT, because we only care about how the SUT functions or behaves when in operation. Test cases are derived from the specification for the SUT, which is a description of how the SUT is supposed to function.

If there is no specification for the SUT, then we can't do any functional testing. We actually need something stronger than just a specification, we need the specification to be testable – a topic we will explore in the next section of this module.

5.1.1 The Functional Testing Process

The functional testing process is depicted in the diagram and described below.



1. The specification is analyzed to ensure it is "testable." If it is not testable, it is referred back to whoever is responsible for specification for correction or modification to make it testable.
2. Once we have a testable specification, we analyze it to identify and classify the functionality of the system under various conditions and inputs, which helps us identify the types of test cases that will be needed.
3. A set of valid test cases are designed to test the correctness of the system. Each test case includes an expected result based on what the specification says should happen when that test case executes. We can think of these test cases "ensuring the system does what is supposed to do."

4. A set of Invalid test case are designed to test the robustness of the system. As with the valid test cases, an expected result is part of the test case. We can think of these test cases as "ensuring the system doesn't do something it shouldn't."
5. The SUT is executed with both the valid and invalid test cases provided as input.
6. The actual output produced by the SUT for each test case is recorded and compared to the expected output previously calculated for that test case.
7. For each test case where the actual output matches the expected output, the test case is recorded as a pass. When the actual output does not match the expected output, the test case is recorded as a failure.

5.1.2 Functional Testing Advantages

Functional testing is one of the most common testing methods used by testers for a number of reasons:

1. The basic approach can be applied to any level of testing; unit, component or system. In fact, as we get into the larger components, the complexity of the component makes other testing approaches impractical and almost forces us rely more on functional test methods.
2. Functional testing does not require any programming knowledge, unlike structural testing. This means that individuals in non-technical roles (interface designers and end users for example) can participate in the testing process.
3. Because functional tests don't depend on the internal structure of the SUT but just on the described behaviour of the SUT developers can modify the code and architecture of the SUT without the necessity of changing the tests. The tests only need to be changed when the specification is changed.

This property is especially important because having a stable set of functional tests is critical for regression testing and newer Agile techniques like test driven development and acceptance test driven development.

4. Functional testing is intuitive to a degree. For example, you naively used black box methods when posed with the triangle problem in an earlier module without any prompting to specifically use it.

Even though functional testing can seem quite natural, the Myers triangle challenge should have demonstrated that there is a real difference between doing functional testing on an ad hoc manner as opposed to doing it in a way that meets the standards for good testing.

5.1.3 Functional Testing Disadvantages

There are two main issues with functional testing that need to be addressed. There is also a third issue called "coincidental correctness" whose impact is highly over stated but will also be dealt with in this section for completeness.



The Infinite Inputs Dilemma

Thinking back to the Myers triangle challenge, it was immediately obvious that we could not test every single possible input, simply because there are an infinite number. The on-going challenge for anyone doing functional testing is the trade-off between testing effectiveness and testing efficiency, which we can define in the following way:

1. **Test Effectiveness:** The functional test have the highest probability of identifying defects in the system under test.
2. **Test Efficiency:** The functional tests use the limited testing resources, including time, in an optimal manner.

In the two extreme cases, we can get 100% test effectiveness by testing every possible input and condition, but since there are often an infinite number of possible inputs, we wind up with 0% test efficiency. On the other hand, if we do no testing at all, we have achieved 100% efficiency but 0% effectiveness.

Most of this module will be spent exploring the functional testing techniques that have been developed by testers to achieve an optimal balance between efficiency and effectiveness in order to develop the smallest set of tests that returns the best results possible.

Surprise Functionality

In functional testing, we write tests based on what the specification tells us the SUT is supposed to do. But what if there is functionality that is not documented in the specification? In that case we can't write tests for it because we don't know it exists. Functionality that is not documented in the specification is called "surprise functionality" because it's usually a surprise to everyone that it's actually in the SUT.

Surprise functionality can be inadvertent or deliberate. Inadvertent surprise functionality generally is caused by one of more of the following:

1. The specification has been allowed to get out of sync with the actual SUT. A fairly common scenario is when functionality is modified or added during development, and the specification is not updated to reflect these changes.
2. External modules or code has been incorporated into the SUT to reuse some functionality developed elsewhere. The problem is that this reused component has more functionality than what is needed by the SUT and this extra functionality is not disabled or removed.
3. Some functionality is introduced to support the development process, like a testing interface that bypasses the normal login process, or administrator account that anyone can log into. Once the development is completed, this temporary functionality is not removed but is left in the production system.

4. Code that has been deprecated or abandoned because of revisions in the specification – often the result to deciding to drop a feature from the design – is not removed from the code base, so the the functionality persists even though it should not.

All of these issues can be prevented through the use of good code management and project management discipline. When we find this sort of surprise functionality during testing, then we are raising a quality assurance issue by identifying a possible source of error on the part of the development team.

Deliberate surprise functionality is where some malicious actor has inserted code into the SUT that compromises the system for their own purposes. Some typical examples of deliberate surprise functionality are:

1. *System Backdoors*: This is where the SUT security is bypassed in an undetectable manner to give unauthorized access to the SUT or its data.
2. *Logic Bombs*: A logic bomb is code that has been secretly inserted into the code base that will do some sort of damage to the system, deleting files for example, when triggered by either some event, like a target date and time being reached, or by some action by whoever put the bomb in place.

Who puts logic bombs into a system? It can range from employees who use them to get revenge on a company if they are fired to state actors engaged in cyber warfare and criminals using them for attempted extortion.

3. *Trojan Horses*. Typically a Trojan horse is a program that provides some clearly stated and useful technology but also contains code that does something else surreptitious. A typical example of a Trojan horse might be an add on to a browser that provides some convenient functionality, but is sending all of users login information to their banking sites to a malicious actor somewhere.

In general, we cannot identify surprise functionality by using only functional testing. However we can often find this surprise code by using structural testing in conjunction with functional testing.

For example, if our functional tests exercise every possible type of functionality in the specification, and then we analyze the execution of the tests using a tool like a code profiler, we might discover that 5% of the code never executed. If the tests have exercised all of the functionality described in the specification, then that 5% is potentially doing something that is not part of the specified functionality.

In some cases, bringing this problematic code to the attention of the developers can resolve the situation through a simple code inspection. However, when an inspection is not feasible for some reason, we can use structural testing methods to develop a set of test cases that forces this code to execute so that we can evaluate what it does.

Coincidental Correctness



Consider testing a program that produces the square of an integer as its output. However, the programmer made the error of writing $x+x$ in the code rather than $x*x$. If the tester picks the integer 2 as the test case. Since $2+2 = 2*2$, the program coincidentally produces the expected result "4" which results in the test producing a false negative.

The reason this is called coincidental correctness is that the tester just happened to pick a test case that coincidentally produced the expected result. This is effectively a non-issue in functional testing, in spite of the attention it gets, because coincidental correctness is incredibly rare.

But if you are concerned about it, or if your manager is concerned about it, what can be done to prevent coincidental correctness?

We can effectively eliminate the probability of encountering the situation by adding additional test cases or avoiding cases that we know from the specification could produce coincidental correctness. For example, knowing how squares work, we would be leery about using 0, 1 or 2 as our only test cases. Throwing a couple of additional test cases into the mix would definitely catch the underlying defect.

However, we also have to recognize that the situation usually has a low probability of occurring, so we can just learn to live with the reality that we can never completely eliminate the chance of it happening. Instead of going to work of adding the extra test cases.

Generally your choice will depend on the testing quality levels you have determined for the testing activities.

5.2 Specification Testing

So far, we have been referring to a "specification" without actually defining what it is. Before things get confusing, we are going to clarify our terminology about exactly what we mean by requirements, specification and design. Unfortunately, in the real world, these terms are not used consistently, unambiguously or clearly. For this course, we will use each of these terms to mean one thing and only one thing and will use these terms consistently.

The reason that we want to evaluate specification is there is one very simple and universal fact:

If the specification is not good enough to write a test to, then it is not good enough to use as a guideline for construction.

First, recall the Chaos report from module one where the quality of the requirements and specification was consistently cited as one of the most critical determiners of whether a project would be successful.

Second, when we look at where the errors occur that result in defects, most of them occur during the specification phase. Generally these errors are where the developer have had to either "fix up" a bad spec or fill in missing gaps. The problem is that while developers may be good at guessing what the system should do in many cases based on their own experience, they do not have the insight into the underlying reasons or requirements that motivated the specification.

A Quick Example

For example, in the specification for a sales data analysis application, there was a data item called "sales quota" which was expressed in dollars. However, the specification did not describe the range of allowable inputs for sales quota. The programmer made the reasonable assumption that the number can't be less than zero since that didn't make sense; how could you sell -\$100 worth of products.

However, because the developer did not understand some of the rules of the commissions policy for sales people, the developer was wrong in their assumption, which was an error that led to a fault.

The business actually counted returns of a product as "reverse sales" and a negative sales quota of say \$100 meant "Even if you have a sales quota of 0 for this month, you can only accept returns from previous months sales up to \$100 without a penalty on your commission." This was a policy that made perfect sense to the business, but was one the developer did not know about, and there was no reason for them to have known about it.



As Fred Brooks, a famous computer engineer observed:

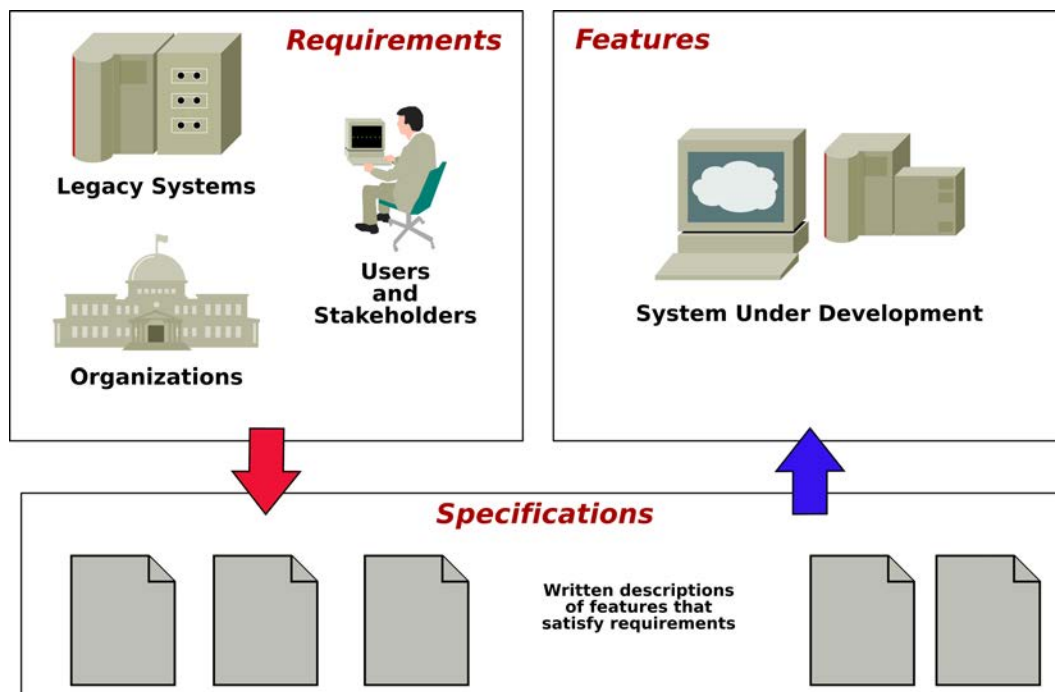
The hardest part of the software task is arriving at a complete and consistent specification, and much of the essence of building a program is in fact the debugging of the specification.

Requirements versus Specifications

One of the first problems we have to sort out is what the term requirement means. According to the IEEE Standard Glossary of Software Engineering Terminology (1990) a requirement is

1. A condition or capability *needed by a user* to solve a problem or achieve an objective.
2. A condition or capability that must be met or *possessed by a system* or system component to satisfy a contract, standard, specification, or other formally imposed document.
3. A *documented representation of a condition* or capability as in 1 or 2.

The term “requirement” now three definitions which all refer to similar but different things. My experience is that this ambiguous terminology can often create confusion in a development project.



For the purposes of this course, we will use the following definitions.

1. A **requirement** is a condition or capability *needed by a stakeholder* to solve a problem or achieve an objective. (“I need to be able to find all the invoices associated with a customer, and I don’t care how you do it, just do it.”)

2. A **feature** is a condition or capability *possessed by a system* or system component to satisfy a contract, standard, specification, or other formally imposed document. (The “search for invoices” button on the customer information screen is a feature.)
3. A **specification** (or feature description) is a *documented representation of a feature*, while a documented representation of a requirement will be called a requirement description. (“The system shall have a search for invoice option which, when clicked, will display a list of all invoices associated with that customer in a new window. The display will be sortable by date issued, date payment received, invoice number and amount”)

Specifications versus Designs

A specification is a description of what the system will look like and how it will behave after it is built. It describes how the finished product behaves, how it functions and how it interacts with other systems and users. The specification describes the goal of the design and construction processes.

A design is a blueprint. A design describes how a specification is to be built given a specific set of resources and a given set of architectural constraints. The specification is architecturally neutral and one specification can be used to produce a number of different designs. A design is always expressed with respect to a given architecture or technology.

For example, a series of screen shots that describe how a web site should look is a specification. To make this into a design we have to choose an architecture, Angular or Drupal perhaps, and then plan the steps to make those screen shots into working web pages using this specific technology. The same specification can also be used as the basis of another design, this one perhaps using .NET or Ruby on Rails.

This distinction is very important if the specification is used as the basis of a Request for Proposal when a software project is being contracted out on a competitive basis. If the specification suggests a design then any of the losing bidders could challenge an awarded contract on the basis that the suggested design in the specification gave another bidder an unfair competitive advantage.

For example, a specification that suggested using Java Server Pages for a web site would be called biased because it gives an unfair advantage to developers who specialize in Java over those that use .NET.

To Summarize

Requirements describe the stakeholders. Requirements describe the needs, wants, wishes and desires of the stakeholders.



Specifications describe the solution. Specifications describe the system to be built. It serves as a design target and a basis for discussion between the stakeholders and developers about what the system should look like and what it should do. It also serves as a baseline for testing and quality assurance. The solution described in the specification should satisfy the requirements of the stakeholders as much as it is feasible to do so within resource constraints.

Designs are blueprints that describe how to build what the specification describes. A design describes how to build the solution given a specific set of resources. This means that a single specification may be implemented by a range of designs. This is the old idea that there's more than one way to skin a cat – presuming that cat skinning can be somehow described by a specification.

Some Other Terminology

There are a variety of different terms that you might come across that describe these three concepts. For example, a specification might be referred to as "executable requirements" or requirements that have been written in a way to describe what the system does to satisfy them. Specifications might also be referred to as "technical requirements" while the requirements themselves are referred to as "user requirement." I have also seen specifications referred to as a "high level design" and the actual design as a "detailed design."

It is important to realize these three concepts – requirements, specification and design – occur all through software development and other types of engineering, but there is no consistent or uniform terminology in use.

One of the basic tasks any tester should undertake is to create a glossary of the terminology used in their organization to see if there might be any form of miscommunication because of inconsistent terminology.

5.3 IEEE Specifications Best Practices

We have made continuous references to the specification so far, but since it is a critical part of functional testing, we now take a closer look at the specification from a quality perspective. The basis of this section is the IEEE "Best Practices for Software Requirements Specifications" which has become a gold standard for software specifications.

The document is not based on a theoretical idea about what makes a good specification, but grew out investigating what historically were the best practices that had been developed in the engineering community.

5.3.1 Defining the SRS Document

This section cites the IEEE document directly and includes excerpted text.

An SRS is the basis of a complete and unambiguous document designed to facilitate the communication of information between the different parties involved in a systems development. The intent, as noted by the IEEE, is to

1. Establish the basis for agreement between the customers and the suppliers on what the software product is to do. The complete description of the functions to be performed by the software specified in the SRS will assist the potential users to determine if the software specified meets their needs or how the software must be modified to meet their needs.
2. Reduce the development effort. The preparation of the SRS forces the various concerned groups in the customer's organization to consider rigorously all of the requirements before design begins and reduces later redesign, recoding, and retesting. Careful review of the requirements in the SRS can reveal omissions, misunderstandings, and inconsistencies early in the development cycle when these problems are easier to correct.
3. Provide a basis for estimating costs and schedules. The description of the product to be developed as given in the SRS is a realistic basis for estimating project costs and can be used to obtain approval for bids or price estimates.
4. Provide a baseline for validation and verification. Organizations can develop their validation and verification plans much more productively from a good SRS. As a part of the development contract, the SRS provides a baseline against which compliance can be measured.
5. Facilitate transfer. The SRS makes it easier to transfer the software product to new users or new machines. Customers thus find it easier to transfer the software to other parts of their organization, and suppliers find it easier to transfer it to new customers.
6. Serve as a basis for enhancement. Because the SRS discusses the product but not the project that developed it, the SRS serves as a basis for later enhancement of the finished product. The SRS may need to be altered, but it does provide a foundation for continued production evaluation.



The SRS provides the basis for a contractual agreement between contractors and customer that defines exactly what the final delivered product is to do without having it being a blueprint or design document.

5.3.2 The SRS as a Legal Document

Any legal contract or piece of legislation needs to be both consistent and complete. There cannot be any loopholes in the legal documents, therefore an SRS needs to have the same amount of rigor that is found in a contract.

The primary purpose of an SRS template is to ensure that nothing is missing or left out of the specification. The SRS template is a completeness checklist.

SRS does not include requirements

A requirement, if you recall, defines what the stakeholder wants the system to do for them, and does not describe the system. Since a requirement describes a stakeholder need, desire or request, it is inherently not measurable and therefore there exists no legal way to decide if a requirement is met – that is a subjective evaluation of the stakeholder.

SRS is not a design document

If the SRS were to contain a design, then if it formed the basis for a legal agreement, the contractor would be bound to the design in the SRS which may prove to be unworkable and not in the best interests of either the customer and contractor. It is a separate issue to contract out development work on a design in which case the specification may or may not be part of the supporting documentation.

According to the IEE recommendation: the SRS

1. Should correctly define all of the system specifications. A system specification may exist because of the nature of the task to be solved or because of a special characteristic of the project.
2. Should not describe any design or implementation details. These should be described in the design stage of the project.
3. Should not impose additional constraints on the system. These are properly specified in other documents such as a quality assurance plan.

Therefore, a properly written SRS limits the range of valid designs, but does not specify any particular design.

The SRS is not a user manual

While a specification may define a system it is not a user manual since it is organized in such a way so as to be essentially unreadable if someone is trying to understand how to use the system. The specification needs to be accompanied by user, support and developer documentation that are developed for those specific purposes.

The SRS is not a project document

A SRS is only a description of the system. It is not a description of the project used by the contractor to build the system. As the IEE recommendation states:

Project requirements represent an understanding between the customer and the supplier about contractual matters pertaining to production of software and thus should not be included in the SRS. These normally include items such as

- a) Cost;
- b) Delivery schedules;
- c) Reporting procedures;
- d) Software development methods;
- e) Quality assurance;
- f) Validation and verification criteria;
- g) Acceptance procedures.

Project requirements are specified in other documents, typically in a software development plan, a software quality assurance plan, or a statement of work.

5.3.3 What the SRS is.

The SRS is a number of things as suggested in the previous section.

The SRS as a Design Target

While the SRS is not a design document, it is the basis for a design. The SRS describes the system to be built and its functionality in enough rigour so that a design can be developed from the SRS.

However, it is very important that a design not be suggested in an SRS since that could be interpreted as giving an unfair advantage to one of a number of parties competing to deliver a design based on the specifications.

However, the IEEE recommendation does distinguish between limiting design and suggesting a design.



The SRS writer(s) should clearly distinguish between identifying required design constraints and projecting a specific design. Note that every specification in the SRS limits design alternatives. This does not mean, though, that every specification is design.

The SRS should specify what functions are to be performed on what data to produce what results at what location for whom. The SRS should focus on the services to be performed. The SRS should not normally specify design items such as the following:

- a) Partitioning the software into modules;
- b) Allocating functions to the modules;
- c) Describing the flow of information or control between modules;
- d) Choosing data structures.

However, in special cases some requirements may severely restrict the design. For example, security or safety requirements may reflect directly into design such as the need to

- a) Keep certain functions in separate modules;
- b) Permit only limited communication between some areas of the program;
- c) Check data integrity for critical variables.

Examples of valid design constraints are physical requirements, performance requirements, software development standards, and software quality assurance standards. Therefore, the specifications should be stated from a purely external viewpoint. When using models to illustrate the requirements, remember that the model only indicates the external behaviour, and does not specify a design.

The SRS as a Testing Target

The SRS is the baseline by from which testers construct their test cases. The software being developed, regardless of design, can be tested to see if it is in conformance with the specification.

The SRS as a Documentation Base

Once the SRS has been developed, it is the primary input for the process of generating all the documentation that is necessary for deployment of the new system. This documentation typically includes user guides, reference manuals, support and service manuals and training materials.

The SRS as Historical Data

Eventually the system under development will have to be modified or replaced. In this case, the SRS is a historical record of what the current system under development does. Since there is no guarantee that those working on future replacements for the system will have any actual knowledge of this system under development, the SRS provides continuity and allows the features and functionality of the existing system under development to be known to future designers.

5.3.3 What Makes a Good Specification

There has been a number of studies as to what constitutes a “good specification.” It was much like the quality issue we saw earlier, no one could say exactly what made a specification “good” but everyone knew a good one when they saw it.

The IEEE eventually consolidated many of the conclusions of multiple studies in the engineering community into a list of properties for software specifications which was validated from their survey of best specification practices across various organizations that were engaged in software development. The material in this section is taken from the IEEE Std 830-1998: recommended practice for software requirements specifications.

More specifically these properties are:

Complete

A specification is complete when it describes how the system will behave under every possible condition it could experience and how it should respond to every possible class of input that could occur, whether or not those conditions and inputs are valid and expected or invalid and unexpected.

Completeness is our “but what if...” analysis. One of the main purposes of completeness is to describe how the system should identify and reject invalid inputs and respond to invalid conditions. Systems built from complete specs display the property of robustness – they are hard to crash.

In terms of our testing, we need to have 100% coverage to ensure completeness. That means we have test cases that cover every possible type of event and input. It is very important to stress we refer to the types of input and not individual inputs. It would be impossible to specify the response to every input or condition, so we look instead at classes of types of inputs and conditions.

If we do not have completeness, then developers do not know the expected behaviour of the system in certain cases and if they write code to those cases then they usually just guess what ought to happen.

One of the important features of completeness is to test to see that the system correctly rejects the cases or conditions that it is not designed to handle.



Consistent

A consistent specification is one where there is no conflict between individual specification statements that define the behaviour of the system in under the same conditions. Inconsistency often arises when two more more stakeholders have different expectations about how the system should respond under the same condition or to the same input, and those different expectations have not been resolved.

In terms of our acceptance tests, this means that if every test case has exactly one result we cannot have inconsistency. If we find two identical test cases that differ in their expected result, we have an inconsistency – a situation that can be rectified as the tests are being developed.

Modifiable

Specifications are modifiable when related concerns are grouped together and unrelated concerns are separated. This characteristic is exhibited by a logical structuring of the test cases and test suites. Simply put, the specification is organized, easy to use and easy to modify.

The only thing that remains constant is that all things change, including requirements or as the The SCRUM Uncertainty Principle states Customers don't know what they want until they see it, and they always reserve the right to change their mind.

Having a modifiable specification means that we can keep the specification current even in the midst of a "churn of requirements."

Correct

For a specification to be correct it must accurately and precisely encode any external business processes, rules and logic so that the system behaviour is consistent with the real word operations. For example, if we are modelling a business process, the model is incorrect if the logic in our specification produces different results than the business process. Incorrectness often emerges when the model is originally correct, but the process being automated changes in the business but the model being used to build the software is not updated to reflect these changes.

Another form of incorrectness is when the business process is not correctly translated into a design model. We will look more at this issue in the module on Model Based Testing.

Correctness is exactly what we have been calling validation. Correctness comes from the stakeholders vetting the correct responses for the set of tests we develop from the specification.

Ranked

The specification items or tests are ranked in a priority order to identify core functionality and to establish a baseline architecture for a design which supports the highest ranked functionality. In general 80% of the value comes from 20% of the features. Architectural choices, design decisions and code development should be prioritized to support that 20%.

The importance of a feature is not determined by the developer but is determined by the value assigned to the benefits it delivers to the user. However the feasibility of a feature in terms of effort and cost can usually only be determined by the developer. Both of these factors have to be considered in a ranking.

Testable

A specification item is testable if it is written in such a manner that pass/fail or quantitative assessment criteria can be derived from the specification itself and/or referenced information. Since we are using tests as the specification items, this is automatic if we write our test cases correctly.

Generally testable means quantified. A non testable specification is The input should not be too large while a testable spec is The input must be greater than 0 and no larger than 500.

However, as we will see later, we may have to develop somewhat different definitions of testable when we are dealing with different kinds of test measurements. More on that later.

Traceable

A traceable specification is one where each described feature is traceable back to the requirement (or user story) that motivated it, and each feature should be traceable forward to the artifact that implements that feature in the design, and that design element, in turn, traces to the code that implements that design element. The tracing forward becomes important when we have to anticipate the effects of changes to system that we might make during an upgrade or enhancement.

This also means that each test in our specification is traceable back to the user story that it originated from, and can also be traced forward to the design component that will provide the functionality described in the test. In agile, forward traceability becomes even more critical for supporting test driven development since the acceptance tests now can be used to generate the unit tests for that design component.



Verifiable

A verifiable specification item is one for which there is finite and cost effective way to test the resulting feature. It is not enough that we can write a pass or fail test for each specification item, we need an efficient way to actually execute those tests. Testability is about being able to write tests, verifiability is about being able to actually run those tests.

Unambiguous

A statement of a specification item is unambiguous if it can be interpreted in only one way. Eliminating ambiguity requires a collaborative approach. Once we see one interpretation, we find it very hard to see other possible interpretations. We see what we expect to see, which is one of the main reasons no one can proofread their own writing (remember selective perception from a previous module. The use of a domain specific language is one common technique to ensure a specification is unambiguous.

Valid

A valid specification is one in which all stakeholders and project roles (product owner, scrum master, tester, developer, etc) can understand, analyze, accept, or approve it.

5.4 Equivalence Class Testing

When most people, testers or not, take the Myers triangle challenge, they inevitably choose one isosceles, one scalene and one equilateral triangle as valid input; however no one ever chooses more than one of each as a valid input.

Equivalence class testing formalizes this intuition we have that "if it works for one equilateral triangle, it should work for all equilateral triangles."

But is that good enough? Or to completely test the program, are we stuck with writing every possible valid set of three integers and every invalid input, which means an infinite number of test cases.

The Fault Model

While the complexity problem is a more theoretical issue, the "just enough" problem is more of a pragmatic issue. In the real world of software testing, we are often constrained in our testing by shortages of time and resources. The problem is how to utilize the resources and time we have to do the most effective testing possible.

In order to do this, software testing employs what we call fault models which are a set of assumptions we make about where errors are likely to be made and, therefore, where we should concentrate our testing efforts.

All software testing is a trade off between completeness of testing and economy – the practical limits we have based on time and resources. A well thought out fault model allows us to get as through testing as possible within the constraints of our economics by selecting the optimal targets for testing.

Coverage

One of the terms that we will be using a lot in this section is coverage which means, in the context of functional testing, the degree to which we have covered off all possible types of inputs.

We know we can't test all possible inputs, so we test all possible types of inputs. Each type, as we shall see, is an equivalence class. Remember our Garden Path problem, by ensuring we have 100% coverage, i.e. we have tested for every possible type of input, we are assured we haven't let any invalid or unexpected cases slip through the testing process.

5.4.1 Equivalence Class Testing

Equivalence class testing, also called equivalence partitioning, is used to reduce the number of test cases to a workable number and still maintaining reasonable test coverage. Notice we say reasonable test coverage and not complete test coverage since we



may not be able to get complete coverage because of logistic or pragmatic reasons.

Equivalence partitioning is a method for grouping test cases by identifying various classes of input conditions with the assumption that each member of a class causes the same kind of processing and output to occur as any other member of the class.

In other words, all test cases which are members of the same equivalence class should be treated the same way by the system. If the system were to handle one case from an equivalence class erroneously, then it should handle all cases in that class erroneously.

This assumption about the system treating all test cases from an equivalence class the same may not always hold. However decades of testing has shown that it is a pretty good assumption that works most of the time.

Equivalence Class Example

We are testing a module for an insurance application that decides the level of risk for insurance coverage associated with an application based on a person's age. The insurance company's rules are:

0–16 years: No Insurance allowed.

16-21 years: Very high risk

21-28 years: High risk.

28-35 years: Moderate risk

35-60 years: Low risk

60-70 years: Moderate Risk

70-80 years: High Risk

80-90 years: Very high risk.

The assumption is that the program does not test against all possible ages. In other words we do not see code like:

```
if (age == 23) risk = "High";
if (age == 24) risk = "High";
... and so on
```

(Notice that the specification has problems. What are they?)

Instead we tend to see code like:

```
if (age >= 21 AND age <= 28) risk="High";
```

So the assumption that any age input from 21 to 28 should produce the same output is a reasonable assumption. Notice again we are making assumptions about what is reasonable and assuming that the programmer is a competent programmer. It is entirely

possible but very unlikely that some programmer did in fact write the code in the first code block above. Then our equivalence class technique does not work.

However, if we do have suspicions that this might be the case, we turn to our white box testing to validate or invalidate our equivalence class assumptions.

So then, instead of testing 0,1,2,3,4,5... we can choose one test case from each equivalence class. So we would get coverage with the following test cases:

0–16 years: No Insurance allowed.	Test case = 13
16-21 years: Very high risk	Test case = 18
21-28 years: High risk.	Test case = 23
28-35 years: Moderate risk	Test case = 30
35-60 years: Low risk	Test case = 49
60-70 years: Moderate Risk	Test case = 65
70-80 years: High Risk	Test case = 75
80-90 years: Very high risk.	Test case = 85

Guidelines for Identifying Equivalence Classes

First, whenever we start to develop equivalence classes, we should keep several basic guidelines in mind. If our resulting equivalence classes do not seem to meet these guidelines, then we have probably not done a good job of creating our equivalence partition.

A group of tests forms an equivalence class if

1. They all test the same thing.
2. If one test case in an equivalence class detects a defect, then every other test case in the same equivalence class will detect the same defect.
3. If one test case in an equivalence class fails to detect a defect, then every other test case in the same equivalence class will fail to detect the same defect.

Types of Data

At this point we take a brief aside to introduce several concepts about data that are useful for our discussion because we use different ways of identifying equivalence classes for these different data types.

When we talk about data, especially in terms of inputs and outputs, we identify three kinds of data:



1. **Categorical Data:** also called a nominal data, is used to for mutual exclusive and unordered categories. For example, gender {"m","f"} is a categorical data type. We could use "1" and "2" for names of the categories but that doesn't change the type of data, the order is arbitrary and any calculations (for example, computing an average gender in a group of people) would be meaningless.
2. **Ordinal Data:** is data that can be ordered, but the differences between the values are meaningless. For example, ranking how happy you are on a scale of 1 to 10, where a "4" is greater than a "2" but doesn't mean we can add a 2 and 4 to get a 6.
3. **Interval Data:** is like ordinal data in that it can be ranked but also the difference between two values is meaningful. The difference between a temperature of 100 degrees and 90 degrees is the same difference as between 10 degrees and 20 degrees.
4. **Ratio Data:** is the same as interval data with the additional constraint that there is a clear definition of 0 so that when a ratio variable equals 0, there is none of that type of data. Height and weight are examples of ratio data, temperature is not (because we still have temperature at zero degrees).

One way to think of the differences between these types is to compare what sort of things we can do with each data type.

Operation	Categorical	Ordinal	Interval	Ratio
Frequency distribution	Yes	Yes	Yes	Yes
Percentiles and Medians	No	Yes	Yes	Yes
Add and Subtract	No	No	Yes	Yes
Mean and std. dev.	No	No	Yes	Yes
Coeff of variation	No	No	No	Yes

Equivalence Classes for Categorical and Ordinal Data

Generally these sorts of data are presented as lists. For example:

- A list of state postal abbreviations.
- A list of political parties
- Citizenship or country of residence.
- Age category (ie under 16, 16-20, 21-25, etc)

Step One: Grouping the data for valid equivalence classes

This can only be done by looking at the specification. What we want to do is group the values into categories that according to the spec would be processed in the same way or using the same logic. Each such group is a valid equivalence class

For example, lets take the following set of possible inputs to an application representing postal abbreviations for an online retail application:

{AL, AK, AB, CA, MN, TX, ON, AS, PR, VI}

By looking at the specification, we see the following. “Orders originating in Canada must have GST calculated as part of the price, orders originating in US possessions outside the continental US are subject to an additional handling charge. No modifications are done for orders originating in the continental US.”

This means we have three different sets of processing we have to test and this yields three equivalence classes:

{AL, AK, CA, MN, TX}	(US states)
{AB, ON}	(Canadian Provinces)
{AS, PR, VI}	(US Possessions)

However suppose the specification had read “All orders, no matter where they originate are processed the same way.”

In this case we have one equivalence class – the whole list itself.

Step Two: Defining the Invalid Class.

Once we know what the valid classes are, our invalid class is anything that is not in any of the valid classes but yet is drawn from the same domain. This often takes a little art to define this because we have to be careful not just to look for anything that is not on the list.

Consider our example above with the three valid classes, what would be the invalid class?

- Any string not on the list? Eg. “Harp”, “JJUA” “887s”
- And two letter code not on the list? Eg. “VV”, “XX”
- Any actual postal code not on the list? “OH”, “MB”

Intuitively it would seem that the last choice would be best, but in fact we want to test all of the different ways an input can be invalid . Suppose, for example, the programmer decided that they would just test the input to see if it was a string two characters long, then their code might accept “XX” as a valid input. On the other hand, if they just checked the first to letters and not the whole string, their code might accept MIL as a valid postal code, treating it as “MI”.



Basically we have to check to see that the developer has tested for all the possible ways an input can be invalid.

Now our equivalence classes are:

Valid {AL, AK, CA, MN, TX} (US states)
 Valid {AB, ON} (Canadian Provinces)
 Valid {AS, PR, VI} (US Possessions)
 Invalid {Any legal postal code not in the above lists}
 Invalid {Any two letter code that is not a postal code}
 Invalid {And string that is not two characters long}
 Invalid {Data that is non-alphabetic}

Equivalence Classes for Interval and Ratio Data.

When the data is of this type, the valid class expressed as a range or an inequality. For example, consider the balance of a bank account subject to the following description in the specification “According to FDIC regulations, the largest allowable balance for an account is \$60,000. Accounts may be overdrawn to the amount of \$5,000.”

Step One: Determine the Equivalence Class Intervals.

Whenever an interval is specified, there are always three equivalence class intervals.

Valid Interval: -\$5,000 to \$60,000
 Invalid too big: > \$60,000
 Invalid too small: < -\$5,000

We always look for the too small and too large cases since the programmer may have not accounted for both, for example she may have coded

```
if ((Balance > 60000) || (Balance < -6000)) return INVALID_BALANCE;
```

If we had just tested for a value that was too big, say \$75,000, then we would not have picked up the error.

Step Two: Look for Further Valid Classes

Using the same process as before, we look in specification at sub-ranges within the valid class that are processed differently. For example, in our bank example, we might also have additional processing that done on accounts that are overdrawn: overdraft accounts are processed differently than on overdraft accounts.

In this case we have the following set of equivalence classes.

Valid Interval:	0 to \$60,000
Valid Interval: -	5,000 to -1
Invalid too big:	> \$60,000
Invalid too small:	< -\$5,000

Algorithm for Identifying Equivalence Classes.

The following is a systematic a repeatable way to identify equivalence classes. The advantages to using an approach like this are:

1. It is exhaustive, we identify all the equivalence classes.
2. It is efficient, we don't have to look for the classes, they can be quickly computed.
3. It is replicable, which means that we can recheck our work for errors or have others duplicate what we have done.

Step One: Specification Analysis

Read through the specification and list all input variable and their data types.

Step Two: List Criteria

For each variable, list all criteria that must be present for a valid input. This is our first cut at a list of valid classes. Note the data type of each criteria (Note! This may be different than the type of the input!!)

Step Three: Identify Valid Classes

Based on the specification, subdivide the valid classes based on any described differences in processing.

Step Four: Identify Invalid Classes

For each criteria in step two, create an invalid class by negating each criteria in turn.

A technique that many testers use to develop invalid test cases is to take the specifications and invalidate exactly one clause of a specification to get an invalid case.

Example. Our program should accept applicants who are over 18, resident in the US and hold a valid driver's license. Be invalidating each clause in turn, we get three invalid cases and one valid case.

1. Over 18, resident in the US and not holding a valid drivers license.
2. Over 18, holding a valid drivers license and not resident in the US.



3. Holding a valid drivers license, resident in the US and not over 18.
4. (Valid case) Over 18, resident in the US and holding a valid drivers license.

We must always be careful not combine invalid cases. Notice that this gives us 100% coverage since either the input is valid, which means it satisfies all three criteria, or it fails at least one of the criteria is it is invalid. There are no other possibilities.

5.5 Boundary Value Analysis

Boundary value analysis is a way of designing test cases that concentrates testing efforts on cases near the limits of the valid ranges.

Boundary value testing focuses on the boundaries simply because that is where so many defects hide. Experienced testers have encountered this situation many times. Inexperienced testers may have an intuitive feel that mistakes will occur most often at the boundaries (ah-hah, our fault model!)

Lets go back and take a look a the insurance specification we looked at in the last section.

0–16 years: No Insurance allowed.

16-21 years: Very high risk

21-28 years: High risk.

28-35 years: Moderate risk

35-60 years: Low risk

60-70 years: Moderate Risk

70-80 years: High Risk

80-90 years: Very high risk.

Our boundaries are wonky (technical term). What if someone is 21? Are they very high risk or just high risk? The specification is ambiguous, and if this specification made into code, we are not really sure how the programmer interpreted the specification. We will see later why the neither the tester or programmer should never never never rewrite a specification. Let's get our designers to rewrite the specification so that it is no longer ambiguous.

0–16 years: No Insurance allowed.

17-21 years: Very high risk

22-28 years: High risk.

29-35 years: Moderate risk

36-60 years: Low risk

61-70 years: Moderate Risk

71-80 years: High Risk

81-90 years: Very high risk.



Why do we want to look at the boundaries? Consider how the programmer might have coded this specification:

```
if (age <= 16) risk="not allowed";
if (age >= 17 AND age <= 21) risk="Very High";
if (age >= 22 AND age <= 28) risk="High";
if (age > 28 AND age < 35) risk="Moderate";
if (age > 35 AND age < 61) risk="Low";
if (age > 61 AND age < 71) risk="Moderate";
if (age > 70 AND age < 81) risk="High";
if (age > 82) risk ="Very High";
```

Our programmer has made a couple of interesting errors at the boundaries. For example:

- What happens when the age is outside the ranges identified in the specification?
- What happens to a 61 year old?

This little contrived example shows why our fault model is justified – programmers tend to make errors on the boundaries of things.

5.5.1 Guidelines for Selecting Boundary Values

The key strategy for selecting boundary values is quite simple.

Create test cases for each boundary value by choosing one point on the boundary, one point just below the boundary, and one point just above the boundary.

The equivalence classes for the inputs are determined as before by using the input specifications to perform the equivalence class partitioning.

Boundary Values for Interval and Ratio Data

The first step is to identify the boundary values. As can be seen from the examples we have looked at, we need to know exactly which value is invalid. For example in our back problem when we said that deposits were up to \$60,00 dollars, did that mean including or excluding the endpoint of \$60,000.

The next step is is to choose for each class:

1. The largest and smallest values if the class is valid.
2. The smallest value that is invalid for the “too big” invalid class
3. The largest value that is invalid for the “too small” invalid class.

Boundary Values for Ordinal Data

Ordinal values are essentially lists that are ordered. So our strategy is to use the following values:

1. The valid case: the first item or smallest item on the list.
2. The valid case: the last or largest item on the list
3. The invalid case: the item before the first item on the list.
4. The invalid case: the item after the last item on the list.
5. The invalid case: access an item from the empty list.



5.6 Cause and Effect Graphing

So far, we have been working with the unstated assumption that each output is the result of an input. This is a reasonable assumption, but it leaves a gap in testing. What if two different inputs have to be combined in a way to produce one specific output? We can't test to see if the output is correct if we think just in terms of using one input.

One of the reasons is that equivalence class partitioning does not allow testers to combine input conditions, and in some cases an output result can only occur when several input conditions co-occur. However, we also cannot start looking at every possible combination of inputs since that produces an unworkable number of test cases.

What we want to do is work backwards and identify equivalence classes of outputs and then determine which combinations of inputs would be required to produce that output. For example, because of a coding error (or perhaps a nefarious design) an airline might offer a bargain on an airfare, but there might not be any combination of dates and destinations that would every qualify for the bargain.

One technique that we use to address this testing situation is called a cause and effect graph.

The reason we call these cause and effect graphs is that each output class can be seen as an effect of a combination of causes or input classes. A single cause may be necessary for many effects and a single effect may require many causes.

The cause and effect graph is a pictorial way of representing logical relationships. While the cause and effect graph has been a staple of software testers for decades, it is nowadays considered at best "quaint" and at worst inadequate and obsolete. One practical drawback is that developing the graph, especially for a complex module with many combinations of inputs, is difficult and time consuming. However it is still a great learning tool for beginning testers before getting into more advanced concepts like Karanaugh-Veitch matrices. Relax, we are not going to get that advanced in this course where we are actually using techniques named after mathematicians.

Cause-and-effect graphing transforms a specification into a graph that resembles a digital logic circuit. To effectively use the cause and effect graph, the tester should have knowledge of Boolean logic.

5.6.1 Creating a Cause and Effect Graph

1. First we assume we have done our equivalence partitioning as described in a previous section.
2. For each input equivalence class, we create a "cause" node on our graph. Cause nodes traditionally are numbered starting at 1. A node is just a fancy name for a circle in a diagram. Cause nodes are usually drawn on the left side of the graph.

3. For each output, an “effect” node is created on the graph. Effect nodes are traditionally numbered starting at a higher number, like 100 or 1000, to distinguish them from cause nodes. Effect nodes are usually drawn on the right side of the graph.
4. From the specification, logical relationships between the causes and effects are drawn with edges (fancy name for lines) that connect the causes and effects. Logical combinations of causes are represented using logical operators AND, OR and NOT on the edges.
5. Intermediate results (logical combinations of cause nodes) may, for clarity, be represented as nodes in the center of the graph.

That probably was pretty opaque, so to put it all into focus, let's work through an example of drawing a cause and effect graph.

Cause and Effect Example

First we need to consider the specification. Consider the following insurance company example adapted from Robert Binder who adapted it from Weinberg. Consider the following table used as a specification for a program we are to test.

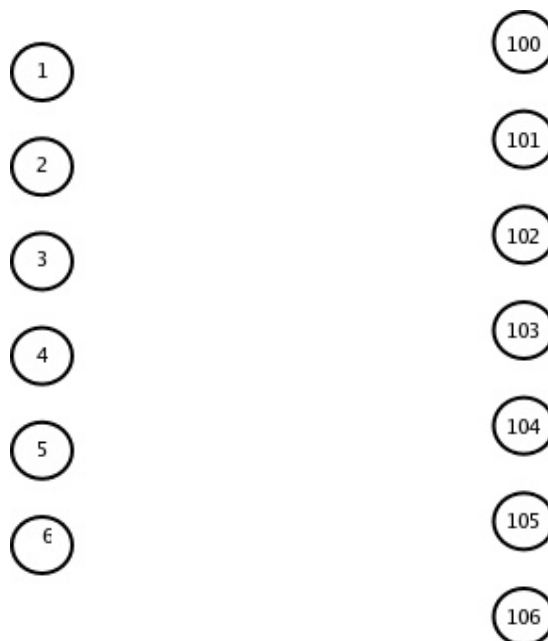
<i>Number of Claims</i>	<i>Age</i>	<i>Premium Increase</i>	<i>Send Warning</i>	<i>Cancel Coverage</i>
0	<= 25	\$50.00	No	No
0	>25	\$25.00	No	No
1	<= 25	\$100.00	Yes	No
1	> 25	\$50.00	No	No
2 to 4	<= 25	\$400.00	Yes	No
2 to 4	> 25	\$200.00	Yes	No
5 or more	Any	0	No	Yes



First we identify our causes and effects.

Node	Cause	Effect
1	≤ 25	
2	> 25	
3	0 claims	
4	1 claim	
5	2 to 4 claims	
6	5 or more claims	
100		\$25 inc
101		\$50 inc
102		\$100 inc
103		\$200 inc
104		\$400 inc
105		Warning
106		Cancel

Now we prepare our graph.



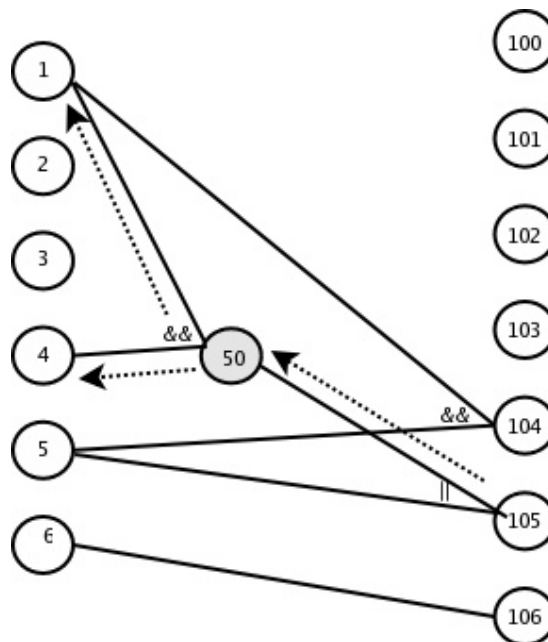
The next step is to connect up the causes and effects. We do this by drawing lines back from an effect to one of more cause using the symbols that look like a “v” and upside down v for OR and AND. However since I do not seem to have them on this word processor, I will use && and || for AND and OR and use ~ for NOT.

Lets pick the effect 106 “Cancel Coverage.” Looking at our table on the previous page, we can see that this effect has only one cause 6 “5 or more claims”. So we draw a line from 6 to 106.

Now consider the effect 104 “\$400” increase”. This effect is the result of two causes that must happen together, 1 “<=25” and 5 “2 to four claims”, so we draw two lines from 1 and 4 respectively and mark them as being “ANDed”.

Consider a more complex effect, 105 “Send Warning”. According to our table this will happen if cause 5 “2 to 4 claims” occurs OR causes 1 “<=25” AND 4 “1 claim” occur together. In logical notation $5 || (1 \&\& 4)$. Once causes get complex like this, the cause and effect graph can get quite messy. So one way to keep it clear is to put an intermediate node representing $(1 \&\& 4)$ in the middle – we have labeled it 50 in our graph.

The partially complete graph is seen below:



5.7 Creating Decision Tables

Decision tables are a standard technique used in a variety of disciplines and well suited to software testing. In fact, you have already used a decision table in the previous exercise to construct a cause and effect graph. This is a bit backwards since cause and effect graphs are often used to help sort through the specification to create decision tables.

It is considered standard practice to use decision tables rather than cause and effect graphs in testing, partly because the more mathematical nature of the tables allows them to more easily automated than a cause and effect graph.

Decision tables are also the starting point form developing more advanced black box testing techniques like developing logic functions and others that we will not touch on in this introductory course.

A decision table has the following format.

		1	2	3	4	5
causes	1					
	2					
	3					
	4					
effects	100					
	101					
	102					
	103					

Explanation

Each numbered cause corresponds to a node in the cause-effect graph. Each vertical column represents a test case. Each cell contains a 1 for true or a 0 for false. A 0 in a cause cell means that the condition associated with the cause is false. A 1 in the cell means it's true. A 0 in an effect cell means that the effect does not take place, while a 1 means it does.

Taken together, the cells in a column describe what effects occur for a specific combinations of input conditions or causes.

In a non-limited table, all possible combinations of causes leading to each effect occurring are shown.

Steps

1. First, examine the case where one of the terminal nodes - an effect - is 1, meaning the output condition occurs.
2. Trace backwards through the graph to identify all combinations of causes that would produce that effect. Keeping tracing back through each edge until you have reached the causes on the left hand side of the graph.
3. Create a column in your decision table for each test case you have identified that shows: the state of each cause in the test case and; a 1 in the cell corresponding to the effect you have been studying.
4. For each column, fill in the states of the other effects. It may happen that the selection of causes you use in the test case not only produces the effect that you want, but also produces other effects as well.
5. Repeat steps 1-4 for each effect

Decision Table Example

A decision table for the insurance example we did last section would look like this:

The effects we wanted to trigger are diagonally indicated by 1s in the effect section. However, notice that there are other effects that also occur as the result of the causes we chose in our test case.

		1	2	3	4	5	6
causes	1	0	1	0	1	1	0
	2	1	0	1	0	0	0
	3	1	1	0	0	0	0
	4	0	0	1	0	1	0
	5	0	0	0	1	0	0
	6	0	0	0	0	0	1
effects	100	1	0	0	0	0	0
	101	0	1	0	0	0	0
	102	0	0	1	0	1	0
	103	0	0	0	0	0	0
	104	0	0	0	1	0	0
	105	0	0	1	1	1	0
	106	0	0	0	0	0	1



5.6.2 Limited Entry Decision Tables

Because decision tables can become very large very fast, there has been considerable work done on how to limit the size of the table while still keeping valuable cases. Tables reduced in size by the application of robust rules are called limited entry tables and are useful in designing a reasonably sized yet effective set of test cases

In the previous section, we derived test cases by assuming an effect. This means that we started at the appropriate node on the graph and traced backwards to determine combinations of causes which would produce that effect.

The idea of a limited entry decision table is that as we trace back a set of rules can be applied to reduce the number of paths that have to be traversed back to a set of causes with running the risk of having the effect “not occur”.

The rules are as follows:

1. If you trace back over an AND, then all of the edges coming into that AND will have to be true. That means you will have to trace back over all edges that
2. If you trace back over an OR, you only have to trace back only one of the edges that lead into the OR since only one of them has to be true.
3. If you trace back over an intermediate result that needs to be false and is an AND, only one of the edges leading to that result has to be traced back to a cause, but that cause must be false for the effect to occur.

